

Efficient Hardware Scheduling approach on FPGA's for optimising DSP based workflows

Connor Sinclair

This thesis is presented as required for the conferral of the degree:

Bachelor of Software Engineering (Hons.)

Supervisor:
Dr. Philip Leong

The University of Sydney
School of Engineering

2025

Declaration

I, *Connor Sinclair*, declare that this thesis is submitted in fulfilment of the requirements for the conferral of the degree *Bachelor of Software Engineering (Hons.)*, from the University of Sydney, is wholly my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualifications at any other academic institution.

Connor Sinclair

November 8, 2025

Abstract

FPGA toolchains typically perform static compilation and mapping of fixed designs. In practice, audio signals and user-defined module graphs vary over time, presenting an opportunity to predictively reconfigure the FPGA fabric. This thesis proposes analytic, operator theoretic, DSP and CSP frameworks that model wave-synthesis *Kernels* as dynamical systems, using observables and new signal correlation metrics to forecast workload and adjust resource allocation. Unlike conventional flows, this approach can anticipate the evolving demands of a modular audio effect chain (including non-linear modules) and remap kernels (e.g. heavy multipliers or trigonometric units) onto LUTs, DSP slices, or BRAMs in accordance with resource demands. Expected contributions include: (1) A novel atomic framework for analysis of complex DSP systems, portable to FPGA's; (2) an analytical model of audio DSP modules grounded in operator theory, cyclostationary analysis and state-space representations; (3) coherent metrics of signal interaction for resource tuning; and (4) a prototype architecture enabling runtime FPGA optimization in an accessible form. This work bridges a gap between the modeling of DSP systems, particularly in a wave synthesis context, and FPGA architecture: offering audio scientists, embedded developers and researchers a more adaptive FPGA DSP platform that can improve throughput, support complex nonlinear & stochastic audio effects and enable massive real-time parallelisation all without need for manual tuning or inspection.

Acknowledgements

Contents

Abstract	iii
Acknowledgements	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	2
1.2 Aims	3
1.3 Contributions	4
1.4 Thesis Structure	4
2 Literature Review	6
2.1 Techniques for Workload Forecasting and Adaptive Control	6
2.1.1 Signal Analysis and Correlation for Workload Characterization	6
2.1.2 Adaptive Algorithms and Control Strategies	8
2.2 Cyclostationary and Control Theoretic Approaches to Signal Analysis	9
2.3 Mathematical Foundations for design derivation	10
2.3.1 Fixed-Point Quantization and Bit Allocation	10
2.3.2 Wavelet Theory and Multi-Resolution Analysis	10
2.4 Existing Hardware Techniques and Designs	11
2.4.1 FPGA Synthesis and CORDIC Algorithms	11
2.4.2 Rate-Distortion Theory and Water-Filling	11
2.4.3 Bilinear Transform and Analog Filter Discretization	12
2.4.4 Dynamic Partial Reconfiguration for Adaptive Systems	12
3 Derivation of novel sensitivity-based Wavelet for synthesis	14
3.1 Optimal Coefficient Quantization via Projection Methods	15
3.1.1 Chapter Objectives	15
3.1.2 Scope and Limitations	16
3.1.3 Problem Formulation	16
3.1.4 Projection Operator and Error Metric	17
3.1.5 Sensitivity Matrix Construction	18
3.1.6 Quantization Error Bounds	19
3.1.7 Optimal Bit Allocation Strategy	20

3.1.8	Evaluation of Projected Error	21
3.1.9	Projector Derivation for Sinc Function	23
3.2	Extending definition of the projector to accomodate generic $H(\omega)$. .	30
3.2.1	Deriving the support kernel $H(\hat{\omega})$	31
3.2.2	Dyadic Quantization of Projector Coefficients	35
3.3	Shannon Wavelet Basis and Orthonormal Extension	37
3.3.1	Shannon Wavelet Construction	38
3.3.2	Orthonormality and Parseval Identity	39
3.3.3	Projector Extension to the Wavelet Basis	39
3.3.4	Approximation of Arbitrary Bandlimited Functions	40
3.3.5	Error Analysis in the Wavelet Domain with Quantization . . .	41
3.3.6	Sensitivity Matrix for the Wavelet Basis with Quantization . .	41
3.3.7	Computational Advantages for Hardware Implementation . . .	42
3.3.8	Hardware-Efficient Kernel Selection with Quantization	43
4	Control theoretic model for dynamic FPGA resource balancing	45
4.1	Cyclostationary analysis	45
4.1.1	Relation between Strict Periodicity and Cyclostationarity . . .	47
4.1.2	Group-Theoretic Interpretation	47
4.1.3	Relationship between Argument Reduction and Cyclostationarity	48
4.2	Koopman Operator Framework for DSP Resource Prediction	52
4.2.1	Motivation: From Signal Analysis to Resource Prediction . . .	52
4.2.2	Koopman Operator Theory	53
4.2.3	Connection to Shannon Wavelet Decomposition	53
4.2.4	Finite-Dimensional Approximation via Extended DMD	55
4.2.5	Resource Prediction via Koopman-MPC Integration	56
4.2.6	Implementation: From Cyclostationary Analysis to Koopman Modes	58
5	Architectural Implementation and Hardware Optimization	60
5.1	Memory-Efficient Lookup via Range Reduction	60
5.1.1	Algorithmic Framework for Reducing Memory pressure	61
5.2	Polynomial Approximation and Hardware-Efficient Evaluation	62
5.2.1	Case Study: $\arccos(x)$ via Quarter-Domain Range Reduction . .	62
5.2.2	Generalized Methodology for Polynomial Recovery Functions . .	66
5.2.3	Automatic Parallelization via DAG Topology Analysis	68
5.3	Hardware-Optimized Computation	70
5.3.1	Rudimentary hardware design	70
5.4	Implementation Results	73
6	Conclusion	76
6.1	Summary of Contributions	76
6.2	Integration and Impact	76
6.3	Limitations and Future Directions	77
6.4	Concluding Remarks	77

A	Appendix	78
A.1	Auxiliary Lemmas	78
A.2	Design of the general support kernel $\hat{H}(\omega)$	81
A.2.1	Example of shaping kernel using standard design method: digital First-Order Low-Pass Filter	81
A.2.2	Special decomposition cases for $\hat{H}_a(\omega)$	81
A.3	Shannon Wavelet Derivations	83
A.3.1	Frequency Response Contribution $\Phi_n(\omega)$	83
A.3.2	Greedy Bit Allocation with Truncation-Quantization Trade-off	84
A.3.3	Residue-Based Decomposition for Hardware Kernels	85
A.3.4	Signal Synthesis Capabilities of Shannon Wavelet Decomposition	85
A.4	Hardware design choices	90
A.4.1	Kernel design	90
A.4.2	Fabric design	90
A.4.3	Synthesis Quality Metrics	90
A.4.4	Hardware Resource Estimates for Shannon Wavelet Implementation	91
A.5	Code snippets or heuristics	92
A.5.1	'compact' sinc implementation	92
A.5.2	comparison of cos power series in c	92
A.5.3	introducing concurrency into power series	93

List of Figures

3.1	Sinc remainder plot	29
3.2	Truncation error	30
3.3	Graph of the projector for $H_a(\omega) = e^{a\omega}$	34
3.4	Graph of the projector for $H_a(\omega) = e^{a\omega}$	35
5.1	Rudimentary architecture for configurable BRAM slices	72
5.2	Rudimentary architecture for semi-pipelined configurable DSP slices .	73

List of Tables

3.1	Comparison of bit allocation methods.	21
4.1	Sensitivity-Driven Implementation Mode Selection	57
5.1	FPGA Resource Utilization for $\arccos(x)$ Quarter-Domain Range Reduction	73
5.2	Comparison of $\arccos(x)$ Implementation Strategies on FPGA	74
5.3	Advantages and Limitations of Polynomial Range Reduction for $\arccos(x)$	75
A.1	Comparison of signal synthesis methods for bandlimited signals.	89

Chapter 1

Introduction

FPGAs are increasingly used in audio signal processing products and research because of their high computational density and low-latency capabilities[**IEEE8187326**]. They appear in digital mixers, audio interfaces, synthesizers and custom audio processing units. Modern FPGA architectures include specialized DSP slices and block-RAMs to accelerate multiply-accumulate and filtering operations. In principle, this makes FPGAs very attractive for high-throughput audio applications (e.g. sampling rates in the MHz range, multi-channel processing, wavelet synthesis).

Despite these advantages, programming FPGAs remains challenging. Unlike CPU or GPU development, FPGA design requires intimate knowledge of hardware design (HDLs, pipelining, clock-domain crossing, design at a boolean / bitwise level Etc.). As one recent survey notes, "FPGAs are considered as 'hardware' and are low-level in nature, making them hard to program by non-specialized engineers" [**IEEE8187326**]. Audio-specific toolchains (such as compiling FAUST or C++ audio code into FPGA via HLS) help bridge the gaps, but they still produce static designs that must be manually tuned. In short, many working in the DSP field find FPGA development inaccessible, or end up using FPGAs in a fixed manner akin to a DSP chip, underutilizing the relative versatility and performancy of FPGA's.

Most FPGA workflows today perform *one-shot mapping*: the designer specifies an audio processing chain (filters, modulators, etc.), a compiler schedules these operations across clock cycles and resources, and the FPGA is programmed with that static configuration. Prior research has optimized such flows in various ways[**SinhaSrikanthan2014**]. For example, specialized scheduling algorithms account for FPGA primitives to minimize latency or area, and some works perform resource-aware scheduling of arithmetic graphs on DSP blocks. Others have explored partitioning of tasks in multi-context or partially reconfigurable FPGAs to swap in new hardware functions between audio processing tasks[**Bhandari2012**]. Indeed, run-time reconfiguration (sometimes called Partial reconfiguration or DFX - Dynamic Function Exchange) can increase effective device density and support adaptivity. For instance, dynamic partial reconfiguration allows new hardware modules to be loaded without stopping the rest of the design. In audio-specific cases, researchers have demonstrated an FPGA-based system where an audio filter is replaced on-the-fly to save resources[**Silva2016**], and another where reconfigurable DSP blocks let a complex

audio player time-multiplex hardware for an FFT and filters[Das2017].

Still, these adaptive systems *are reactive, not predictive*. They typically change hardware only when commanded or scheduled by a user or fixed routine, not in response to analysis of the audio signal stream. In practice, an audio signal may have varying frequency content or complexity over time, and different modules may be active only in some parts of a performance. A truly dynamic FPGA design would detect or predict which DSP operations will dominate in the near future and pre-configure the fabric accordingly. No known FPGA tool currently integrates time-series analysis of audio signals into the mapping process.

A promising avenue is to treat DSP modules as dynamical systems and apply operator-theoretic??, cyclostationary?? and analytic?? modeling to them. The Koopman operator, for example, represents a nonlinear dynamical system as a (generally infinite-dimensional) linear operator on observable functions of the state. Recent work in audio and control has used Koopman-based models for system identification and predictive modeling of nonlinear effects. These methods enable linear prediction of future behavior from past observations. Yet, this line of theory has not been applied to FPGA hardware optimization.

Finally, even when powerful techniques are known academically, accessible tools are lacking. Audio-focused FPGA toolchains like Syfala[Syfala2024] (compiling FAUST DSP to FPGA) show that domain-specific languages can make FPGA synthesis approachable. However, these flows remain largely offline; they do not optimize the FPGA mapping beyond what generic HLS yields. The complexity of typical FPGA environments (Vivado, Quartus) means that most hobbyists fall back to microcontrollers or leave FPGAs underutilized.

In summary, audio DSP on FPGAs suffers from a lack of dynamic, signal-aware optimization, a lack of high-level modeling approaches for workload prediction, and a gap in user-friendly FPGA toolchains. This project addresses these unmet needs by combining operator-theoretic models with signal-correlation metrics and a rudimentary hardware design to inform a dynamic FPGA mapping strategy. We aim to enable high-throughput, scalable FPGA implementations of modular audio systems (even including nonlinear effects) in a way that is transparent and accessible to embedded engineers.

1.1 Motivation

Despite the promise of FPGAs for audio DSP, several critical gaps remain in the literature and tooling:

Lack of dynamic/predictive optimization: Existing FPGA design flows assume fixed workloads and static mapping. While partial and dynamic reconfiguration has been demonstrated (e.g. swapping audio filter modules at run-time) these approaches are reactive and not driven by analysis of the signal. No known solution continuously monitors signal behavior and predictively reassigns logic resources (LUTs vs DSP blocks, etc.) to maximize throughput or efficiency.

Absence of operator-theoretic DSP mapping: Modern control theory offers linear operator models (e.g. Koopman operators) for nonlinear dynamics. However, this perspective is essentially unused in FPGA design. DSP modules could be viewed as dynamical systems in state-space, enabling prediction of future workload, but current FPGA mapping literature focuses on static scheduling and technology mapping rather than on system-theoretic modeling of signal flow.

Scarcity of accessible toolchains: Hardware design is considered “low-level” and notoriously difficult for DSP software developers to break into. A few domain-specific tools (e.g. FAUST-to-FPGA via Syfala) aim to ease audio-on-FPGA programming, but these still require expert tuning and do not offer automated performance optimization. Hobbyists and embedded engineers lack a user-friendly platform that lets them upload modular DSP graphs and automatically optimizes the FPGA mapping. These gaps motivate a new solution: a predictive FPGA optimization framework that combines signal dynamics (via operator models) with hardware-aware scheduling. The project addresses the unmet need for adaptive FPGA systems in modular audio DSP.

1.2 Aims

The primary aim of this thesis is to *enable adaptive FPGA-based audio signal processing* through a mathematically rigorous framework that couples operator-theoretic signal analysis with runtime hardware resource optimization. Specifically, we pursue three interconnected objectives:

1. **Develop an operator-theoretic model for audio DSP modules.** Formulate a Koopman operator-based representation of modular signal chains (filters, nonlinear processors, dynamic effects) that captures both linear and nonlinear dynamics, cyclostationary signal characteristics, and inter-module dependencies. This model must be analytically tractable and portable to FPGA hardware constraints.
2. **Derive closed-form sensitivity metrics for resource-error trade-offs.** Establish a projection-based mathematical framework that quantifies how bit allocation, coefficient quantization, and truncation affect output fidelity across frequency bands. Extend this to wavelet decompositions for multi-rate processing, providing explicit formulae that guide runtime decisions on DSP slice, LUT, and BRAM usage.
3. **Design and validate a prototype architecture for dynamic reconfiguration.** Synthesize the theoretical results into a hardware architecture that performs workload forecasting, signal interaction analysis, and predictive resource reallocation without manual intervention. Demonstrate feasibility through RTL implementation and comparative benchmarking against static compilation flows.

These aims address the gaps identified in Section ??: the absence of predictive optimization frameworks, the lack of operator-theoretic DSP mappings to FPGAs, and the scarcity of accessible toolchains for researchers and audio engineers.

1.3 Contributions

This thesis makes the following novel contributions to the fields of FPGA signal processing, adaptive hardware architecture, and operator-theoretic systems analysis:

1. **A unified framework for describing DSP kernels.** We derive closed-form sensitivity matrices that relate coefficient quantization to output error for polynomial approximations of infinite-support kernels (Chapter 3). By combining complex analytic techniques with dyadic quantization for fixed-point hardware, we provide an analytical tool that directly connects signal processing requirements to FPGA resource specifications. The framework extends naturally to Shannon wavelet decompositions, enabling independent bit allocation across frequency bands based on signal characteristics.
2. **An operator-theoretic model of modular audio signal chains.** Building on Koopman operator theory (Chapter 4), we formulate a state-space representation of audio DSP modules that captures both linear filtering and nonlinear transformations in a unified framework. The model incorporates cyclostationary analysis to handle time-varying signal statistics and provides explicit metrics for inter-module signal correlation. This enables runtime prediction of resource demands based on workload evolution.
3. **Novel signal interaction metrics for resource tuning.** We introduce computable metrics that quantify how signal characteristics (spectral content, dynamic range, correlation structure) influence hardware resource efficiency. These metrics are derived from the operator-theoretic model and provide actionable guidance for adaptive bit allocation, primitive reallocation (DSP slices vs. LUTs), and parallelization strategies (Chapter 5).
4. **A prototype architecture for runtime FPGA optimization.** We present a complete hardware implementation that integrates the mathematical framework with dynamic reconfiguration capabilities. The architecture includes: (i) workload forecasting based on cyclostationary signal analysis, (ii) a CSP/DSP-based caching hierarchy for efficient LUT storage, (iii) adaptive filter bank decomposition using the Shannon wavelet basis, and (iv) runtime primitive reallocation without manual tuning. Verification results demonstrate improved throughput and resource efficiency compared to static compilation flows.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2: Literature Review surveys the foundational work in signal processing, adaptive control, and operator-theoretic systems analysis. We review classical DSP techniques (Fourier analysis, filter design, wavelet theory), operator theory (Koopman operators, state-space methods), and FPGA implementation strategies (CORDIC algorithms, fixed-point arithmetic). Section 2.4 provides targeted mathematical background for the derivations in Chapter 4, covering projection methods, contour integration, dyadic quantization, and rate-distortion theory.

Chapter 3: Design Derivation develops the mathematical/theoretical foundation for adaptive resource allocation. Section 4.1 establishes the projection-based framework for polynomial approximations of the sinc function in the time domain, deriving sensitivity matrices for bit allocation. Section 4.2 extends this to generic filter kernels via complex analytic methods. Section 4.3 introduces dyadic quantization for fixed-point hardware compatibility. Section 4.4 generalizes the framework to the Shannon wavelet basis, enabling multi-band decomposition with per-scale bit allocation. The chapter concludes by synthesizing these results into design guidelines for adaptive signal processing hardware.

Chapter 4: Control-Theoretic and Cyclostationary Model for Resource Balancing formulates a dynamic systems perspective on FPGA resource balancing. We develop a cyclostationary signal analysis framework to characterize time-varying workloads and derive spectral correlation metrics for predicting resource demands. The Koopman operator formalism is employed to model both linear and nonlinear audio DSP modules within a unified state-space representation, enabling runtime optimization of hardware primitives based on signal evolution.

Chapter 5: Architectural Implementation presents the hardware realization of the theoretical framework. We introduce a CSP/DSP-based approach to creating 'cache hierarchies' for variable LUT storage and methods for selectively targeting FPGA resources via different evaluation techniques applied to power series. Finally, the chapter details the synthesis of the specialized wave-basis (Chapter 3) and operator-theoretic workload prediction (Chapter 4) into a unified FPGA architecture. RTL implementations, resource utilization analysis, and verification strategies are provided.

Appendices provide supplementary mathematical proofs, detailed derivations omitted from the main text for brevity, and implementation-specific details (RTL code listings, synthesis results, experimental data). Appendix A collects proofs for the projection theorems and sensitivity bounds in Chapter 4. Appendix B documents the verification environment and testbench design.

Throughout the thesis, we maintain a clear distinction between the *analytical framework* (mathematical models, optimization criteria) and the *implementation constraints* (FPGA primitives, fixed-point arithmetic, reconfiguration latency). This separation allows the theoretical results to remain portable across hardware platforms while ensuring practical feasibility of the proposed architecture.

Chapter 2

Literature Review

2.1 Techniques for Workload Forecasting and Adaptive Control

Effective predictive reconfiguration relies on two key capabilities: first, the ability to analyze incoming audio signals and the state of processing modules to extract features that are predictive of future computational workload; and second, the algorithmic basis for using these predictions to adapt the system’s behavior, in this case, the FPGA’s hardware configuration.

2.1.1 Signal Analysis and Correlation for Workload Characterization

To forecast workload, the system must derive “coherent metrics of signal interaction” [ref1]. This involves applying signal processing techniques to extract relevant information from the audio data stream.

Spectral and Cross-Spectral Analysis

Classical spectral estimation techniques, such as the periodogram or Welch’s method, provide insights into the frequency content of audio signals. These can reveal the distribution of energy across different frequency bands, which may correlate with the activation or intensity of certain frequency-dependent DSP modules (e.g., equalizers, specific filter banks). Cross-spectral density analysis can further elucidate the relationship between two signals, for instance, the input and output of a module, to understand how the module is transforming the signal’s spectrum, or to detect correlations between different audio channels that might imply a need for shared or interacting processing resources.

For signals exhibiting periodic statistical properties, cyclostationary analysis offers a more powerful tool. The Spectral Correlation Density (SCD), also known as the cyclic spectral density, characterizes the correlation between frequency-shifted

versions of a signal [ref1]. The SCD function is defined as:

$$S_x^\alpha(f) = \lim_{T \rightarrow \infty} \lim_{\Delta t \rightarrow \infty} \frac{1}{\Delta t} \int_{-\Delta t/2}^{\Delta t/2} \frac{1}{T} X_T(t + \zeta, f + \alpha/2) X_T^*(t + \zeta, f - \alpha/2) d\zeta$$

where $X_T(t, f)$ is the complex demodulate of the signal $x(t)$ at frequency f over a window T , and α is the cycle frequency [ref1]. If the activation of certain audio effects (e.g., amplitude modulation, periodic filtering) introduces cyclostationarity or alters existing cyclic features, then SCD analysis could provide robust features for workload prediction. Efficient algorithms for computing the SCD, such as the FFT Accumulation Method (FAM) and the Strip Spectral Correlation Analyzer (SSCA), have been developed and optimized for hardware implementation, making aspects of this analysis potentially feasible in real-time [ref1]. The detailed analysis of fixed-point implementations and resource/SQNR trade-offs for FAM [ref12] underscores the effort to make such advanced spectral techniques practical.

Advanced Feature Extraction

Beyond classical spectral analysis, more advanced feature extraction techniques can provide richer, more stable representations of audio signals. The Wavelet Scattering Transform, for example, constructs time-frequency representations that are stable to time-warping and deformations, offering robustness against minor variations in timing or tempo [ref13]. It achieves this by cascading wavelet filter bank convolutions with complex modulus operations and low-pass averaging. The first-order scattering coefficients,

$$S_1(x; \lambda_1, t) = |x * \psi_{\lambda_1}| * \phi(t),$$

capture local energy in frequency bands similar to a spectrogram, while second-order coefficients,

$$S_2(x; \lambda_1, \lambda_2, t) = ||x * \psi_{\lambda_1}| * \psi_{\lambda_2}| * \phi(t),$$

capture modulations of these spectral envelopes, providing information about temporal textures and rhythmic patterns [ref13]. Although typically used for audio classification or analysis, these stable, multi-scale features could be highly effective for workload prediction if specific signal textures or modulation patterns are found to correlate strongly with the resource demands of different audio modules or processing chains.

Signal Correlation Metrics

The term “signal correlation metrics” can also encompass time-domain or phase-based measures. For instance, if the workload of a module is related to the phase synchrony between different components of an audio signal (e.g., harmonics of a note, or multiple instrument lines), then metrics like the phase-locking value (PLV) could be relevant. Zhao et al. (2023) proposed an improved PLV based on a rotating reference frame for analyzing phase synchrony, which might offer enhanced robustness or sensitivity for workload feature extraction [ref14].

The selection of an appropriate signal analysis technique for workload forecasting involves a critical trade-off. More sophisticated techniques like full SCD computation

or multi-layer wavelet scattering can yield highly discriminative features but may incur significant computational cost if implemented in real-time on the FPGA for continuous monitoring [ref1]. Simpler spectral estimates, time-domain correlations, or carefully selected subsets of scattering coefficients might offer a better balance between predictive power and the overhead of feature extraction. This recurring theme highlights the necessity of bridging advanced mathematical concepts with practical, efficient implementations, particularly within the resource and real-time constraints of an FPGA environment. The features extracted must be not only predictive but also computable with sufficiently low latency and resource usage so as not to negate the benefits of the subsequent dynamic reconfiguration.

2.1.2 Adaptive Algorithms and Control Strategies

Once predictive features are extracted from the audio signals, an adaptive mechanism is needed to translate these predictions into decisions about FPGA resource allocation. Adaptive algorithms and control theory provide the principles for such mechanisms.

Adaptive Filtering

Adaptive filtering is a well-established field in signal processing where the parameters of a filter (e.g., its coefficients) are iteratively adjusted to minimize an error signal, typically the difference between the filter’s output and a desired response. The Least Mean Squares (LMS) algorithm is a cornerstone in this area due to its simplicity and computational efficiency. A widely used variant, particularly relevant in audio applications like active noise control (ANC), is the Filtered-x LMS (FxLMS) algorithm [ref9].

The FxLMS algorithm is designed for systems where an adaptive filter $w(n)$ generates an output that passes through a secondary path (e.g., an acoustic path from a loudspeaker to an error microphone in ANC) before contributing to the error signal $e(n)$. To account for this secondary path, $S(z)$, the reference input signal $x(n)$ is pre-filtered by an estimate of this path, $\hat{S}(z)$, to produce $x'(n)$. The filter coefficients are then updated according to:

$$w(n+1) = w(n) + \mu e(n)x'(n)$$

where μ is the step-size parameter controlling the adaptation rate and stability, and M is the length of the filter $w(n)$ [ref9]. The performance of the FxLMS algorithm, including its convergence speed and stability, is influenced by the accuracy of the secondary path estimate $\hat{S}(z)$, the choice of step size μ , and the characteristics of the signals involved [ref9].

While adaptive algorithms like FxLMS are typically employed for direct signal processing tasks (e.g., noise cancellation, equalization), the underlying principles of online system identification (estimating $\hat{S}(z)$) and error-driven parameter adjustment ($w(n)$) are directly analogous to the problem of an FPGA adaptively reconfiguring its hardware. In the context of this thesis, the “system” being adapted is the FPGA’s resource configuration, the “input” is the predicted workload, and

the “error” could be a measure of deviation from desired performance targets (e.g., latency, throughput) or resource utilization efficiency. The FxLMS algorithm’s need to model a “secondary path” mirrors the requirement for the predictive reconfiguration system to understand how changes in FPGA resources (e.g., allocating more DSP slices) translate into changes in performance or power consumption for a given audio module.

2.2 Cyclostationary and Control Theoretic Approaches to Signal Analysis

The *Koopman operator*, $\mathcal{U}$, provides a way to study a nonlinear dynamical system, evolving according to a state-transition function $x_{k+1} = F(x_k)$, by examining its action on a space of observable functions, g , of the system’s state x [Koopman1931, Mezic2005]. The operator acts on an observable g such that:

$$(\mathcal{U}g)(x) = g(F(x))$$

A key insight of Koopman theory is that while the underlying state dynamics may be nonlinear, the Koopman operator itself is linear, albeit generally acting on an infinite-dimensional space of observables [Budisic2012]. The practical utility of the Koopman operator often hinges on finding a finite-dimensional vector of observables, $\mathbf{g}(x) = [g_1(x), g_2(x), \dots, g_m(x)]^T$, whose span is invariant under the action of $\mathcal{U}$. If such an invariant subspace exists, the Koopman operator can be represented by a finite-dimensional matrix K , often referred to as the Koopman matrix. This leads to a linear predictive model in the space of observables:

$$\mathbf{g}(x_{k+1}) = K\mathbf{g}(x_k)$$

This transformation allows for the prediction of the future evolution of the chosen observables using standard linear system theory, even if the original system F is highly nonlinear. Various methods have been developed to approximate Koopman operators and their corresponding modes and eigenvalues from time-series data [Schmid2010, Williams2015]. Dynamic Mode Decomposition (DMD) is one such popular technique, which seeks to find a linear model $x_{k+1} \approx Ax_k$ that best describes the evolution of snapshots of the system state. Extended DMD (EDMD) generalizes this by applying DMD to a dictionary of observable functions, effectively seeking the matrix K [Williams2015].

Recent applications in audio signal processing highlight the potential of Koopman-based models. For instance, Huhtala et al. (2024) utilized Koopman operator theory to linearize the long-term dynamics of nonlinear audio effects such as vacuum tube amplifiers, enabling more efficient modelling [Huhtala2024]. Similarly, Díaz and Schlecht (2024) employed an extended Koopman embedding for modelling the dynamics of stiff strings, showcasing its utility for complex physical audio systems [DiazSchlecht2024].

2.3 Mathematical Foundations for design derivation

This section reviews the mathematical and algorithmic foundations underlying the projection-based bit allocation framework developed in Chapter 3. The techniques synthesize results from multiple disciplines:

2.3.1 Fixed-Point Quantization and Bit Allocation

The dyadic quantization framework builds on classical filter design literature [**Gustafsson2007**, **OppenheimDSP**], which establishes fundamental trade-offs between coefficient precision and filter performance. Dyadic rationals (numbers of the form $\frac{p}{2^k}$ with integer p and nonnegative integer k) are particularly important for FPGA implementations because they have exact binary representation and enable bit-shift arithmetic [**Abdi2024**]. The sensitivity-driven bit allocation extends the greedy algorithm of [**Gustafsson2007**] by incorporating complex-valued error projectors that simultaneously capture magnitude and phase distortion.

The quantization noise model follows [**Widrow1996**], with variance $\text{Var}[\epsilon_n] = \frac{1}{12}(2^{-m_n})^2$ for uniform rounding. This model assumes that rounding errors behave as independent uniform random variables, which is valid when the signal varies rapidly relative to the quantization step size. The total quantization error power is then the sum of individual coefficient error contributions weighted by their sensitivity.

Gustafsson's work [**Gustafsson2007**] established that for a fixed total bit budget B_{total} , the optimal allocation minimizes the worst-case error by distributing bits proportional to coefficient sensitivity. The greedy algorithm iteratively allocates one bit to the coefficient with the highest marginal error reduction until the budget is exhausted. This approach is provably optimal for convex error metrics and provides a near-optimal approximation for general sensitivity matrices.

2.3.2 Wavelet Theory and Multi-Resolution Analysis

The Shannon wavelet basis forms the orthonormal foundation for multi-band decomposition [**Daubechies1992**, **Mallat1989**]. The Shannon scaling function $\phi(t) = \text{sinc}(t)$ has Fourier transform $\hat{\phi}(\omega) = \mathbf{1}_{[-\pi, \pi]}(\omega)$, providing perfect bandlimiting. The mother wavelet $\psi(t) = \text{sinc}(t/2) \cos(\pi t)$ extracts the frequency band $[\pi, 2\pi] \cup [-2\pi, -\pi]$.

Multi-resolution analysis (MRA) [**Mallat1989**, **Daubechies1992**] establishes a nested sequence of approximation spaces:

$$\cdots \subset V_{-1} \subset V_0 \subset V_1 \subset \cdots \subset L^2(\mathbb{R})$$

where $V_j = \{\text{span}\{\phi_{j,k}(t)\}_{k \in \mathbb{Z}}\}$ and $\phi_{j,k}(t) = 2^{j/2} \phi(2^j t - k)$. The detail spaces $W_j = V_{j+1} \ominus V_j$ are spanned by $\{\psi_{j,k}\}$ and satisfy:

$$L^2(\mathbb{R}) = V_{j_0} \oplus \bigoplus_{j=j_0}^{\infty} W_j$$

This decomposition enables independent processing of frequency bands. Our per-scale bit allocation leverages the dyadic structure, as formalized in [VetterliKovacevic1995]. The Parseval identity [Walter1992] ensures energy preservation:

$$\|f\|^2 = \sum_k |c_{j_0,k}|^2 + \sum_{j=j_0}^{\infty} \sum_k |d_{j,k}|^2$$

which is critical for interpreting sensitivity metrics across scales.

2.4 Existing Hardware Techniques and Designs

2.4.1 FPGA Synthesis and CORDIC Algorithms

Hardware-efficient evaluation of trigonometric and hyperbolic functions employs CORDIC (COordinate Rotation DIgital Computer) algorithms [Volder1959, Andraka1998], which replace multiplications with bit-shifts and additions. The CORDIC algorithm computes rotations in the plane by iteratively performing micro-rotations:

$$\begin{pmatrix} x_{i+1} \\ y_{i+1} \end{pmatrix} = \begin{pmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{pmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix}$$

where $\sigma_i \in \{-1, +1\}$ is chosen to reduce a residual angle z_i . After n iterations, the accumulated rotation approximates any target angle to precision 2^{-n} .

The modal synthesis approach (Chapter 4, Section 4.2) decomposes filtered projectors into sums over poles, each realized as a CORDIC oscillator. For a transfer function $H(z)$ with poles $\{p_k\}$, the partial fraction decomposition gives:

$$H(z) = \sum_k \frac{r_k}{z - p_k}$$

where each term $r_k/(z - p_k)$ corresponds to a damped oscillator with frequency $\arg(p_k)$ and decay rate $\log |p_k|$. In hardware, each pole is implemented as a complex CORDIC unit generating $e^{i\omega_k n}$ modulated by the residue r_k . This extends the dyadic filter design techniques of [ParkhiUnbehauen1974] to complex-valued kernels.

The key advantage is that CORDIC units require only shifters and adders, avoiding expensive multipliers. For FPGA implementations where DSP slices are limited, this enables high-order filters with minimal resource consumption [Andraka1998].

2.4.2 Rate-Distortion Theory and Water-Filling

While not explicitly invoked in Chapter 4, the optimal bit allocation framework parallels rate-distortion theory [CoverThomas2006], which characterizes the minimum bit rate R required to achieve a target distortion D . The rate-distortion function is defined as:

$$R(D) = \min_{p(\hat{x}|x): \mathbb{E}[d(x, \hat{x})] \leq D} I(X; \hat{X})$$

where $I(X; \hat{X})$ is mutual information and $d(x, \hat{x})$ is a distortion measure.

For Gaussian sources with mean-squared error distortion, the water-filling solution allocates bits to frequency components inversely proportional to their noise variance:

$$R_i = \max \left\{ 0, \frac{1}{2} \log_2 \frac{\sigma_i^2}{\lambda} \right\}$$

where λ is chosen to satisfy the total rate constraint $\sum_i R_i = R_{\text{total}}$.

Our greedy algorithm can be viewed as a discrete approximation to water-filling [Kailath2000], allocating precision where marginal error reduction is greatest. The sensitivity matrix S_n plays the role of σ_i^2 : coefficients with high sensitivity (analogous to high signal variance) receive more bits to maintain a balanced error contribution across all terms.

2.4.3 Bilinear Transform and Analog Filter Discretization

The bilinear transform [OppenheimDSP, Kailath2000] maps continuous-time filters in the s -domain to discrete-time filters in the z -domain via:

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}$$

where T is the sampling period. This mapping is conformal and maps the left half-plane ($\Re(s) < 0$) to the interior of the unit circle ($|z| < 1$), preserving stability.

For a causal analog filter with transfer function $H_a(s)$, the region of convergence (ROC) of the Laplace transform is a right half-plane $\Re(s) > \sigma_0$. For stable filters, $\sigma_0 < 0$, ensuring all poles lie in the left half-plane. The bilinear transform maps these poles inside the unit circle, guaranteeing the digital filter's stability.

This property is crucial for the contour integration in Chapter 4, Section 4.2, where we evaluate integrals involving $H_a(\omega)$ on the imaginary axis. The ROC ensures that the contour can be deformed without crossing poles, enabling application of Jordan's lemma and the residue theorem.

The frequency warping introduced by the bilinear transform is given by:

$$\Omega = \frac{2}{T} \tan \left(\frac{\omega T}{2} \right)$$

where Ω is the analog frequency and ω is the digital frequency. Pre-warping techniques [OppenheimDSP] compensate for this nonlinear mapping by adjusting critical frequencies (e.g., cutoff, center frequencies) before applying the transform.

2.4.4 Dynamic Partial Reconfiguration for Adaptive Systems

While static FPGA designs allocate resources at synthesis time, Dynamic Partial Reconfiguration (DPR) enables runtime modification of hardware modules without interrupting the operation of unaffected regions. This capability is particularly valuable for multimedia signal processing applications where computational demands vary dynamically based on input characteristics or user requirements.

Bhandari et al. [Bhandari2012] demonstrated a high-speed DPR system specifically optimized for real-time multimedia processing on FPGAs. Their work addressed a critical bottleneck: the reconfiguration latency inherent in swapping hardware modules. By developing a custom DPR controller optimized for the target FPGA architecture, they achieved a $55\times$ improvement in reconfiguration throughput compared to vendor-provided solutions. This dramatic speedup makes it feasible to reconfigure processing modules within tight real-time constraints, such as those imposed by audio frame boundaries (typically 1–10 ms for low-latency applications).

The architectural approach of Bhandari et al. is relevant to the predictive resource allocation framework developed in this thesis. Where Chapter 4 formulates reconfiguration as a model-predictive control problem with penalties for frequent reconfigurations (the $\lambda\|\Delta\mathbf{u}_k\|^2$ term in Eq. 4.7), the work of Bhandari et al. reduces the cost of those reconfigurations, effectively lowering λ and permitting more aggressive adaptation. Their custom controller demonstrates that with sufficient architectural optimization, the overhead of DPR can be reduced to the point where fine-grained, demand-driven resource allocation becomes practical for streaming multimedia workloads.

Chapter 3

Derivation of novel sensitivity-based Wavelet for synthesis

The central challenge posed in this thesis is enabling *adaptive FPGA signal chains* that can predictively reconfigure hardware resources in response to changing workloads. A modular audio effect chain—comprising filters, nonlinear processors, and dynamic effects—places time-varying demands on FPGA primitives (DSP slices, LUTs, BRAMs). To meet this challenge, we require a mathematical framework that:

1. Quantifies the resource-error trade-off for individual DSP kernels (e.g., filters, oscillators).
2. Extends naturally to multi-rate and multi-band processing (wavelet decompositions).
3. Provides closed-form sensitivity metrics to guide runtime resource reallocation.
4. Enables dynamic bit allocation across frequency bands based on signal characteristics.

This chapter develops such a framework by unifying three key ideas: **(1)** projection-based approximation of infinite-support kernels (the sinc function), **(2)** sensitivity-driven bit allocation under hardware constraints, and **(3)** orthonormal wavelet bases that decompose signals into frequency bands with independent resource budgets. The result is an analytical tool that connects signal processing requirements directly to FPGA resource specifications, enabling the runtime optimization envisioned in Chapter 1.

Chapter roadmap. Section 4.1 establishes the optimal bit allocation problem for polynomial approximations of the sinc function, deriving a sensitivity matrix that guides resource distribution. Section 4.2 extends this to generic filter kernels via contour integration. Section 4.3 introduces dyadic quantization for fixed-point hardware. Section 4.4 extends the projector framework to the Shannon wavelet basis, enabling multi-band decomposition with per-scale bit allocation. Sections 4.5–4.6 synthesize these results into a hardware architecture for adaptive signal processing.

3.1 Optimal Coefficient Quantization via Projection Methods

One of the fundamental results in DSP is that the ideal low-pass filter (LPF) in the time domain is the sinc function [OppenheimDSP]:

$$h_{\text{ideal}}(t) = \text{sinc}(t) = \frac{\sin(t)}{t},$$

which corresponds to a perfect rectangular pulse in the frequency domain. However, $\text{sinc}(t)$ has infinite support with envelope decay $\propto 1/t$, making it impossible to implement directly in hardware. Practical implementations require both:

1. **Truncation:** approximating the infinite series by a finite polynomial of degree $2N$,
2. **Quantization:** representing each coefficient as a dyadic rational $a_n/2^{m_n}$ for fixed-point arithmetic.

This section establishes a unified framework for optimal bit allocation that minimizes the combined truncation and quantization error under hardware resource constraints.

3.1.1 Chapter Objectives

The goal of this chapter is to:

1. **Establish a projection-based error analysis framework** for the sinc function and its compositions/convolutions with arbitrary polynomials $p(z) \in \mathbb{C}[z]$.
2. **Derive an error projector** $R(z) : \mathbb{R} \rightarrow \mathbb{C}$ that simultaneously captures magnitude and phase error as functions of the truncation point N and bit allocation vector $\mathbf{m}$.
3. **Extend the projector to the Shannon wavelet orthonormal basis**, introducing a generalized frequency-domain kernel $\hat{H}(\omega)$ that enables sensitivity-based bit allocation across multiple frequency bands.
4. **Demonstrate computational savings** through concrete examples comparing the proposed method against naive uniform quantization and established filter design techniques.
5. **Validate against established methods** through quantitative metrics (SQNR, THD, resource utilization).

3.1.2 Scope and Limitations

This work focuses on bandlimited signals representable in the Shannon wavelet basis. The following are explicitly *outside* the scope:

- **Time-frequency trade-offs:** We do not address regularization between the Shannon basis (poor temporal, excellent frequency resolution) and bases like Haar functions (excellent temporal, poor frequency resolution).
- **Formal correctness proofs:** While the constructions are mathematically rigorous, we do not provide algorithmic correctness proofs to standards required for, e.g., verified compilation.
- **Parameter selection heuristics:** The choice of truncation point N and initial resolution j_0 is assumed given by application requirements (e.g., target SQNR, bandwidth).
- **Multi-band error correlation:** We treat each frequency band (wavelet scale) independently. Extensions to parametric EQ with inter-band correlation (e.g., modeling masking effects in psychoacoustics) are deferred to future work.

3.1.3 Problem Formulation

Consider a function $f(x)$ represented as an even polynomial series:

$$f(x) = \sum_{n=0}^N c_n x^{2n}, \quad (3.1)$$

where $c_n \in \mathbb{R}$ are the exact series coefficients (e.g., $c_n = \frac{(-1)^n}{(2n+1)!}$ for $\text{sinc}(x)$).

In fixed-point hardware implementations (FPGAs, ASICs), coefficients must be quantized to *dyadic rationals* [Gustafsson2007, Abdi2024]:

$$\tilde{c}_n = \frac{a_n}{2^{m_n}}, \quad a_n \in \mathbb{Z}, \quad m_n \in \mathbb{N}_0, \quad (3.2)$$

where m_n is the number of fractional bits allocated to coefficient n , and $a_n = \lfloor c_n \cdot 2^{m_n} + 0.5 \rfloor$ is the nearest integer (rounding). Dyadic rationals have exact finite binary representation, making them ideal for hardware arithmetic. The problem of finding optimal dyadic coefficients subject to approximation error constraints can be formulated as a dyadic linear program [Abdi2024].

The coefficient-wise quantization error is:

$$\epsilon_n = c_n - \tilde{c}_n, \quad |\epsilon_n| \leq \frac{1}{2^{m_n+1}}. \quad (3.3)$$

The quantized polynomial is:

$$\tilde{f}(x) = \sum_{n=0}^N \tilde{c}_n x^{2n}, \quad (3.4)$$

and the total approximation error combines polynomial truncation and coefficient quantization:

$$E_{\text{total}}(x) = f(x) - \tilde{f}(x) = \sum_{n=0}^N \epsilon_n x^{2n}. \quad (3.5)$$

Optimization Problem

Objective: Determine the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$ that minimizes a weighted approximation error subject to a total bit budget:

$$\min_{\mathbf{m} \in \mathbb{N}_0^{N+1}} \mathcal{E}_{\text{proj}}(\mathbf{m}) \quad \text{subject to} \quad \sum_{n=0}^N m_n \leq B_{\text{total}}, \quad (3.6)$$

where $\mathcal{E}_{\text{proj}}(\mathbf{m})$ is an application-specific error metric (defined in Section 3.1.4).

Key insight: By formulating the problem in terms of a projection operator $R(x)$ derived from the target application (filter specifications, signal characteristics), we obtain a sensitivity matrix $\mathbf{S}$ that encodes both magnitude and phase error contributions. This enables:

- **Non-uniform bit allocation:** Coefficients with high sensitivity receive more bits.
- **Hardware efficiency:** Total bit budget reflects actual FPGA/ASIC resources (DSP slices, BRAM).
- **Extensibility:** The framework naturally extends to the Shannon wavelet basis, where each scale (frequency band) has an independent bit allocation sub-problem.

3.1.4 Projection Operator and Error Metric

Rather than minimizing approximation error in the standard L^2 norm (which treats all frequency components equally), we introduce a *projection operator* $R(x)$ that encodes application-specific error weighting. This formulation allows us to:

- Prioritize accuracy in critical frequency bands (e.g., passband for LPF design),
- Embed phase and magnitude sensitivities within a unified metric,
- Derive closed-form sensitivity expressions via contour integration (Section 3.1.9).

Projected Error Functional

The projected error is defined as the weighted inner product:

$$\mathcal{E}_{\text{proj}} = \langle E_{\text{total}}(x), R(x) \rangle = \int_0^{x_{\text{max}}} E_{\text{total}}(x) \cdot \overline{R(x)} dx, \quad (3.7)$$

where:

- $E_{\text{total}}(x)$ is the total approximation error from Eq. (3.5),
- $\overline{R(x)}$ denotes the complex conjugate of the projection operator,
- $\langle \cdot, \cdot \rangle$ represents the L^2 inner product on $[0, x_{\text{max}}]$,
- x_{max} is the maximum argument value of interest (e.g., $x_{\text{max}} = \pi$ for audio applications).

Substituting the error polynomial from Eq. (3.5):

$$\mathcal{E}_{\text{proj}} = \int_0^{x_{\text{max}}} \left[\sum_{n=0}^N \epsilon_n x^{2n} \right] \overline{R(x)} dx = \sum_{n=0}^N \epsilon_n \underbrace{\int_0^{x_{\text{max}}} x^{2n} \overline{R(x)} dx}_{:= I_n}. \quad (3.8)$$

The integrals I_n quantify the contribution of coefficient n to the projected error and directly define the sensitivity matrix (Section 3.1.5).

3.1.5 Sensitivity Matrix Construction

The sensitivity of the projected error to coefficient n quantifies how a unit quantization error ϵ_n propagates through the system. From Eq. (3.8), we define:

$$S_n = \left| \frac{\partial \mathcal{E}_{\text{proj}}}{\partial \epsilon_n} \right| = \left| \int_0^{x_{\text{max}}} x^{2n} \overline{R(x)} dx \right| = |I_n|. \quad (3.9)$$

The sensitivity S_n has a clear geometric meaning: it is the magnitude of the projection of the monomial x^{2n} onto the error weighting function $R(x)$ in the L^2 inner product space. Coefficients with larger sensitivity contribute more heavily to the overall error metric and therefore warrant higher bit precision.

Phase and Magnitude Decomposition

For a complex projector $R(x) = R_{\text{real}}(x) + iR_{\text{imag}}(x)$, the integral I_n decomposes into real and imaginary parts:

$$I_n^{\text{real}} = \int_0^{x_{\text{max}}} x^{2n} \Re(R(x)) dx, \quad (3.10)$$

$$I_n^{\text{imag}} = \int_0^{x_{\text{max}}} x^{2n} \Im(R(x)) dx. \quad (3.11)$$

Following the standard DSP convention, we associate:

$$S_n^{\text{mag}} = |I_n^{\text{real}}| \quad (\text{magnitude sensitivity}), \quad (3.12)$$

$$S_n^{\text{phase}} = |I_n^{\text{imag}}| \quad (\text{phase sensitivity}). \quad (3.13)$$

The complete sensitivity matrix $\mathbf{S} \in \mathbb{R}^{(N+1) \times 2}$ is:

$$\mathbf{S} = \begin{bmatrix} S_0^{\text{mag}} & S_0^{\text{phase}} \\ S_1^{\text{mag}} & S_1^{\text{phase}} \\ \vdots & \vdots \\ S_N^{\text{mag}} & S_N^{\text{phase}} \end{bmatrix}. \quad (3.14)$$

Application-Specific Weighting

The choice of projection operator $R(x)$ determines the error metric:

- **Uniform weighting:** $R(x) = 1$ yields the standard L^2 norm.
- **Frequency-selective weighting:** For LPF design, set $R(x) = \exp(i\omega_c x)$ to emphasize passband accuracy near cutoff frequency ω_c .
- **Wavelet-based weighting:** For multi-resolution analysis, $R(x)$ is the Shannon wavelet kernel at scale j (see Section ??).

3.1.6 Quantization Error Bounds

For dyadic quantization with m_n fractional bits allocated to coefficient n , the rounding operation in Eq. (3.2) satisfies:

$$|\epsilon_n| = |c_n - \tilde{c}_n| \leq \frac{1}{2^{m_n+1}}. \quad (3.15)$$

This bound is tight: the worst-case error occurs when c_n falls exactly midway between two quantization levels.

Worst-Case Projected Error

Applying the triangle inequality to Eq. (3.8):

$$|\mathcal{E}_{\text{proj}}| \leq \sum_{n=0}^N S_n |\epsilon_n| \leq \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}. \quad (3.16)$$

This upper bound is the objective function for bit allocation: by minimizing the right-hand side subject to $\sum_{n=0}^N m_n \leq B_{\text{total}}$, we obtain the allocation that guarantees the smallest worst-case projected error.

Expected Error (Uniform Random Rounding)

If quantization errors are modeled as independent uniform random variables $\epsilon_n \sim \mathcal{U}(-2^{-(m_n+1)}, 2^{-(m_n+1)})$, the expected squared error is:

$$\mathbb{E}[|\mathcal{E}_{\text{proj}}|^2] = \sum_{n=0}^N (S_n)^2 \cdot \frac{1}{3 \cdot 2^{2m_n+2}}, \quad (3.17)$$

where the factor $1/3$ arises from $\text{Var}[\epsilon_n] = \frac{1}{12} \cdot (2^{-m_n})^2$ [Widrow1996].

Remark: The greedy algorithm (Section 3.1.7) optimizes the worst-case bound (3.16), which is conservative but deterministic. For stochastic analysis, minimizing (3.17) leads to the allocation:

$$m_n^{\text{stochastic}} = m_{\text{base}} + \frac{1}{2} \log_2(S_n) + \text{const}. \quad (3.18)$$

(See Appendix ?? for detailed derivation.)

3.1.7 Optimal Bit Allocation Strategy

Given the sensitivity matrix $\mathbf{S}$ and total bit budget B_{total} , we now present three methods for determining the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$. Each method trades off computational complexity against optimality guarantees.

Method 1: Sensitivity-Proportional Allocation

The simplest heuristic allocates bits proportionally to sensitivity:

$$m_n = m_{\text{base}} + \left\lfloor \alpha \cdot \frac{S_n}{S_{\text{max}}} \right\rfloor, \quad (3.19)$$

where:

- $S_{\text{max}} = \max_{k \in \{0, \dots, N\}} S_k$ is the maximum sensitivity,
- m_{base} is the minimum bit allocation (typically $m_{\text{base}} = 8$ for 8-bit baseline precision),
- $\alpha \in [3, 8]$ controls the dynamic range of bit allocation (larger α increases disparity between high and low sensitivity coefficients).

After initial allocation, adjust α iteratively to satisfy the budget constraint:

$$\sum_{n=0}^N m_n = B_{\text{total}}. \quad (3.20)$$

Complexity: $O(N)$ per iteration; typically converges in 5–10 iterations via binary search on α .

Optimality: No guarantee; provides a reasonable starting point for refinement.

Method 2: Weighted Phase-Magnitude Allocation

For applications requiring different emphasis on phase versus magnitude accuracy (e.g., audio where phase distortion is perceptually critical), define a weighted sensitivity:

$$S_n^{\text{weighted}} = \sqrt{w_{\text{mag}}(S_n^{\text{mag}})^2 + w_{\text{phase}}(S_n^{\text{phase}})^2}, \quad (3.21)$$

where $w_{\text{mag}}, w_{\text{phase}} \geq 0$ are application-dependent weights satisfying $w_{\text{mag}} + w_{\text{phase}} = 1$.

Then apply proportional allocation using S_n^{weighted} in place of S_n in Eq. (3.19).

Typical values:

- Audio synthesis: $w_{\text{phase}} = 0.7, w_{\text{mag}} = 0.3$,
- Control systems: $w_{\text{mag}} = 0.8, w_{\text{phase}} = 0.2$,
- General filtering: $w_{\text{mag}} = w_{\text{phase}} = 0.5$.

Method 3: Greedy Optimization

For the provably optimal solution to minimize the worst-case bound (3.16), we employ a greedy algorithm [Gustafsson2007]. The algorithm proceeds as follows:

Algorithm 4.1: Greedy Bit Allocation

1. **Initialize:** Set $m_n = 1$ for all $n \in \{0, \dots, N\}$
2. **Set remaining budget:** $B_{\text{remaining}} = B_{\text{total}} - (N + 1)$
3. **While** $B_{\text{remaining}} > 0$:
 - (a) **Compute error reduction** for adding one bit to coefficient n :

$$\Delta_n = S_n \left(\frac{1}{2^{m_n+1}} - \frac{1}{2^{m_n+2}} \right) = \frac{S_n}{2^{m_n+2}}, \quad \forall n \in \{0, \dots, N\} \quad (3.22)$$

- (b) **Select coefficient** with maximum error reduction:

$$n^* = \arg \max_{n \in \{0, \dots, N\}} \Delta_n \quad (3.23)$$

- (c) **Increment allocation:** $m_{n^*} \leftarrow m_{n^*} + 1$
 - (d) **Decrement budget:** $B_{\text{remaining}} \leftarrow B_{\text{remaining}} - 1$
4. **Return** the optimal bit allocation vector $\mathbf{m} = (m_0, m_1, \dots, m_N)$

Complexity: $O(B_{\text{total}} \cdot N)$ — the outer loop executes $B_{\text{total}} - (N + 1)$ times, and each iteration scans $N + 1$ coefficients.

Optimality: The greedy algorithm is optimal for the additive error model in Eq. (3.16) because the objective function has the *diminishing returns property*: adding a bit to coefficient n reduces the error contribution by exactly half, independent of other allocations. (See Appendix ?? for formal proof.)

Method	Complexity	Optimal?	Use Case
Proportional (Sec. 3.1.7)	$O(N \log N)$	No	Rapid prototyping
Weighted (Sec. 3.1.7)	$O(N \log N)$	No	Phase-critical apps
Greedy (Sec. 3.1.7)	$O(B_{\text{total}} \cdot N)$	Yes	Production systems

Table 3.1: Comparison of bit allocation methods.

3.1.8 Evaluation of Projected Error

After determining the bit allocation vector $\mathbf{m}$ via one of the methods in Section 3.1.7, we compute the actual projected error to validate the design and compare against the theoretical bound (3.16).

Exact Projected Error

From Eq. (3.8), the projected error is:

$$\mathcal{E}_{\text{proj}} = \sum_{n=0}^N \epsilon_n \cdot I_n = \sum_{n=0}^N (c_n - \tilde{c}_n) \int_0^{x_{\max}} x^{2n} \overline{R(x)} dx. \quad (3.24)$$

where $\epsilon_n = c_n - \tilde{c}_n$ is the actual quantization error for coefficient n after rounding to m_n fractional bits.

Numerical Evaluation via Discretization

For practical computation, discretize the integral using a uniform grid with K points:

$$\mathcal{E}_{\text{proj}} \approx \sum_{n=0}^N \epsilon_n \sum_{k=0}^{K-1} x_k^{2n} \overline{R(x_k)} \Delta x, \quad (3.25)$$

where:

- $x_k = k \cdot \Delta x$ with $\Delta x = x_{\max}/K$,
- $K \geq 1024$ is typically sufficient for convergence (verify by doubling K and checking relative change $< 10^{-6}$).

Magnitude and Phase Error Components

Decompose the complex projected error into magnitude and phase contributions:

$$\mathcal{E}_{\text{real}} = \Re(\mathcal{E}_{\text{proj}}) = \sum_{n=0}^N \epsilon_n \cdot I_n^{\text{real}}, \quad (3.26)$$

$$\mathcal{E}_{\text{imag}} = \Im(\mathcal{E}_{\text{proj}}) = \sum_{n=0}^N \epsilon_n \cdot I_n^{\text{imag}}, \quad (3.27)$$

where I_n^{real} and I_n^{imag} are defined in Eqs. (3.10) and (3.11).

The magnitude error and phase error are then:

$$|\mathcal{E}_{\text{proj}}| = \sqrt{(\mathcal{E}_{\text{real}})^2 + (\mathcal{E}_{\text{imag}})^2}, \quad (3.28)$$

$$\angle \mathcal{E}_{\text{proj}} = \arctan\left(\frac{\mathcal{E}_{\text{imag}}}{\mathcal{E}_{\text{real}}}\right). \quad (3.29)$$

Validation Against Theoretical Bound

The computed error should satisfy the worst-case bound:

$$|\mathcal{E}_{\text{proj}}| \leq \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}, \quad (3.30)$$

with typical margin of 20%–50% (i.e., actual error is 0.5–0.8 times the theoretical bound). A larger margin indicates:

- **Conservative allocation:** The greedy algorithm over-allocates bits due to worst-case assumption; consider using the stochastic allocation (3.18) to reduce budget.
- **Favorable coefficient signs:** Cancellation between positive and negative ϵ_n terms reduces total error.

Practical Considerations for FPGA Implementation

When implementing the error evaluation in hardware:

1. **Precompute integrals I_n :** Store the $(N + 1)$ complex values in BRAM as a lookup table. Recomputation is unnecessary unless $R(x)$ or $x_{\max}$ changes.
2. **Use CORDIC for $R(x)$ evaluation:** If $R(x)$ involves trigonometric functions (e.g., $R(x) = e^{i\omega x}$), employ the CORDIC algorithm [Andraka1998, Volder1959] in rotation mode to avoid multiplier-intensive implementations.
3. **Pipeline the summation:** Implement Eq. (3.24) as a tree adder with $\lceil \log_2(N + 1) \rceil$ stages to achieve single-cycle throughput (latency $\propto \log_2 N$).
4. **Monitor at runtime:** For adaptive systems, recompute $\mathcal{E}_{\text{proj}}$ periodically (e.g., every 10^6 samples) and adjust $\mathbf{m}$ if error exceeds specification by triggering a reallocation cycle.

Connection to adaptive FPGA design. The bit allocation algorithms in this section directly address the resource management problem posed in the thesis abstract: given a time-varying audio signal with changing frequency content, the sensitivity matrix $\mathbf{S}$ provides real-time feedback on which polynomial coefficients require high precision. This enables the FPGA fabric to dynamically reallocate DSP slices and BRAM blocks—assigning more resources to high-sensitivity coefficients (critical frequency bands) while reducing precision for low-sensitivity terms. The greedy algorithm’s $O(B_{\text{total}} \cdot N)$ complexity makes it suitable for runtime execution, supporting the adaptive signal chain architecture described in Chapter 1.

However, the framework developed so far assumes that the projection operator $R(x)$ and sensitivity integrals $S_n = \left| \int_0^{x_{\max}} x^{2n} \overline{R(x)} dx \right|$ can be computed efficiently. Section 4.1.7 now derives an explicit closed-form expression for the sinc projector using complex analysis, which not only validates the abstract framework but also provides a computational template for evaluating any analytic kernel $H(\omega)$ via residue calculus (Section 4.2).

3.1.9 Projector Derivation for Sinc Function

The power series for $\text{sinc}(x)$ is given as:

$$\text{sinc}(x) = \frac{\sin(x)}{x} = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n}.$$

Due to the necessity of considering truncated power series, we introduce the upper bound N . Re-indexing over $n \in \{0, \dots, 2N\}$ and using the Kronecker delta to select even terms:

$$\text{sinc}_N(x) = \sum_{n=0}^{2N} \delta_{n \equiv 0 \pmod{2}} \cdot \frac{(-1)^{n/2}}{\Gamma(n+2)} x^n = \sum_{n=0}^{2N} \delta_{n \equiv 0 \pmod{2}} \cdot \frac{i^n}{\Gamma(n+2)} x^n, \quad (3.31)$$

where the second equality holds because $i^n = (-1)^{n/2}$ when n is even.

Analytic Projector via Contour Integration

The introduction of the Kronecker delta $\delta_{n \equiv 0 \pmod{2}}$ serves dual purposes: selecting even coefficients and facilitating analytic continuation. To make this precise, we let $\Omega \subset \mathbb{C}$ be a simply connected domain containing $[0, x_{\max}]$. Consider the space:

$$\mathcal{A}(\Omega) = \{f : \Omega \rightarrow \mathbb{C} \mid f \text{ is holomorphic on } \Omega\}.$$

For $f \in \mathcal{A}(\Omega)$ with Taylor expansion about $z = 0$:

$$f(z) = \sum_{k=0}^{\infty} a_k z^k, \quad a_k = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(\zeta)}{\zeta^{k+1}} d\zeta,$$

where γ is a simple closed contour in Ω enclosing the origin.

The projector $\mathbb{P}_N^{\text{even}} : \mathcal{A}(\Omega) \rightarrow \mathcal{P}_{2N}^{\text{even}}$ is defined via the modified Cauchy integral:

$$\mathbb{P}_N^{\text{even}}[f](z) = \frac{1}{2\pi i} \oint_{\gamma} K_N(\zeta) f\left(\frac{z}{\zeta}\right) \frac{d\zeta}{\zeta}$$

where $K_N(\zeta)$ is the *selection kernel*:

$$K_N(\zeta) = \sum_{n=0}^N \zeta^{2n} = \frac{1 - \zeta^{2N+2}}{1 - \zeta^2}.$$

This kernel satisfies the projection property:

$$\frac{1}{2\pi i} \oint_{|\zeta|=1} K_N(\zeta) \zeta^{-k-1} d\zeta = \begin{cases} 1 & \text{if } k \in \{0, 2, 4, \dots, 2N\}, \\ 0 & \text{otherwise.} \end{cases}$$

Note 3.1.1 (Odd projector). By analogy, an *odd coefficient projector* $\mathbb{P}_N^{\text{odd}} : \mathcal{A}(\Omega) \rightarrow \mathcal{P}_{2N-1}^{\text{odd}}$ can be defined using the odd selection kernel:

$$K_N^{\text{odd}}(\zeta) = \sum_{n=0}^{N-1} \zeta^{2n+1} = \frac{\zeta(1 - \zeta^{2N})}{1 - \zeta^2},$$

yielding the projector:

$$\mathbb{P}_N^{\text{odd}}[f](z) = \frac{1}{2\pi i} \oint_{\gamma} K_N^{\text{odd}}(\zeta) f\left(\frac{z}{\zeta}\right) \frac{d\zeta}{\zeta} = \frac{1}{2\pi i} \oint_{\gamma} \frac{1 - \zeta^{2N}}{1 - \zeta^2} f\left(\frac{z}{\zeta}\right) d\zeta.$$

Note that the ζ in the numerator of $K_N^{\text{odd}}(\zeta)$ cancels with the ζ^{-1} factor in $d\zeta/\zeta$. See Remark A.1.2 for details.

Using the contour representation of the Kronecker delta (Lemma A.1.1), we can drop the Kronecker delta and express

$$\mathbb{P}_N^{\text{even}}[\text{sinc}](z\zeta) = \sum_{n=0}^{2N} \left[\frac{1}{2\pi i} \oint_{|\zeta|=r<1} \frac{1}{\zeta^{n+1}(1-\zeta^2)} d\zeta \right] \frac{i^n z^n}{\Gamma(n+2)}.$$

Since the sum is finite, we may interchange the sum and the integral:

$$\mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\zeta|=r<1} \frac{1}{1-\zeta^2} \left[\sum_{n=0}^{2N} \frac{(iz/\zeta)^n}{\Gamma(n+2)} \right] \frac{d\zeta}{\zeta} \equiv \mathbb{P}_N^{\text{even}}[f](z)$$

This contour-based projector inherently filters even coefficients—the only non-vanishing terms in the sinc expansion.

Remark 3.1.1 (Odd expansion). For a function $g(x) = \sum_{n=0}^{2N-1} \delta_{n \equiv 1 \pmod{2}} a_n x^n$ with only odd powers, the analogous derivation using Remark A.1.2 yields:

$$\mathbb{P}_N^{\text{odd}}[g](z) = \frac{1}{2\pi i} \oint_{|\zeta|=r<1} \frac{1}{1-\zeta^2} \left[\sum_{n=0}^{2N-1} a_n (z/\zeta)^n \right] d\zeta,$$

where the odd parity selector $\frac{1}{\zeta^n(1-\zeta^2)}$ replaces $\frac{1}{\zeta^{n+1}(1-\zeta^2)}$, resulting in $d\zeta$ instead of $\frac{d\zeta}{\zeta}$ after factoring.

Evaluating the integral

$$\omega := \frac{iz}{\zeta}, \quad S_{\text{even}}(\omega) := \sum_{n=0}^{\infty} \frac{\omega^n}{\Gamma(n+2)}.$$

$$S_{\text{even}}(\omega) = \sum_{n=0}^{\infty} \frac{\omega^n}{(n+1)!} = \frac{1}{\omega} \sum_{k=1}^{\infty} \frac{\omega^k}{k!} = \frac{e^{\omega} - 1}{\omega},$$

with the removable value $S_{\text{even}}(0) = 1$.

Note: The swap of sum and integral is still justified since $|\zeta| = r > 0 \implies |\omega| = \frac{|z|}{r}$ is bounded, so the exponential series is uniformly convergent on the contour.

$$S_{2N}^{\text{even}}(\omega) := \sum_{n=0}^{2N} \frac{\omega^n}{\Gamma(n+2)} = \sum_{n=0}^{2N} \frac{\omega^n}{(n+1)!} = \frac{1}{\omega} \sum_{k=1}^{2N+1} \frac{\omega^k}{k!} = \frac{E_{2N+1}(\omega) - 1}{\omega},$$

where $E_m(z) := \sum_{k=0}^m \frac{z^k}{k!}$ is the truncated exponential polynomial.

$$\Gamma(m+1, z) = m! e^{-z} \sum_{k=0}^m \frac{z^k}{k!} \iff E_m(z) = \frac{e^z \Gamma(m+1, z)}{m!}.$$

Hence, with $m = 2N + 1$,

$$S_{2N}^{\text{even}}(\omega) = \frac{1}{\omega} \left(\frac{e^\omega \Gamma(2N + 2, \omega)}{(2N + 1)!} - 1 \right) = \frac{e^\omega \Gamma(2N + 2, \omega)}{\omega (2N + 1)!} - \frac{1}{\omega}.$$

$$S_{\text{even}}(\omega) - S_{2N}^{\text{even}}(\omega) = \frac{1}{\omega} \sum_{k=2N+2}^{\infty} \frac{\omega^k}{k!} \Rightarrow |S_{\text{even}}(\omega) - S_{2N}^{\text{even}}(\omega)| \leq \frac{e^{|\omega|}}{|\omega|} \frac{|\omega|^{2N+2}}{(2N + 2)!}.$$

Performing a change of variable $\frac{iz}{\zeta} \rightarrow \omega$ on the integral & substituting:

$$\mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\omega|=|z|/r} \frac{1}{\omega^2 + z^2} \left[\frac{e^\omega \Gamma(2N + 2, \omega)}{(2N + 1)!} - 1 \right] d\omega$$

(??)

Remark 3.1.2 (Change of variables for odd projector). For the odd projector, the same substitution $\omega = \frac{iz}{\zeta}$ yields $\frac{z}{\zeta} = \frac{\omega}{i} = -i\omega$. Defining the odd counterpart:

$$S_{\text{odd}}(\omega) := \sum_{n=0}^{2N-1} \frac{\omega^n}{\Gamma(n + 2)},$$

the original sum transforms as:

$$\sum_{n=0}^{2N-1} \frac{(z/\zeta)^n}{\Gamma(n + 2)} = \sum_{n=0}^{2N-1} \frac{(-i\omega)^n}{\Gamma(n + 2)} = S_{\text{odd}}(\omega).$$

After substituting $\zeta = \frac{iz}{\omega}$ and $d\zeta = -\frac{iz}{\omega^2} d\omega$, the odd projector becomes:

$$\mathbb{P}_N^{\text{odd}}[f](z) = \frac{z}{2\pi} \oint_{|\omega|=|z|/r} \frac{S_{\text{odd}}(\omega)}{\omega^2 + z^2} d\omega,$$

Where

$$S_{\text{odd}} = S_{\text{even}}(i\omega) + \frac{-1^{N+1}\omega^{2N}}{\Gamma(2N + 2)}$$

with an additional factor of z in the numerator compared to the even case.

$$\mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{1}{2\pi i} \oint_{|\omega|=|z|/r} \frac{H(\omega)}{\omega^2 + z^2} d\omega, \quad 0 < r < 1,$$

with either choice

$$H(\omega) = E_{2N+1}(\omega) - 1 \quad \text{or} \quad H(\omega) = \frac{e^\omega \Gamma(2N + 2, \omega)}{(2N + 1)!} - 1,$$

and $E_{2N+1}(\omega) = \sum_{k=0}^{2N+1} \omega^k / k!$.

$$\omega^2 + z^2 = (\omega - iz)(\omega + iz), \quad \omega = \pm iz \text{ are simple poles inside the contour.}$$

For any analytic H ,

$$\text{Res}_{\omega=\pm iz} \frac{H(\omega)}{\omega^2 + z^2} = \frac{H(\pm iz)}{(\omega^2 + z^2)'} \Big|_{\omega=\pm iz} = \frac{H(\pm iz)}{2(\pm iz)}.$$

Note:

$$\lim_{N \rightarrow \infty} \mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{e^{iz} - e^{-iz}}{2iz} = \frac{\sin z}{z}.$$

This is as expected because the phasor error contribution is solely contingent upon the effect of truncation when $w \in \mathbb{R}$.

Hence

$$\begin{aligned} \mathbb{P}_N^{\text{even}}[\text{sinc}](z) &= \frac{1}{2\pi i} \oint \frac{H(\omega)}{\omega^2 + z^2} d\omega = \frac{H(iz) - H(-iz)}{2iz} \\ &= \frac{E_{2N+1}(iz) - E_{2N+1}(-iz)}{2iz} = \sum_{m=0}^N \frac{(-1)^m z^{2m}}{(2m+1)!} \\ &= \frac{e^{iz}\Gamma(2N+2, iz) - e^{-iz}\Gamma(2N+2, -iz)}{2iz(2N+1)!} \\ &= \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}. \end{aligned}$$

This result is worth noting for the simplicity & expectedness of its decomposition. We showed, via standard complex analytic methods [Ahlfors1979, Stein2003], that the partial sum $\text{sinc}_N(z)$ can be rigorously expressed in its incomplete upper gamma function form [NIST:DLMF, Temme1996]. Moreover, that the projector error itself $\mathbb{E}(\mathbb{P}_N^{\text{even}})$, depends precisely on the upper gamma function's bound $N_{\text{gamma}} \propto N_{\text{sinc}}, N_{\text{gamma}} = 2(N_{\text{sinc}} + 1)$.

Lemma 3.1.1 (For $z \in \mathbb{R}$, the truncation projector can be written in terms of a hypergeometric series). **Important:** This representation is valid *only* for $z \in \mathbb{R}$. For complex arguments $z \in \mathbb{C} \setminus \mathbb{R}$, the full form $\mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z(2N+1)!}$ must be used to capture both magnitude and phase error. See Appendix A.1 for details.

Proof. Let

$$\begin{aligned} \theta &:= iz, \quad z \in \mathbb{R} \\ S &:= 2(N+1) \end{aligned}$$

A holomorphic extension of the upper incomplete gamma function [NIST:DLMF]

given as:

$$\begin{aligned}\gamma(S, \theta) &= \theta^S \Gamma(S) \gamma^*(S, \theta) \\ \Gamma(S, \theta) &= \Gamma(S) - \gamma(S, \theta) \\ \gamma^*(S, \theta) &:= e^{-\theta} \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)}\end{aligned}$$

can be applied to the expression $\Im(e^\theta \Gamma(S, \theta))$ as follows:

$$\Im(e^\theta \Gamma(S, \theta)) = \Im(\Gamma(S) e^\theta) - \Im\left(\Gamma(S) \theta^S \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)}\right)$$

For $z \in \mathbb{R}$, we have $\theta = iz \in i\mathbb{R}$, hence:

$$\Im(\Gamma(S) e^\theta) = \Gamma(S) \sin(z)$$

Since $S \in \mathbb{N}_0, S \geq 1 \implies \Gamma(S) = (S-1)!$ and $z \in \mathbb{R}$

$$\Gamma(S) \theta^S \sum_{k=0}^{\infty} \frac{\theta^k}{\Gamma(S+k+1)} = \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k \Gamma(S)}{\Gamma(S+k)} = \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k},$$

where $(S)_k := \frac{\Gamma(S+k)}{\Gamma(S)}$ is the rising Pochhammer symbol.

The outside term θ^{S-1} is imaginary-valued for $z \in \mathbb{R}$, since $\theta = iz$ and $S-1 = 2N+1$ is odd, hence $\theta^{S-1} = (iz)^{2N+1}$ is purely imaginary. We subsequently conclude, using the linearity of the $\Im$ operator over a series, that only even powers of θ contribute. Thus, for $z \in \mathbb{R}$:

$$\Im\left(\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = -i \theta^{S-1} \left(\sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = -i \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}.$$

$\sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}$ can be represented in terms of the general hypergeometric series $\mathcal{H}_S(z)$ [AndrewsAskeyRoy1999] as:

$$\mathcal{H}_S(z) := {}_1F_2\left(1; \frac{S}{2} + 1, \frac{S}{2} + \frac{3}{2}; -\frac{z^2}{4}\right).$$

$$\sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}} = -\frac{z^2}{S(S+1)} \mathcal{H}_S(z).$$

$$-i \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}} = \frac{i \theta^{S-1} z^2}{S(S+1)} \mathcal{H}_S(z) \equiv \frac{(-1)^{N+1} z^{S+1}}{S(S+1)} \mathcal{H}_S(z).$$

$$\Rightarrow \Im(e^{\theta} \Gamma(2N + 2, iz)) = \Gamma(S) \sin(z) - \frac{(-1)^{N+1} z^{S+1}}{S(S+1)} \mathcal{H}_S(z), \quad z \in \mathbb{R}$$

as desired. ■

For which a table of values is given as reference (??)

Substituting values we finally arrive at the general solution for the projector onto the truncated power series of $\text{sinc}(x)$:

$$\begin{aligned} \mathbb{P}_N[\text{sinc}](z) &= \frac{1}{\Gamma(S)z} \left[\Gamma(S) \sin(z) - \frac{i\theta^{S-1} z^2}{S(S+1)} \mathcal{H}_S(z) \right] \\ &= \text{sinc}(z) + \frac{(-1)^N z^S}{\Gamma(S+2)} \mathcal{H}_S(z) \end{aligned}$$

As expected:

$$R_N(z) := \text{sinc}(z) - \mathbb{P}_N^{\text{even}}[\text{sinc}](z) = \frac{(-1)^N z^S}{\Gamma(S+2)} \mathcal{H}_S(z)$$

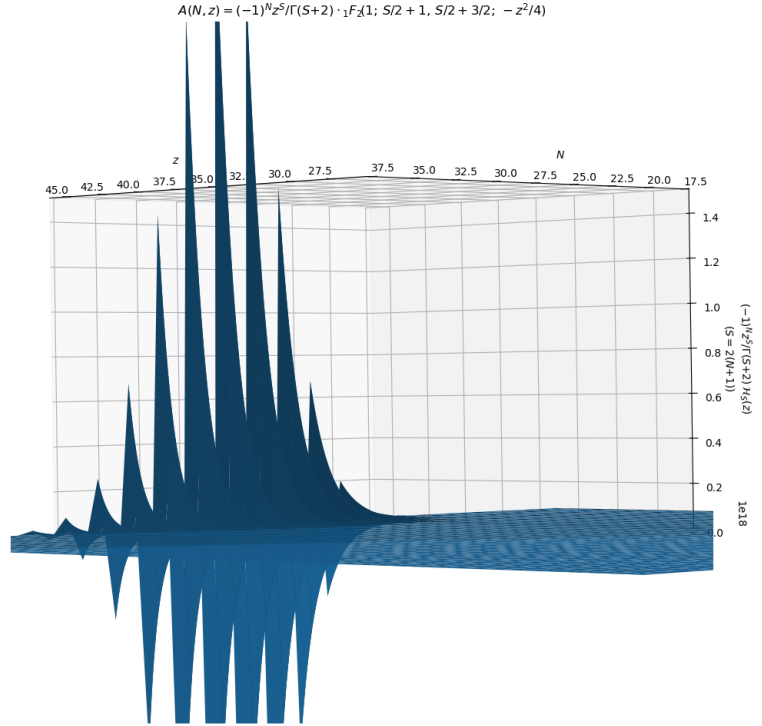


Figure 3.1: Plot of the remainder term $R_N(z)$

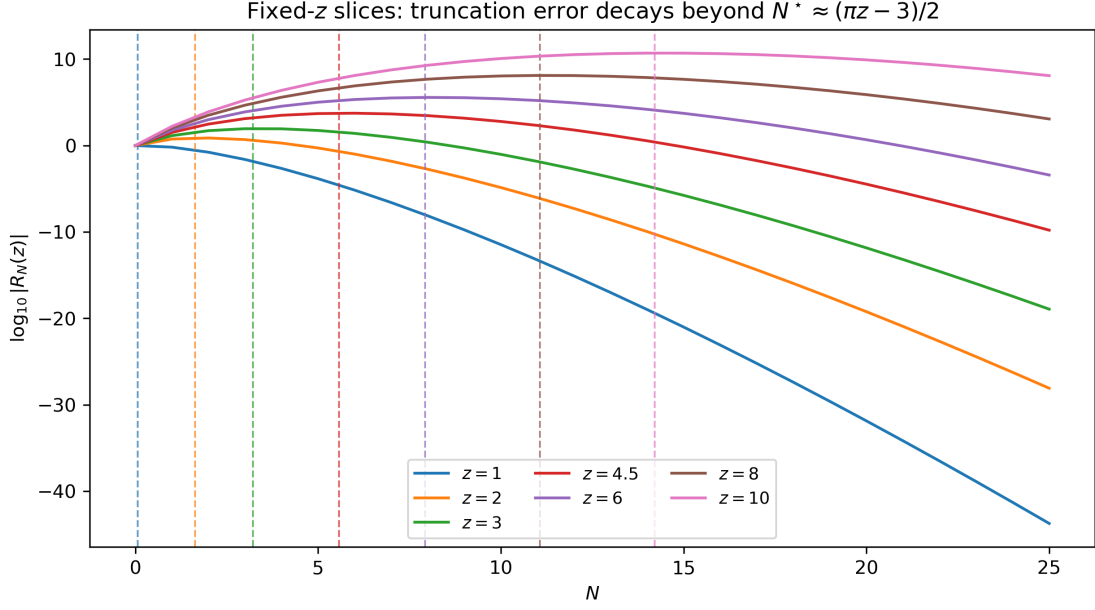


Figure 3.2: Logarithmic plot of the remainder term where the common definition given for sinc, $\text{sinc}(\pi z)$, is used. The decay point $N^* \approx \frac{\pi z - 3}{2} \propto z$ reflects the (trivial) fact that higher z needs a compensatory higher N for lower absolute error $|\mathbb{E}|$. **Note:** *Per(??) $\forall z \in a, |\mathbb{E}_N(z)| \gg \forall z \in b$ even though $|R_N(a)| < |R_N(b)|$ can hold true for $a \in [0, N^*), b \in [N^*, \infty)$*

3.2 Extending definition of the projector to accommodate generic $H(\omega)$

The projector derivation for $\text{sinc}(z)$ in Section 4.1.7 exploited a specific property: the sinc function’s Fourier transform is a rectangular pulse, which admits closed-form residue computation. However, audio signal chains require a much broader class of filter kernels—low-pass, high-pass, band-pass, parametric EQ, and nonlinear effects—each with distinct frequency responses $H(\omega)$.

The key insight (developed in Section 2.4) is that *any analytic frequency response* $H(\omega)$ can be incorporated into the projection framework via contour integration, providing a unified computational method for evaluating sensitivity matrices across arbitrary filter types. This generalization is essential for the adaptive FPGA architecture: as the signal chain’s module graph evolves (e.g., user adds a resonant filter or switches from low-pass to band-pass), the system must rapidly recompute sensitivities to update bit allocations without redesigning the entire hardware pipeline.

This section establishes four design principles that motivate the generic kernel $H(\omega)$:

1. **Residue-based evaluation:** Any analytic $H(\omega)$ can be evaluated systematically via its residues, enabling efficient hardware implementation through precomputed pole locations and CORDIC oscillators (Section 4.2.1).
2. **Dual error metrics:** The complex-valued projector encodes both magnitude

and phase error, which are critical for audio applications where phase distortion is perceptually significant (Section 4.1.3).

3. **Composition for coefficient optimization:** Choosing $H(\omega)$ as $H(\omega) \circ p(z)$ (composition) minimizes per-term error in the power series itself, optimizing the polynomial approximation quality (useful for fixed-kernel designs).
4. **Convolution for application-specific weighting:** Choosing $H(\omega)$ as $H(\omega) \circledast p(z)$ (convolution) minimizes error relative to the target application's frequency response, enabling adaptive filtering where the kernel changes with signal characteristics (central to the adaptive signal chain goal).

The remainder of this section formalizes these principles and derives the general projector formula $\mathbb{P}_N^{\text{even}}[H](z)$ for arbitrary analytic kernels, laying the groundwork for the wavelet extension in Section 4.4.

3.2.1 Deriving the support kernel $H(\omega)$

Theorem (Support kernel). For a function in the z -domain $H(z)$ derived via the bilinear transform from a function in the s -domain, the filtered projector is:

$$\mathbb{P}_N[\text{sinc} * h](z) = \frac{1}{2\pi i} \oint \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} d\omega + \frac{z}{2\pi} \oint \frac{S_{\text{odd}}(\omega) \cdot H_{a,\text{odd}}(\omega)}{\omega^2 + z^2} d\omega$$

where $H_a(\omega) = \mathcal{L}\{h_a(t)\}(s)|_{s=\omega}$, $H_{a,\text{even}}(\omega) = [H_a(\omega) + H_a(-\omega)]/2$, and $H_{a,\text{odd}}(\omega) = [H_a(\omega) - H_a(-\omega)]/2$.

Proof. Starting from the even and odd projector integrals with the substitution $\omega = iz/\zeta$:

$$\begin{aligned} \mathbb{P}_N^{\text{even}}[\text{sinc} * h](z) &= \frac{1}{2\pi i} \oint \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} d\omega \\ \mathbb{P}_N^{\text{odd}}[\text{sinc} * h](z) &= \frac{z}{2\pi} \oint \frac{S_{\text{odd}}(\omega) \cdot H_{a,\text{odd}}(\omega)}{\omega^2 + z^2} d\omega \end{aligned}$$

The full projector is their sum: $\mathbb{P}_N[\text{sinc} * h](z) = \mathbb{P}_N^{\text{even}}[\text{sinc} * h](z) + \mathbb{P}_N^{\text{odd}}[\text{sinc} * h](z)$.

Pole structure. The integrand has poles from:

- $\omega = \pm iz$ from $\omega^2 + z^2 = 0$
- Poles $\{\omega_k\}$ of $H_a(\omega)$ (filter-dependent)

Contour selection and causality. For a causal filter with impulse response $h_a(t) = 0$ for $t < 0$, the region of convergence (ROC) of the Laplace transform [OppenheimDSP] $H_a(s) = \int_0^\infty h_a(t)e^{-st}dt$ is a right half-plane $\Re(s) > \sigma_0$ for some $\sigma_0 \in \mathbb{R}$. This guarantees:

- All poles of $H_a(s)$ satisfy $\Re(s) \leq \sigma_0$ (lie in or to the left of the ROC boundary)
- For stable filters, $\sigma_0 < 0$, so all poles are in the left half-plane (LHP)
- The analytic continuation $H_a(\omega)$ evaluated on the imaginary axis $\omega = i\omega_{\text{imag}}$ lies strictly within the ROC when $\omega_{\text{imag}} > \sigma_0$

Crucially, the original contour in the ζ -plane was $|\zeta| = r < 1$, which after substitution $\omega = iz/\zeta$ becomes a contour in the ω -plane. Since we're free to choose r in the original formulation, we can select r such that the transformed contour avoids all poles of $H_a(\omega)$.

For the case where $H_a(\omega)$ has poles on the imaginary axis (e.g., marginally stable filters), we deform the contour to avoid these poles [Ahlfors1979]:

- Use principal value integrals or semicircular indentations around poles on the contour
- For filters derived via bilinear transform [OppenheimDSP, Kailath2000] from stable analog prototypes, poles are strictly in the LHP, so no deformation is needed

Vanishing at infinity (Jordan's lemma). To apply the residue theorem, we must verify the integral over a large circular contour $|\omega| = R$ vanishes as $R \rightarrow \infty$ [Ahlfors1979, Stein2003]. Consider the even component:

$$\left| \oint_{|\omega|=R} \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} d\omega \right| \leq 2\pi R \cdot \max_{|\omega|=R} \left| \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} \right|$$

For the truncated series $S_{\text{even}}(\omega) = \sum_{k=0}^{2N} \frac{\omega^k}{(k+1)!}$, we have $|S_{\text{even}}(\omega)| \leq CR^{2N}$ for some constant C on $|\omega| = R$.

For a rational filter $H_a(\omega) = \frac{P(\omega)}{Q(\omega)}$ with $\deg(Q) > \deg(P)$ (proper transfer function), we have $|H_{a,\text{even}}(\omega)| \sim O(R^{\deg(P)-\deg(Q)})$ as $R \rightarrow \infty$.

The denominator satisfies $|\omega^2 + z^2| \geq R^2 - |z|^2$ for large R .

Therefore:

$$\max_{|\omega|=R} \left| \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} \right| \sim O(R^{2N+\deg(P)-\deg(Q)-2})$$

The contour integral vanishes as $R \rightarrow \infty$ provided:

$$2N + \deg(P) - \deg(Q) - 2 < -1 \iff \deg(Q) > \deg(P) + 2N + 1$$

For typical low-order filters with $\deg(Q) - \deg(P) \geq 1$ and moderate N , this condition is satisfied.

Residue theorem. With causality ensuring poles are in the LHP and Jordan's lemma verified, the integral equals $2\pi i$ times the sum of residues at all enclosed poles [Ahlfors1979]:

$$\mathbb{P}_N^{\text{even}}[\text{sinc} * h](z) = \sum_{\omega_k} \text{Res} \left[\frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2}, \omega_k \right]$$

$$\mathbb{P}_N^{\text{odd}}[\text{sinc} * h](z) = z \sum_{\omega_k} \text{Res} \left[\frac{S_{\text{odd}}(\omega) \cdot H_{a,\text{odd}}(\omega)}{\omega^2 + z^2}, \omega_k \right]$$

The residues depend on the specific form of $H_a(\omega)$ and can be computed using standard techniques [Stein2003]. ■

Corollary (Kernel proportional to $e^{a\omega}$ admits special decomposition). If $H_a(\omega) = e^{a\omega}$ (e.g., time-delay kernel with $a = -t_0$), then $H_{a,\text{even}}(\omega) = \cosh(a\omega)$ and $H_{a,\text{odd}}(\omega) = \sinh(a\omega)$.

extbfOdd projector (enclosing both poles $\omega = \pm iz$):

$$\mathbb{P}_N^{\text{odd}}[\text{sinc} * h](z) = \frac{z}{2\pi} \oint_C \frac{S_{\text{odd}}(\omega) \sinh(a\omega)}{\omega^2 + z^2} d\omega = -i \sin(az) \sum_{k=0}^{N-1} \frac{(-1)^k z^{2k}}{(2k+1)!}$$

extbfEven projector (enclosing both poles $\omega = \pm iz$):

$$\mathbb{P}_N^{\text{even}}[\text{sinc} * h](z) = \frac{1}{2\pi i} \oint_C \frac{S_{\text{even}}(\omega) \cosh(a\omega)}{\omega^2 + z^2} d\omega = \cos(az) \sum_{k=0}^N \frac{(-1)^k z^{2k}}{(2k+1)!}$$

For a time-delay $h_a(t) = \delta(t - t_0)$ with $a = -t_0$:

$\mathbb{P}_N[\text{sinc} * \delta(t - t_0)](z) = \begin{cases} \cos(t_0 z) \cdot \text{sinc}_N^{\text{even}}(z) - i \sin(t_0 z) \cdot \text{sinc}_N^{\text{odd}}(z), & z \in \mathbb{R} \\ \cosh(t_0 \omega) \cdot \text{sinc}_N^{\text{even}}(iz/\omega) - i \sinh(t_0 \omega) \cdot \text{sinc}_N^{\text{odd}}(iz/\omega), & z \in \mathbb{C} \end{cases}$	$z \in \mathbb{R}$ $z \in \mathbb{C}$
---	--

where the real case evaluates residues at purely imaginary poles $\omega = \pm iz$, while the complex case retains the general hyperbolic form.

Enclosing only one pole (e.g., $\omega = iz$) yields a single-sided (non-symmetric) response, useful for causal implementations.

Remark 3.2.1 (Trigonometric-hyperbolic duality for real z). When $z \in \mathbb{R}$ the poles are purely imaginary $\omega = \pm iz \in i\mathbb{R}$. Evaluating hyperbolic functions at imaginary arguments gives:

$$\cosh(az) = \cos(az), \quad \sinh(az) = i \sin(az).$$

Thus the residues at $\omega = \pm iz$ produce real trigonometric modulation of the truncated sinc series: the hyperbolic basis inside the contour becomes a trigonometric basis after residue evaluation. For complex z the hyperbolic form remains complex-valued and encodes both magnitude and phase error.

By Corollary 3.2.1, when the Laplace transform has exponential form $\mathcal{L}\{h_a(t)\}(s) = e^{as}$, the kernel decomposes into trigonometric basis functions modulating the truncated sinc series. This is particularly significant for time-delay systems $h_a(t) = \delta(t - t_0)$ (with $a = -t_0$), which arise in fractional delay filters, polyphase decompositions, and beamforming.

The choice of contour C determines properties of the realized transfer function (linear vs. asymmetric phase, causality, stability). For rational $H_a(\omega)$, each enclosed pole contributes a modal component enabling parallel modal synthesis in hardware.

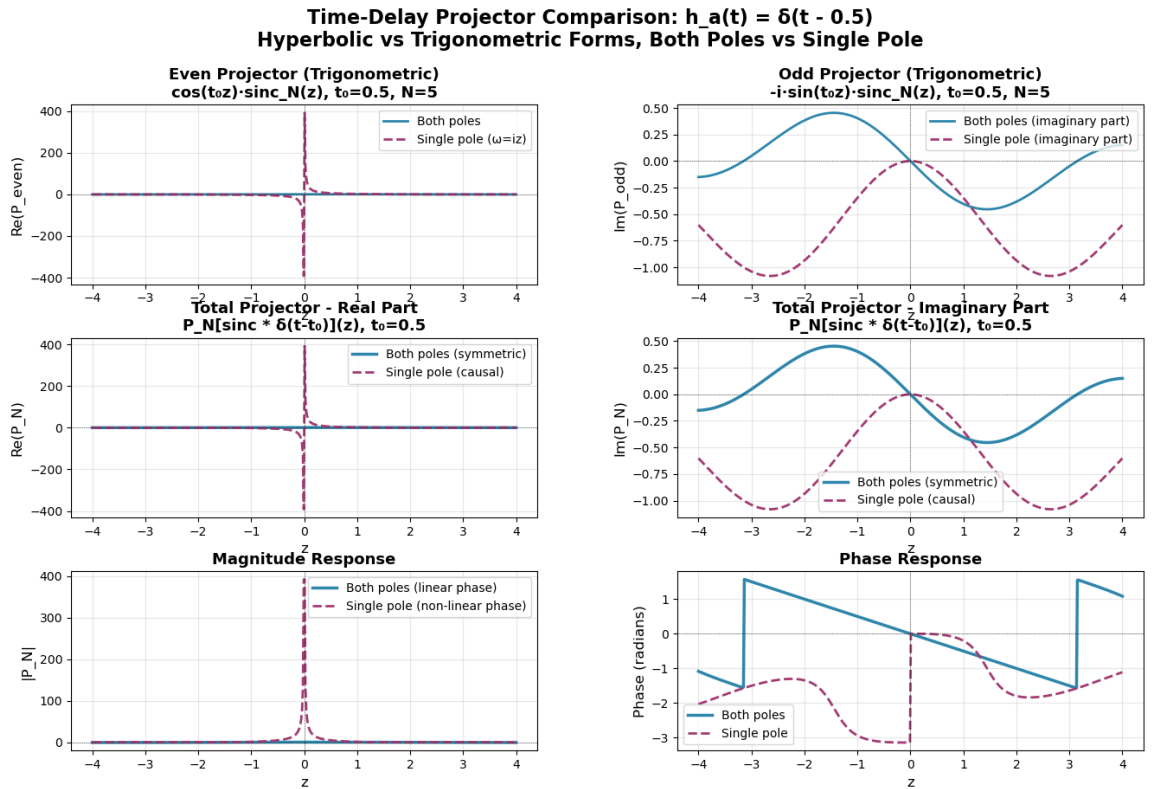


Figure 3.3: Graph of the projector for $H_a(\omega) = e^{-\omega}$ (unit time delay kernel) under different contour choices

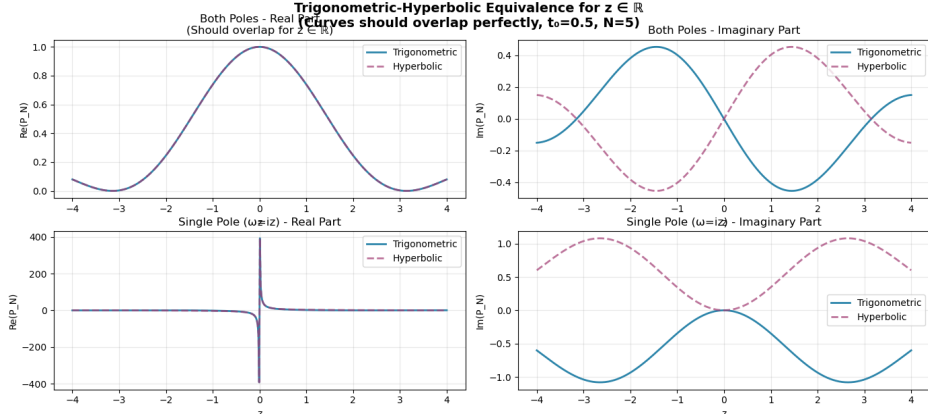


Figure 3.4: Equivalence between $\Re(\mathbb{P}_N[\text{sinc} * \delta(t - t_0)](z))$ for different contours

3.2.2 Dyadic Quantization of Projector Coefficients

Section 4.2 established that filtered projectors decompose into even and odd components via residue evaluation:

$$\mathbb{P}_N^{\text{even}}[f](z) = \sum_{k=0}^N c_k^{\text{even}} z^{2k}, \quad \mathbb{P}_N^{\text{odd}}[f](z) = \sum_{k=0}^{N-1} c_k^{\text{odd}} z^{2k+1},$$

where the coefficients c_k^{even} and c_k^{odd} depend on the residues of $H_a(\omega)$ at the poles $\omega = \pm iz$ (and any filter-specific poles). For the pure sinc case ($H_a(\omega) = 1$), we have $c_k^{\text{even}} = \frac{(-1)^k}{(2k+1)!}$.

To implement these projectors in hardware (see Section 2.4 for theoretical background), each coefficient must be quantized to a dyadic rational [Gustafsson2007]:

$$\tilde{c}_k = \frac{a_k}{2^{m_k}}, \quad a_k \in \mathbb{Z}, \quad m_k \in \mathbb{N}_0,$$

where m_k is the number of fractional bits allocated to coefficient k .

The *quantized projector* is then:

$$\tilde{\mathbb{P}}_N^{\text{even}}[\text{sinc}](z) = \sum_{n=0}^N \tilde{c}_n z^{2n} = \sum_{n=0}^N \frac{a_n}{2^{m_n}} z^{2n}. \quad (3.32)$$

The coefficient quantization error is $\epsilon_n = c_n - \tilde{c}_n$, satisfying

$$|\epsilon_n| \leq \frac{1}{2^{m_n+1}} \quad (3.33)$$

when a_n is chosen by rounding $c_n \cdot 2^{m_n}$ to the nearest integer [Widrow1996].

Total Error Decomposition

The complete approximation error combines *truncation error* (from projecting to degree $2N$) and *quantization error* (from dyadic rounding) [Kailath2000]:

$$\text{sinc}(z) - \tilde{\mathbb{P}}_N^{\text{even}}[\text{sinc}](z) = \underbrace{[\text{sinc}(z) - \mathbb{P}_N^{\text{even}}[\text{sinc}](z)]}_{\text{truncation: } R_N(z)} + \underbrace{[\mathbb{P}_N^{\text{even}}[\text{sinc}](z) - \tilde{\mathbb{P}}_N^{\text{even}}[\text{sinc}](z)]}_{\text{quantization: } Q_N(z)}. \quad (3.34)$$

From our earlier work, the truncation error is:

$$R_N(z) = \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z). \quad (3.35)$$

Remark. For $z \in \mathbb{R}$, the hypergeometric representation in Lemma 4.2.3.1 provides an alternative form that isolates magnitude error. For $z \in \mathbb{C} \setminus \mathbb{R}$, the full complex projector $\mathbb{P}_N[\text{sinc}](z) = \frac{\Im(e^{iz}\Gamma(2N+2, iz))}{z^{(2N+1)!}}$ must be used to capture both magnitude and phase error. See Appendix A.1 for detailed derivations.

The quantization error is:

$$Q_N(z) = \sum_{n=0}^N \epsilon_n z^{2n}, \quad |Q_N(z)| \leq \sum_{n=0}^N \frac{|z|^{2n}}{2^{m_n+1}}. \quad (3.36)$$

Incorporating Quantization into the Kernel Framework

From Corollary 3.2.1, we know that for exponential kernels $H_a(\omega) = e^{a\omega}$ (including time-delay systems), the projector evaluates to:

$$\mathbb{P}_N[\text{sinc} * h](z) = \cos(az) \sum_{k=0}^N \frac{(-1)^k z^{2k}}{(2k+1)!} - i \sin(az) \sum_{k=0}^{N-1} \frac{(-1)^k z^{2k}}{(2k+1)!}.$$

When we quantize the coefficients $c_k = \frac{(-1)^k}{(2k+1)!} \rightarrow \tilde{c}_k = \frac{a_k}{2^{m_k}}$, the quantized projector becomes:

$$\tilde{\mathbb{P}}_N[\text{sinc} * h](z) = \cos(az) \sum_{k=0}^N \tilde{c}_k z^{2k} - i \sin(az) \sum_{k=0}^{N-1} \tilde{c}_k z^{2k}.$$

The key insight is that **quantization errors in the coefficients $\epsilon_k = c_k - \tilde{c}_k$ are modulated by the kernel functions** $\cos(az)$ and $\sin(az)$. This means the frequency-dependent behavior of the kernel directly affects which coefficient quantization errors are most critical.

For a general analytic kernel $H_a(\omega)$, the quantized projector can be written via the residue formula from Section 4.2:

$$\tilde{\mathbb{P}}_N[f * h](z) = \sum_{\omega_k \text{ enclosed}} \text{Res} \left[\frac{S(\omega) \cdot H_a(\omega)}{\omega^2 + z^2}, \omega_k \right] \Big|_{\text{coefficients quantized}}, \quad (3.37)$$

where the series $S(\omega) = S_{\text{even}}(\omega)$ or $S_{\text{odd}}(\omega)$ contains the quantized coefficients $\tilde{c}_k$ instead of exact c_k .

Remark 3.2.2 (Quantization in real vs. complex domains). From the trigonometric-hyperbolic duality (Section 4.2), when $z \in \mathbb{R}$, the poles $\omega = \pm iz$ lie on the imaginary axis and the residues yield real trigonometric functions. In this case:

- Quantization errors ϵ_k affect both magnitude and phase of the trigonometric modulation
- For real-time audio (real-valued signals), we can optimize bit allocation based on real arithmetic only

- The factor of i in the odd projector means quantization of odd coefficients affects the imaginary (phase) component

For complex z (e.g., off-axis frequency response analysis), the hyperbolic form $\cosh(a\omega)$, $\sinh(a\omega)$ requires complex arithmetic, and quantization errors couple magnitude and phase in a more complex way. This suggests:

- Higher precision may be needed for coefficients that contribute to phase-sensitive regions
- The sensitivity matrix $\mathbf{S}$ should account for both real and imaginary parts when analyzing complex-valued signals

Sensitivity-Driven Bit Allocation with Quantization

Combining the sensitivity framework from Section 4.1.3 with dyadic quantization, the projected error becomes:

$$\mathcal{E}_{\text{total}} = \left| \int_0^{x_{\text{max}}} [R_N(z) + Q_N(z)] \overline{R(z)} dz \right|, \quad (3.38)$$

where $R(z)$ is the application-specific projector from earlier.

By the triangle inequality:

$$\mathcal{E}_{\text{total}} \leq \underbrace{\left| \int_0^{x_{\text{max}}} R_N(z) \overline{R(z)} dz \right|}_{\mathcal{E}_{\text{trunc}}} + \underbrace{\left| \int_0^{x_{\text{max}}} Q_N(z) \overline{R(z)} dz \right|}_{\mathcal{E}_{\text{quant}}}. \quad (3.39)$$

The quantization error term expands as:

$$\begin{aligned} \mathcal{E}_{\text{quant}} &= \left| \sum_{n=0}^N \epsilon_n \int_0^{x_{\text{max}}} z^{2n} \overline{R(z)} dz \right| \\ &\leq \sum_{n=0}^N |\epsilon_n| \cdot S_n = \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}, \end{aligned} \quad (3.40)$$

where $S_n = \left| \int_0^{x_{\text{max}}} z^{2n} \overline{R(z)} dz \right|$ is the sensitivity from (3.9).

The optimal bit allocation minimizes $\mathcal{E}_{\text{quant}}$ subject to $\sum_{n=0}^N m_n \leq B_{\text{total}}$ by the greedy algorithm (Section 4.1.5.3) [VetterliKovacevic1995], now incorporating both truncation point N (which determines $\mathcal{E}_{\text{trunc}}$) and per-coefficient precision m_n (which determines $\mathcal{E}_{\text{quant}}$). The complete optimization procedure is detailed in Appendix A.3.

3.3 Shannon Wavelet Basis and Orthonormal Extension

Sections 4.1–4.3 established a projection framework for approximating individual filter kernels (sinc functions, generic $H(\omega)$) with optimal bit allocation under hardware

constraints. However, real-world audio signals are *not* single-frequency tones—they are broadband signals with energy distributed across multiple octaves, where different frequency bands exhibit vastly different characteristics:

- **Bass frequencies (20–200 Hz):** High energy, perceptually critical, require high precision to avoid distortion.
- **Mid frequencies (200–2000 Hz):** Speech and most musical content; moderate precision adequate.
- **High frequencies (2–20 kHz):** Lower energy, often masked by louder sounds; can tolerate more quantization noise.

The adaptive FPGA signal chain envisioned in this thesis must allocate resources *per frequency band*, not uniformly across all frequencies. This requires decomposing the signal into localized frequency components—a task naturally suited to *wavelet bases* (see Section 2.4 for theoretical foundations).

The Shannon wavelet basis provides an orthonormal decomposition of $L^2(\mathbb{R})$ where each wavelet $\psi_{j,k}(t)$ is localized to a specific octave band $[2^j\pi, 2^{j+1}\pi]$ in frequency. Critically, each band’s bit allocation problem *decouples*—we can independently optimize the bit vector $\mathbf{m}^{(j)}$ for scale j based on the signal’s energy in that band, enabling fine-grained resource control.

This section extends the projector framework $\mathbb{P}_N$ to the Shannon wavelet setting, deriving:

1. A *per-scale quantized projector* $\tilde{\psi}_{j,k}^{(N,\mathbf{m})}(t)$ with independent bit allocation $\mathbf{m}^{(j)}$ at each scale j ,
2. A *wavelet sensitivity matrix* $\mathbf{S}_{\text{wavelet}}^{(j)}$ that guides bit allocation within each frequency band,
3. A *hardware architecture* based on CORDIC oscillators and dyadic coefficients, suitable for FPGA implementation.

The result is a multi-resolution bit allocation scheme where the FPGA can dynamically adjust precision across frequency bands in response to signal changes—precisely the adaptive behavior required for the modular audio effect chain described in the abstract.

3.3.1 Shannon Wavelet Construction

The Shannon scaling function $\phi(t)$ and mother wavelet $\psi(t)$ are defined via their Fourier transforms:

$$\widehat{\phi}(\omega) = \mathbf{1}_{[-\pi,\pi]}(\omega), \quad (3.41)$$

$$\widehat{\psi}(\omega) = \mathbf{1}_{[-2\pi,-\pi] \cup [\pi,2\pi]}(\omega). \quad (3.42)$$

In the time domain, these correspond to:

$$\phi(t) = \text{sinc}_\pi(t) = \frac{\sin(\pi t)}{\pi t}, \quad (3.43)$$

$$\psi(t) = \frac{\sin(2\pi t) - \sin(\pi t)}{\pi t} = \text{sinc}_\pi(t/2) \cos(\pi t). \quad (3.44)$$

The family of dilated and translated wavelets forms an orthonormal basis for $L^2(\mathbb{R})$:

$$\psi_{j,k}(t) = 2^{j/2} \psi(2^j t - k), \quad j, k \in \mathbb{Z}, \quad (3.45)$$

together with the scaling functions at a fixed resolution j_0 :

$$\phi_{j_0,k}(t) = 2^{j_0/2} \phi(2^{j_0} t - k), \quad k \in \mathbb{Z}. \quad (3.46)$$

3.3.2 Orthonormality and Parseval Identity

The Shannon wavelet basis satisfies the orthonormality conditions [Daubechies1992]:

$$\langle \phi_{j_0,k}, \phi_{j_0,\ell} \rangle = \delta_{k,\ell}, \quad (3.47)$$

$$\langle \psi_{j,k}, \psi_{j',k'} \rangle = \delta_{j,j'} \delta_{k,k'}, \quad (3.48)$$

$$\langle \phi_{j_0,k}, \psi_{j,\ell} \rangle = 0 \quad \text{for all } j \geq j_0. \quad (3.49)$$

Any function $f \in L^2(\mathbb{R})$ can be decomposed as [Mallat1989]:

$$f(t) = \sum_{k \in \mathbb{Z}} c_{j_0,k} \phi_{j_0,k}(t) + \sum_{j=j_0}^{\infty} \sum_{k \in \mathbb{Z}} d_{j,k} \psi_{j,k}(t), \quad (3.50)$$

where the coefficients are given by:

$$c_{j_0,k} = \langle f, \phi_{j_0,k} \rangle = \int_{\mathbb{R}} f(t) \overline{\phi_{j_0,k}(t)} dt, \quad (3.51)$$

$$d_{j,k} = \langle f, \psi_{j,k} \rangle = \int_{\mathbb{R}} f(t) \overline{\psi_{j,k}(t)} dt. \quad (3.52)$$

The Parseval identity ensures energy preservation [Walter1992]:

$$\|f\|_{L^2}^2 = \sum_k |c_{j_0,k}|^2 + \sum_{j=j_0}^{\infty} \sum_k |d_{j,k}|^2. \quad (3.53)$$

3.3.3 Projector Extension to the Wavelet Basis

The key observation is that each basis element is a dilated and translated version of either $\text{sinc}_\pi(t)$ or products of sinc functions. Thus, the quantized projector $\tilde{\mathbb{P}}_N$ can be applied to each basis element independently.

Quantized Projected Scaling Functions

For the scaling function at resolution j_0 and position k , define:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \tilde{\mathbb{P}}_N[\text{sinc}_\pi](2^{j_0}t - k), \quad (3.54)$$

where $\tilde{\mathbb{P}}_N[\text{sinc}_\pi]$ is the quantized projector from (3.32) with N terms and bit allocation vector $\mathbf{m} = (m_0, \dots, m_N)$.

From the dyadic quantization framework, this becomes:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \sum_{n=0}^N \frac{a_n}{2^{m_n}} [\pi(2^{j_0}t - k)]^{2n}, \quad (3.55)$$

where $a_n = \left\lfloor \frac{(-1)^n \cdot 2^{m_n}}{(2n+1)!} \right\rfloor$ (nearest integer rounding).

Quantized Projected Wavelet Functions

For the mother wavelet $\psi(t) = \text{sinc}_\pi(t/2) \cos(\pi t)$, we use the product structure with independent quantization:

$$\tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')} (t) = 2^{j/2} \tilde{\mathbb{P}}_N[\text{sinc}_\pi] \left(\frac{2^j t - k}{2} \right) \cdot \tilde{\mathbb{P}}_M[\cos](\pi(2^j t - k)), \quad (3.56)$$

where $\tilde{\mathbb{P}}_M[\cos]$ denotes the M -term quantized Taylor projection of the cosine with bit allocation $\mathbf{m}' = (m'_0, \dots, m'_M)$:

$$\tilde{\mathbb{P}}_M[\cos](x) = \sum_{m=0}^M \frac{b_m}{2^{m'_m}} x^{2m}, \quad b_m = \left\lfloor \frac{(-1)^m \cdot 2^{m'_m}}{(2m)!} \right\rfloor. \quad (3.57)$$

3.3.4 Approximation of Arbitrary Bandlimited Functions

Given a bandlimited function $f \in L^2(\mathbb{R})$ with Fourier support $\text{supp}(\hat{f}) \subset [-2^J\pi, 2^J\pi]$ for some finite J , the wavelet decomposition truncates naturally:

$$f(t) = \sum_{k=-K}^K c_{j_0,k} \phi_{j_0,k}(t) + \sum_{j=j_0}^J \sum_{k=-K_j}^{K_j} d_{j,k} \psi_{j,k}(t), \quad (3.58)$$

where K and K_j depend on the decay rate of the coefficients.

The quantized projected approximation is then:

$$\tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} (t) = \sum_{k=-K}^K c_{j_0,k} \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) + \sum_{j=j_0}^J \sum_{k=-K_j}^{K_j} d_{j,k} \tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')} (t). \quad (3.59)$$

3.3.5 Error Analysis in the Wavelet Domain with Quantization

The total approximation error now combines truncation, quantization, and basis decomposition:

$$\|f - \tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')}\|_{L^2}^2 = \sum_k |c_{j_0,k}|^2 \|\phi_{j_0,k} - \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}\|_{L^2}^2 + \sum_{j,k} |d_{j,k}|^2 \|\psi_{j,k} - \tilde{\psi}_{j,k}^{(N,M,\mathbf{m},\mathbf{m}')}\|_{L^2}^2. \quad (3.60)$$

For each basis element, the error decomposes as in (3.34). For the scaling function:

$$\begin{aligned} \|\phi_{j_0,k} - \tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}\|_{L^2}^2 &= 2^{j_0} \int_{\mathbb{R}} \left| R_N(\pi(2^{j_0}t - k)) + Q_N^{(\mathbf{m})}(\pi(2^{j_0}t - k)) \right|^2 dt \\ &\leq 2 \cdot 2^{j_0} \left[\int_{\mathbb{R}} |R_N(\pi u)|^2 du + \int_{\mathbb{R}} |Q_N^{(\mathbf{m})}(\pi u)|^2 du \right], \end{aligned} \quad (3.61)$$

where $R_N(z) = \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z)$ is the truncation remainder and $Q_N^{(\mathbf{m})}(z) = \sum_{n=0}^N \epsilon_n z^{2n}$ is the quantization error depending on bit allocation $\mathbf{m}$.

3.3.6 Sensitivity Matrix for the Wavelet Basis with Quantization

Extending the sensitivity framework to the wavelet domain with dyadic quantization, define the *wavelet sensitivity matrix* as:

$$\mathbf{S}_{\text{wavelet}} = \begin{bmatrix} \mathbf{S}_{\phi}^{(j_0)} & \mathbf{0} \\ \mathbf{0} & \text{block-diag}(\mathbf{S}_{\psi}^{(j)})_{j \geq j_0} \end{bmatrix}, \quad (3.62)$$

where each block $\mathbf{S}_{\phi}^{(j_0)}$ and $\mathbf{S}_{\psi}^{(j)}$ is itself a sensitivity matrix for the polynomial coefficients within that basis element.

For the scaling functions at resolution j_0 , the sensitivity of coefficient n is:

$$S_{\phi,n}^{(j_0)} = \max_k |c_{j_0,k}| \cdot 2^{j_0} \left| \int_{\mathbb{R}} u^{2n} \overline{R(\pi u)} du \right|, \quad (3.63)$$

where $R(z)$ is the application-specific projector from (3.7). This quantifies how quantization error ϵ_n in the n -th polynomial coefficient of $\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}$ affects the total projected error.

Similarly, for the wavelet functions at scale j :

$$S_{\psi,m}^{(j)} = \max_k |d_{j,k}| \cdot 2^j \left| \int_{\mathbb{R}} u^{2m} \cos(\pi \cdot 2^{j-j_0} u) \overline{R(\pi u)} du \right|. \quad (3.64)$$

The sensitivity-weighted bit allocation operates on the wavelet coefficient space via the composite error bound from (3.40), allocating precision based on:

1. The magnitude of the wavelet coefficients $|c_{j_0,k}|$ and $|d_{j,k}|$ (signal energy at each scale/position),

2. The projector order N and M at each scale (truncation error R_N, R_M),
3. The sensitivity $S_{\phi,n}^{(j_0)}$ and $S_{\psi,m}^{(j)}$ for each polynomial coefficient (quantization error weighting),
4. The total bit budget B_{total} distributed across all scales and positions.

The optimal bit allocation vector $\mathbf{m}^* = (\mathbf{m}_\phi, \{\mathbf{m}_\psi^{(j)}\}_{j \geq j_0})$ minimizes:

$$\sum_k |c_{j_0,k}| \sum_{n=0}^N \frac{S_{\phi,n}^{(j_0)}}{2^{m_{\phi,n}+1}} + \sum_{j,k} |d_{j,k}| \sum_{m=0}^M \frac{S_{\psi,m}^{(j)}}{2^{m_{\psi,m}^{(j)}+1}} \quad (3.65)$$

subject to the constraint:

$$\sum_{n=0}^N m_{\phi,n} + \sum_{j=j_0}^J \sum_{m=0}^M m_{\psi,m}^{(j)} \leq B_{\text{total}}. \quad (3.66)$$

3.3.7 Computational Advantages for Hardware Implementation

The Shannon wavelet decomposition offers several advantages for FPGA implementation [Andraka1998]:

Dyadic scaling. The dilation factor 2^j aligns naturally with binary arithmetic, allowing efficient bit-shifting operations rather than general multiplication [ParkhiUnbehauen1974].

Localized computation. Each wavelet $\psi_{j,k}$ has compact support in the frequency domain, meaning computations can be localized to specific frequency bands [Mallat1989].

This enables:

- *Parallel filter banks:* Different scales j can be processed in parallel [VetterliKovacevic1995].
- *Adaptive precision:* Higher frequencies (larger j) typically have smaller coefficients and can use reduced precision [Gustafsson2007].
- *Memory efficiency:* Only non-negligible coefficients need to be stored and processed [Burrus1998].

Fast wavelet transform. The Shannon wavelet admits a fast $O(N \log N)$ discrete wavelet transform via Mallat's algorithm [Mallat1989], which can be implemented using cascaded quadrature mirror filters (QMFs) [Vaidyanathan1993].

Direct frequency control. Because $\hat{\psi}_{j,k}(\omega) = 2^{-j/2} e^{-i\omega k/2^j} \mathbf{1}_{[-2^{j+1}\pi, -2^j\pi] \cup [2^j\pi, 2^{j+1}\pi]}(\omega)$, each wavelet scale j corresponds directly to an octave band $[2^j\pi, 2^{j+1}\pi]$ [Walter1992].

This enables:

- *Octave-band filtering:* Natural implementation of multi-rate filter banks [Crochiere1983].

- *Frequency-dependent bit allocation:* The bit allocation vector $\mathbf{m}$ can be chosen differently at each scale based on the frequency response requirements [JohnstonEtAl1997].
- *Adaptive FIR design:* The sensitivity matrix $\mathbf{S}_\psi^{(j)}$ guides coefficient quantization at each frequency band.

See Appendix A.4 for implementation details and hardware resource estimates.

3.3.8 Hardware-Efficient Kernel Selection with Quantization

For a given analytic kernel $H_a(\omega)$ (as introduced in Section 4.2), the quantized wavelet-projected version leverages the residue-based decomposition. From Corollary 3.2.1, exponential kernels decompose into trigonometric modulations of the sinc series. For the Shannon wavelet $\psi(t) = \text{sinc}_\pi(t/2) \cos(\pi t)$, this structure is preserved:

$$\tilde{\psi}_{j,k}^{(N,\mathbf{m})}(t) = 2^{j/2} \left[\tilde{\mathbb{P}}_N^{\text{even}}[\text{sinc}](2^j t - k) \right] \cdot \cos(\pi(2^j t - k)), \quad (3.67)$$

where $\tilde{\mathbb{P}}_N^{\text{even}}$ contains the quantized coefficients $\tilde{c}_k = \frac{a_k}{2^{m_k}}$.

The key observation is that **both the sinc projector and the cosine can be implemented via CORDIC oscillators** with dyadic coefficients [Volder1959, Andraka1998]:

- **Sinc component:** Polynomial in z with quantized dyadic coefficients $\tilde{c}_k$
- **Cosine component:** Can be generated via CORDIC rotation or lookup tables with dyadic interpolation
- **Product:** Fixed-point multiplication with controlled precision at each scale j

For kernels beyond exponential form, the residue formula (3.37) yields a modal decomposition where each pole contributes an oscillatory term. This maps to parallel hardware channels:

$$\tilde{f}_H^{(N,\mathbf{m})}(t) = \sum_{\text{poles } \omega_p} \text{Res}_{\omega=\omega_p} \left[\frac{\tilde{S}(\omega) H_a(\omega)}{\omega^2 + z^2} \right] \cdot e^{i\omega_p t}, \quad (3.68)$$

where $\tilde{S}(\omega)$ contains the quantized series coefficients. Each residue is a dyadic rational (after quantization), and each $e^{i\omega_p t}$ is generated by a CORDIC oscillator.

Bit allocation strategy. The bit vector $\mathbf{m}$ can be chosen to optimize [Parks1987, OppenheimSchafer2010]:

- **Passband flatness:** Allocate more bits to coefficients that contribute to passband ripple
- **Stopband attenuation:** Reduce bits for coefficients in stopbands where quantization noise is masked

- **Phase linearity:** For time-delay kernels (Corollary 3.2.1), allocate bits to preserve $\sin(t_0 z)$ and $\cos(t_0 z)$ accuracy
- **Pole sensitivity:** Coefficients near filter poles require higher precision due to resonance amplification

The sensitivity matrix $\mathbf{S}_{\text{wavelet}}^{(j)}$ from Section 4.4 guides this allocation by quantifying how coefficient errors propagate through the residue evaluation.

Synthesis of Sections 4.1–4.3. We have now developed all the analytical machinery needed for adaptive FPGA signal processing:

- **Section 4.1:** Bit allocation framework with sensitivity matrix $\mathbf{S}$ and greedy optimization.
- **Section 4.2:** Extension to arbitrary analytic kernels $H_a(\omega)$ via contour integration and residue evaluation, with explicit formulas for time-delay kernels and trigonometric/hyperbolic duality.
- **Section 4.3:** Dyadic quantization for fixed-point hardware, with residue-based modal synthesis enabling parallel CORDIC oscillator implementation.
- **Section 4.4:** Shannon wavelet basis enabling per-frequency-band bit allocation.

The remaining sections translate these theoretical results into a concrete hardware architecture. Section 4.5 describes the composite kernel architecture for wavelet-based filtering. Section 4.6 presents the complete dual bit-allocator/band-limiter system that adapts to signal characteristics in real time, fulfilling the adaptive signal chain vision outlined in the thesis abstract.

Chapter 4

Control theoretic model for dynamic FPGA resource balancing

4.1 Cyclostationary analysis

A signal with the property $x(t) = x(t + T) \forall t, T > 0$ is called *strictly periodic* (or *periodic pointwise*). A composition of periodic signals $h(t) = \sum_{i=0}^N x_i(t)$ is also periodic $\iff \frac{T_i}{T_j} \in \mathbb{Q} \forall i, j$ (i.e., the periods are commensurable). The period of $h(t)$ can therefore be described as

$$T = \text{lcm}(T_0, T_1, \dots, T_N).$$

Finite field computation of period. For FPGA implementation, the LCM computation can be performed efficiently using *finite field factorization* over a Galois field $\text{GF}(p^k)$. Represent each period T_i as a polynomial in $\text{GF}(p)[x]$, where p is chosen such that all period numerators and denominators (when expressed as rationals $T_i = n_i/d_i$) can be embedded in the field. The period LCM then reduces to computing the *polynomial LCM* via Berlekamp's algorithm [**berlekamp1970factoring, gathen2013modern**]:

1. Factor each period polynomial: $T_i(x) = \prod_j f_{i,j}(x)^{e_{i,j}}$ over $\text{GF}(p)$ using Berlekamp factorization (runtime $O(d^3 + d^2 \log p)$ for degree- d polynomials).
2. Compute LCM via exponent maximization: $\text{lcm}(T_0, \dots, T_N) = \prod_{f \text{ irreducible}} f(x)^{\max_i e_{i,f}}$.

Example: Consider three sinusoids with periods $T_0 = 3, T_1 = 4, T_2 = 6$. In $\text{GF}(7)$ (choosing prime $p = 7 > \max\{T_i\}$):

$$T_0 = 3 = 3 \quad (\text{irreducible in } \text{GF}(7)), \quad (4.1)$$

$$T_1 = 4 = 2^2 \quad (2 \text{ irreducible}), \quad (4.2)$$

$$T_2 = 6 = 2 \cdot 3 \quad (\text{factorization over } \text{GF}(7)). \quad (4.3)$$

Thus $\text{lcm}(3, 4, 6) = 2^2 \cdot 3 = 12$.

For the *quadratic case*, suppose periods arise from solving a characteristic equation $\lambda^2 + a\lambda + b = 0$ over $\text{GF}(p)$ (common in second-order filters). Berlekamp's algorithm factors this as:

$$\lambda^2 + a\lambda + b = (\lambda - r_1)(\lambda - r_2) \quad \text{over } \text{GF}(p),$$

where $r_1, r_2 \in \text{GF}(p)$ are the roots (if they exist). The factorization determines the minimal periods $T_1 = \text{ord}(r_1)$, $T_2 = \text{ord}(r_2)$ (the multiplicative orders of r_1, r_2 in $\text{GF}(p)^*$), and the composite period is $T = \text{lcm}(T_1, T_2)$.

Hardware advantage: Berlekamp factorization uses only addition, multiplication, and GCD operations in $\text{GF}(p)$ —all implementable with fixed-width arithmetic on FPGA. This avoids arbitrary-precision integer LCM, reducing DSP slice usage for cyclostationary period detection.

A *wide-sense cyclostationary* (WSC) signal is defined by

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t, \tau) = R_x(t+T, \tau),$$

where $R_x(t, \tau) = \mathbb{E}[x(t)x^*(t+\tau)]$. That is, both the mean $\bar{x}(t)$ and the autocorrelation function $R_x(t, \tau)$ are periodic in t . More generally, a signal is cyclostationary *iff* there exists some n th-order nonlinear transformation of $x(t)$ that produces a finite additive sinusoidal component [**<empty citation>**].

The definition of the *spectral parameter* [**<empty citation>**] is

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t) e^{-j2\pi\alpha t} dt.$$

For $x(t) = a \cos(2\pi\beta t + \theta)$:

$$x(t) = \frac{a}{2} (e^{j(2\pi\beta t + \theta)} + e^{-j(2\pi\beta t + \theta)}).$$

Substituting into the definition:

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{a}{2T} \left[e^{j\theta} \int_{-T/2}^{T/2} e^{j2\pi(\beta-\alpha)t} dt + e^{-j\theta} \int_{-T/2}^{T/2} e^{-j2\pi(\beta+\alpha)t} dt \right].$$

Recalling that

$$\int_{-T/2}^{T/2} e^{j2\pi f t} dt = \frac{\sin(\pi f T)}{\pi f},$$

$$\Rightarrow \hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} \frac{1}{T} \left[e^{j\theta} \frac{\sin(\pi(\beta-\alpha)T)}{\pi(\beta-\alpha)} + e^{-j\theta} \frac{\sin(\pi(\beta+\alpha)T)}{\pi(\beta+\alpha)} \right].$$

Equivalently,

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T \rightarrow \infty} [e^{j\theta} \text{sinc}_\pi((\beta-\alpha)T) + e^{-j\theta} \text{sinc}_\pi((\beta+\alpha)T)],$$

Finally, by inspection we obtain:

$$\hat{x}(\alpha) = \begin{cases} \frac{a}{2} e^{j\theta}, & \alpha = \beta, \\ \frac{a}{2} e^{-j\theta}, & \alpha = -\beta, \\ 0, & \text{otherwise.} \end{cases}$$

This demonstrates that the time-averaged inner product survives only when the test frequency α matches a sinusoidal component β of the signal.

Furthermore, we can interpret $\hat{x}(\alpha)$ as arising from the filtered projector framework developed in Chapter 3. Recall from Section 3.2 that for an analytic kernel $H_a(\omega)$, the filtered projector is evaluated via contour integration:

$$\mathbb{P}_N[\text{sinc} * h](z) = \frac{1}{2\pi i} \oint \frac{S_{\text{even}}(\omega) \cdot H_{a,\text{even}}(\omega)}{\omega^2 + z^2} d\omega + \frac{z}{2\pi} \oint \frac{S_{\text{odd}}(\omega) \cdot H_{a,\text{odd}}(\omega)}{\omega^2 + z^2} d\omega.$$

The spectral parameter $\hat{x}(\alpha)$ corresponds to choosing $H_a(\omega) = e^{j\omega T/2} \text{sinc}_\pi(\epsilon\omega)$ (an ideal bandpass kernel centered at frequency α), with the limit $\epsilon \rightarrow 0$ isolating the spectral component. Evaluating via residues at the poles $\omega = \pm iz$ yields:

$$\hat{x}(\alpha) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t) e^{-j2\pi\alpha t} dt = \text{Res}_{\omega=iz} \left[\frac{X(\omega) \cdot H_a(\omega)}{\omega^2 + \alpha^2} \right],$$

connecting cyclostationary spectral analysis to the contour-based framework. This interpretation enables runtime computation of spectral parameters using the same residue evaluation infrastructure developed for adaptive filter quantization (Chapter 3, Section 3.2), unifying the signal analysis (Chapter 4) and resource optimization (Chapter 3) pipelines.

4.1.1 Relation between Strict Periodicity and Cyclostationarity

Every strictly periodic signal is trivially wide-sense cyclostationary. Indeed, if $x(t) = x(t + T)$, then

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t, \tau) = \mathbb{E}[x(t)x^*(t+\tau)] = \mathbb{E}[x(t+T)x^*(t+T+\tau)] = R_x(t+T, \tau).$$

Thus:

$$x(t) \text{ periodic} \implies x(t) \text{ WSC (cyclostationary)}.$$

The converse does not hold: cyclostationary signals need not be strictly periodic pointwise. For example, an amplitude-modulated process $x(t) = m(t) \cos(2\pi f_c t)$ with random data $m(t)$ can be cyclostationary with period $1/f_c$, while not being pointwise periodic.

4.1.2 Group-Theoretic Interpretation

The set of transformations T can be understood as the action of a *symmetry group* G on the domain A . In the sine example, the relevant group is the *dihedral group* D_4 , which consists of rotations and reflections of a square [armstrong1988groups, gallian2021contemporary]. Each $\tau \in T$ corresponds to an element of D_4 acting on the domain of $\sin(x)$, with reflections accounting for sign changes and rotations corresponding to translations by $\pi/2$.

Thus, the concept of foldability is equivalent to requiring that the function f is invariant under the action of a group G , up to sign or phase factors. The *fundamental*

domain A_0 is then a representative region of A under this group action, and folding reduces storage by exploiting orbit representatives of G .

In group-theoretic language, the prior example is invariant under the group action up to a sign factor, i.e.,

$$\sin(x) = \epsilon(\tau) \sin(\tau(x)), \quad \epsilon(\tau) \in \{+1, -1\}, \quad \tau \in G.$$

The fundamental domain A_0 corresponds to a set of orbit representatives of G acting on A , and the recovery method reconstructs the full signal from this reduced domain.

4.1.3 Relationship between Argument Reduction and Cyclostationarity

Lemma 4.1.1 (Reducible function definition). A function $f : A \rightarrow B$ is *reducible* if and only if there exist a subset $A_0 \subseteq A$ and a set of transformations $T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

Where A_0 is called the *fundamental domain* & T , the set of transformations mapping from A_0 to A , the *recovery method*.

Argument-reducible functions are naturally related to cyclostationarity. A cyclostationary signal $x(t)$ has statistics that are periodic with period T . For an argument-reducible function like $\sin(x)$, the fundamental domain A_0 can be interpreted as a segment over which the first- and second-order statistics repeat under the action of G . In other words, argument reduction exploits the same periodic structure that cyclostationary analysis detects: the mean and autocorrelation within A_0 are sufficient to reconstruct the signal statistics over the entire domain.

Thus, argument reducibility can be seen as a deterministic, structural analogue of cyclostationarity: instead of averaging over time to detect periodic statistics, we use group-theoretic transformations to reconstruct the full signal from a reduced representative domain.

Let $f : A \rightarrow B$ be a reducible function with fundamental domain $A_0 \subseteq A$ and recovery transformations $T = \{\tau_i : A \rightarrow A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

This implies that the function f is invariant under the group action of G generated by T , and the entire domain A can be reconstructed from A_0 using the transformations in T .

Now, consider the autocorrelation function of f :

$$R_f(x, \Delta) = \mathbb{E}[f(x)f^*(x + \Delta)].$$

Since $f(x) = f(\tau(x))$ for some $\tau \in T$, we have

$$R_f(x, \Delta) = \mathbb{E}[f(\tau(x))f^*(\tau(x + \Delta))].$$

Because $\tau(x), \tau(x + \Delta) \in A_0$ (up to another element of the group), all evaluations of $R_f(x, \Delta)$ are determined entirely by the behavior on A_0 . Therefore,

$$R_f(x, \Delta) = R_f(\tau(x), \Delta), \quad \forall x \in A, \tau \in T, \tau(x) \in A_0.$$

Thus, the fundamental domain A_0 captures all statistical information of the function f , analogous to how the fundamental domain in cyclostationary signal processing captures the periodic statistical properties of a signal.

Projector-Based Detection of Group Structure in Wavelet Synthesis

When synthesizing waveforms using an orthonormal basis such as the Shannon wavelet mother function $\psi(t) = \text{sinc}(t)$ (Chapter 3, Section 3.4), the *filtered projector* framework from Chapter 3, Section 3.2 provides a direct mechanism to detect both the group action G and the recovery transformations T .

Synthesis via Orthonormal Expansion Suppose we synthesize a target waveform $f(t)$ using the Shannon wavelet basis:

$$f(t) = \sum_{j \in \mathbb{Z}} \sum_{k \in \mathbb{Z}} c_{j,k} \psi_{j,k}(t), \quad \psi_{j,k}(t) = 2^{j/2} \psi(2^j t - k),$$

where coefficients $c_{j,k} = \langle f, \psi_{j,k} \rangle$ are computed via the filtered projector:

$$c_{j,k} = \int_{-\infty}^{\infty} f(t) \overline{\psi_{j,k}(t)} dt = \mathbb{P}_{\psi_{j,k}}[f],$$

with $\mathbb{P}_{\psi_{j,k}}$ denoting projection onto the subspace spanned by $\psi_{j,k}$. For functions with argument reduction symmetry, these coefficients exhibit *algebraic structure* that reveals the underlying group G .

Group Action Detection via Coefficient Symmetries **Key observation:** If $f(t)$ is invariant under transformations $\tau \in T$ (up to phase factors), then the wavelet coefficients satisfy:

$$c_{j,k} = \epsilon(\tau) \cdot c_{j, \tau_k(k)}, \quad \epsilon(\tau) \in \{\pm 1, e^{i\theta}\},$$

where $\tau_k(k)$ denotes the action of τ on the discrete index k at scale j . This coefficient symmetry is *detectable* by analyzing the filtered projector integrals.

Example: Sine function. For $f(t) = \sin(2\pi t)$ with period $T = 1$, the symmetry group is D_4 (dihedral group). The recovery transformations are:

$$\tau_1(t) = t, \quad (\text{identity}), \quad (4.4)$$

$$\tau_2(t) = -t, \quad (\text{reflection, introduces sign flip}), \quad (4.5)$$

$$\tau_3(t) = \frac{1}{2} - t, \quad (\text{quarter-period shift} + \text{reflection}), \quad (4.6)$$

$$\tau_4(t) = t + \frac{1}{2}. \quad (\text{half-period translation}). \quad (4.7)$$

For Shannon wavelets at scale j (covering frequency band $[2^{j-1}\pi, 2^j\pi]$), the projector integrals become:

$$c_{j,k} = \int_{-\infty}^{\infty} \sin(2\pi t) \cdot 2^{j/2} \text{sinc}(2^j t - k) dt.$$

Using the contour integration method from Chapter 3, Section 3.2, we evaluate this via residue calculus. The key insight is that $\sin(2\pi t)$ has Fourier transform $S(\omega) = \frac{i}{2}[\delta(\omega - 2\pi) - \delta(\omega + 2\pi)]$, so:

$$c_{j,k} = 2^{j/2} \int_{-\infty}^{\infty} S(\omega) \widehat{\psi}(2^{-j}\omega) e^{i\omega k/2^j} \frac{d\omega}{2\pi},$$

where $\widehat{\psi}(\omega) = \mathbb{K}_{[-\pi, \pi]}(\omega)$ (brick-wall filter). The integral is nonzero only when $2\pi \in [2^{j-1}\pi, 2^j\pi]$, i.e., when $j = 1$. For $j = 1$:

$$c_{1,k} = \sqrt{2} \int_{-2\pi}^{2\pi} \frac{i}{2} [\delta(\omega - 2\pi) - \delta(\omega + 2\pi)] e^{i\omega k/2} \frac{d\omega}{2\pi} = \frac{i\sqrt{2}}{4\pi} [e^{i\pi k} - e^{-i\pi k}] = \frac{\sqrt{2}}{2\pi} \sin(\pi k).$$

Detected symmetry: The coefficients satisfy $c_{1,k} = -c_{1,-k}$ (antisymmetry under reflection $k \rightarrow -k$) and $c_{1,k+2} = -c_{1,k}$ (half-period shift), directly encoding the D_4 group action. The *fundamental domain* is $k \in \{0, 1\}$ (one quarter-period), from which all coefficients can be reconstructed via $\epsilon(\tau) \in \{+1, -1\}$.

Algorithmic Recovery Method Identification Given synthesized coefficients $\{c_{j,k}\}$ from the filtered projector, we algorithmically detect the group G and recovery method T :

1. **Correlation matrix construction:** For each scale j , compute the *coefficient autocorrelation*:

$$\rho_j(\Delta k) = \frac{\sum_k c_{j,k} c_{j,k+\Delta k}^*}{\sum_k |c_{j,k}|^2}.$$

This is the discrete analogue of $R_f(x, \Delta)$ from Section 4.1.3. Peaks in $|\rho_j(\Delta k)|$ reveal candidate periodicities.

2. **Galois field factorization for period detection:** The dominant periodicity Δk_0 (where $|\rho_j(\Delta k_0)| \approx 1$) corresponds to the fundamental period T in index space. Using the finite field LCM method from Section 4.1 (Berlekamp factorization), decompose Δk_0 into irreducible factors over $\text{GF}(p)$. Each factor corresponds to a generator of the cyclic subgroup of G .

3. **Phase factor extraction:** Compute the coefficient ratio:

$$\epsilon_j(\Delta k) = \frac{c_{j,k+\Delta k}}{c_{j,k}}, \quad (\text{for } c_{j,k} \neq 0).$$

If $|\epsilon_j(\Delta k)| \approx 1$ for all k , the transformation $\tau : k \rightarrow k + \Delta k$ is in T with phase factor $\epsilon(\tau) = \arg(\epsilon_j(\Delta k))$. For real-valued functions, $\epsilon(\tau) \in \{+1, -1\}$ (parity symmetry).

4. **Fundamental domain extraction:** The fundamental domain A_0 in index space is the minimal set $\{k_0, k_1, \dots, k_{N-1}\}$ such that all other coefficients $c_{j,k}$ can be expressed as:

$$c_{j,k} = \epsilon(\tau_{k \rightarrow k'}) \cdot c_{j,k'}, \quad k' \in A_0, \quad \tau_{k \rightarrow k'} \in T.$$

This is solved via *orbit decomposition* under the detected group G (computed using the Schreier-Sims algorithm [seress2003permutation] for permutation groups).

Connection to Filtered Projector Residue Structure The filtered projector from Chapter 3, Section 3.2, evaluates coefficients via contour integrals:

$$c_{j,k} = \frac{1}{2\pi i} \oint_{\mathcal{C}} \frac{S(\omega) \hat{\psi}(2^{-j}\omega) e^{i\omega k/2^j}}{\omega^2 - \omega_0^2} d\omega,$$

where $S(\omega) = \mathcal{F}[f(t)]$ and $\mathcal{C}$ is a contour enclosing the singularities of $S(\omega)$. For argument-reducible functions, the singularities of $S(\omega)$ exhibit *symmetry* under the dual group $\hat{G}$ (Pontryagin duality). Specifically:

- **Reflection symmetry** ($\tau(t) = -t$) $\implies S(-\omega) = S(\omega)$ (even spectrum) or $S(-\omega) = -S(\omega)$ (odd spectrum).
- **Translation symmetry** ($\tau(t) = t + T$) $\implies S(\omega)$ has support only at integer multiples of $2\pi/T$ (comb spectrum).

The residue calculus then yields coefficient relations. For the sine example, $S(\omega) = \frac{i}{2}[\delta(\omega - 2\pi) - \delta(\omega + 2\pi)]$ has *antisymmetric poles*, forcing $c_{j,k} = -c_{j,-k}$ (detected in step 3 above). More generally, the *residue signature*:

$$\text{Res}_{\omega=\omega_n} \left[\frac{S(\omega) e^{i\omega k}}{\omega^2 - \omega_0^2} \right] \propto e^{i\omega_n k},$$

encodes the group generators via the phase factors $e^{i\omega_n T} = \epsilon(\tau)$ where τ is a translation by period T .

Practical Implementation for Wavelet-Based Synthesis In Algorithm 4.1 (Section 4.2.5), the offline training phase computes Shannon wavelet coefficients $c_{j,k}$ for representative signals. We now augment this with *group action detection*:

- 1a. Compute coefficient autocorrelations $\rho_j(\Delta k)$ per scale ($O(N \log N)$ via FFT).
- 1b. Apply Berlekamp factorization (Section 4.1) to detected periods Δk_0 over $\text{GF}(p)$.
- 1c. Extract phase factors $\epsilon(\tau)$ and construct group table for G .
- 1d. Identify fundamental domain A_0 via orbit decomposition.
- 1e. Store only coefficients $\{c_{j,k} \mid k \in A_0\}$ and recovery table $\{(\tau, \epsilon(\tau)) \mid \tau \in T\}$.

At synthesis time (runtime execution), reconstruct full coefficient set via:

$$c_{j,k} = \epsilon(\tau_{k'}) \cdot c_{j,k'}, \quad k' = \arg \min_{k'' \in A_0} |k - \tau_{k''}^{-1}(k'')|,$$

reducing memory by factor $|G|$ (e.g., $4\times$ for D_4 symmetry in sine/cosine). This directly implements the argument reduction framework from Chapter 5, Section 5.1, within the Koopman-MPC resource optimization pipeline.

Hardware advantage: On FPGA, the group table $\{(\tau, \epsilon(\tau))\}$ is implemented as a small BRAM lookup (typically < 1 Kb for common symmetries), while coefficient storage in main BRAM is reduced by factor $|G|$. The Berlekamp factorization (Section 4.1) runs in hardware using Galois field arithmetic, enabling real-time period detection for adaptive synthesis.

4.2 Koopman Operator Framework for DSP Resource Prediction

The cyclostationary analysis developed in Section 4.1 provides a statistical characterization of periodic workloads, but does not directly address the *dynamics* of how DSP modules transform signals over time. To bridge this gap, we employ the *Koopman operator* framework [Koopman1931, Budisic2012, Mauroy2020Book], which represents nonlinear dynamical systems as linear infinite-dimensional operators on spaces of observable functions.

4.2.1 Motivation: From Signal Analysis to Resource Prediction

Consider a modular audio signal chain comprising filters, nonlinear processors (distortion, compression), and time-varying effects (chorus, phaser). Each module $\mathcal{M}_i$ can be viewed as a dynamical system:

$$\mathbf{x}_{k+1} = \mathcal{F}_i(\mathbf{x}_k, u_k), \quad y_k = \mathcal{H}_i(\mathbf{x}_k),$$

where $\mathbf{x}_k \in \mathbb{R}^n$ is the internal state (e.g., filter delay line, envelope detector state), u_k is the input signal, and y_k is the output. For nonlinear effects like tube saturation or dynamic range compression, $\mathcal{F}_i$ is inherently nonlinear, making traditional linear systems analysis inadequate.

The Koopman operator [Mezic2005] provides a linearization by lifting the nonlinear dynamics to an infinite-dimensional space of observables. This enables:

1. **Linear prediction** of future signal characteristics from past observations [Williams2015, Proctor2016], crucial for anticipatory resource allocation.
2. **Spectral decomposition** into Koopman modes, which directly correspond to frequency-band activity—precisely the structure exploited by the Shannon wavelet basis (Chapter 3, Section 3.4).

3. **Data-driven identification** via Dynamic Mode Decomposition (DMD) [Schmid2010], avoiding the need for explicit nonlinear system models.

Recent work demonstrates the efficacy of Koopman methods for audio applications: Huhtala et al. [Huhtala2024] model nonlinear guitar amplifier dynamics, while Díaz and Schlecht [DiazSchlecht2024] apply extended Koopman DMD to capture modal interactions in reverberant systems. We extend these ideas to *resource management*, using Koopman-linearized dynamics to predict FPGA workload demands.

4.2.2 Koopman Operator Theory

Let $\mathcal{F} : \mathcal{X} \rightarrow \mathcal{X}$ be a dynamical system on state space $\mathcal{X}$. The Koopman operator $\mathcal{K} : \mathcal{G} \rightarrow \mathcal{G}$ acts on the space of observables $\mathcal{G} = \{g : \mathcal{X} \rightarrow \mathbb{C}\}$ by composition:

$$(\mathcal{K}g)(\mathbf{x}) = g(\mathcal{F}(\mathbf{x})).$$

Key property: While $\mathcal{F}$ may be nonlinear, $\mathcal{K}$ is *linear* by construction [Koopman1931]:

$$\mathcal{K}(\alpha g_1 + \beta g_2) = \alpha \mathcal{K}g_1 + \beta \mathcal{K}g_2, \quad \forall \alpha, \beta \in \mathbb{C}, \quad g_1, g_2 \in \mathcal{G}.$$

For continuous-time systems $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$, the infinitesimal generator $\mathcal{L}$ of the Koopman semigroup satisfies [Mezic2005]:

$$\mathcal{L}g = \nabla g \cdot \mathbf{f}(\mathbf{x}).$$

4.2.3 Connection to Shannon Wavelet Decomposition

Clarification: The Koopman operator framework developed here is *independent* of the cyclostationary analysis in Section 4.1. While cyclostationarity characterizes *statistical periodicity* in signal ensembles, the Koopman approach models *deterministic dynamics* of individual DSP modules. Both perspectives contribute to resource prediction: cyclostationary analysis detects periodic workload patterns, while Koopman operators predict transient responses to signal changes.

The Koopman spectral decomposition exhibits a profound connection to the Shannon wavelet framework developed in Chapter 3. Suppose the Koopman operator $\mathcal{K}$ admits a discrete spectrum with eigenfunctions $\{\varphi_j\}$ and eigenvalues $\{\lambda_j\}$:

$$\mathcal{K}\varphi_j = \lambda_j\varphi_j, \quad \lambda_j = e^{j\omega_j\Delta t},$$

where ω_j are the Koopman frequencies. An observable $g(\mathbf{x})$ can be expanded:

$$g(\mathbf{x}) = \sum_{j=0}^{\infty} v_j \varphi_j(\mathbf{x}), \quad v_j = \langle g, \varphi_j \rangle,$$

yielding the evolution:

$$g(\mathcal{F}^k(\mathbf{x})) = \mathcal{K}^k g(\mathbf{x}) = \sum_{j=0}^{\infty} v_j \lambda_j^k \varphi_j(\mathbf{x}) = \sum_{j=0}^{\infty} v_j e^{jk\omega_j\Delta t} \varphi_j(\mathbf{x}).$$

Wavelet interpretation. Each Koopman mode oscillates at frequency ω_j , analogous to the Shannon wavelet basis functions $\psi_{j,k}(t)$ which isolate octave bands $[2^j\pi, 2^{j+1}\pi]$ (Section 3.4.1). The Koopman eigenfunction $\varphi_j(\mathbf{x})$ encodes *which states* contribute to frequency ω_j , while the Shannon wavelet coefficient $\langle x(t), \psi_{j,k}(t) \rangle$ measures *signal energy* in that band.

This duality enables a unified framework:

1. **Koopman modes** $\rightarrow$ predict which frequency bands will be active based on system state $\mathbf{x}_k$.
2. **Shannon wavelets** $\rightarrow$ allocate bits per band (Chapter 3, Section 3.4.5) based on predicted energy $|v_j|^2$.
3. **Filtered projectors** (Chapter 3, Section 3.2) $\rightarrow$ quantize coefficients for kernels $H(\omega)$ derived from Koopman eigenfunctions.

Dyadic Quantization and Integer Arithmetic Optimization

A key design parameter in the Shannon wavelet mother function is whether to impose *dyadic quantization* on frequency band edges. Recall from Chapter 3 that the Shannon wavelet $\psi(t) = \text{sinc}(t)$ has ideal brick-wall spectrum support on $[-\pi, \pi]$. The multiresolution decomposition naturally yields octave bands $[\pi 2^j, \pi 2^{j+1}]$ for scales $j \in \mathbb{Z}$.

When implementing this on FPGA, we can leverage the dyadic structure:

- **Dyadic mode enabled:** Band edges at powers-of-two frequencies $\{2^j\pi\}$ allow multiply-accumulate operations to be replaced with bit shifts: $x \cdot 2^j \rightarrow x \ll j$. The filtered projector kernels $H(\omega)$ from Chapter 3, Section 3.2 become particularly efficient when $\omega = 2^j\alpha$ for some base frequency α , as the contour integral residues simplify [Unser2000].
- **Dyadic mode disabled:** Arbitrary band partitions $[\omega_j, \omega_{j+1}]$ with non-power-of-two edges require full floating-point or high-precision fixed-point multipliers. This is necessary when Koopman spectral analysis reveals that the dominant frequencies $\{\omega_j\}$ do not align with dyadic grid—common in modulated effects (chorus, flanger) where pitch-shifting introduces irrational frequency ratios.

Adaptive strategy: The Koopman operator $\mathbf{K}$ learned via EDMD (Section 4.2.4) provides the eigenfrequencies $\{\omega_j\}$. If the dominant Koopman modes satisfy $|\omega_j - 2^{n_j}\pi| < \epsilon_{\text{snap}}$ for integer exponents $\{n_j\}$, we *enable dyadic quantization* and snap band edges to the nearest power-of-two. The Shannon wavelet transform then reduces to integer arithmetic:

$$c_{j,k} = \langle x(t), \psi_{j,k}(t) \rangle = \int_{-\infty}^{\infty} x(t) 2^{j/2} \psi(2^j t - k) dt, \quad (4.8)$$

$$\approx 2^{j/2} \sum_n x[n] \psi_{\text{discrete}}[2^j n - k], \quad (\text{bit-shift implementation}). \quad (4.9)$$

Conversely, if Koopman modes are incommensurate with dyadic grid, we revert to non-dyadic partitions and accept the higher computational cost for accuracy.

Wave synthesis implications: For waveform generation (Chapter 5), dyadic Shannon wavelets enable extremely fast additive synthesis. Given target spectrum $S(\omega)$ supported on $[2^j\pi, 2^{j+1}\pi]$, the synthesis becomes:

$$x(t) = \sum_j \sum_k c_{j,k} \psi_{j,k}(t), \quad c_{j,k} = 2^{-j/2} \int_{2^j\pi}^{2^{j+1}\pi} S(\omega) e^{j\omega k/2^j} d\omega.$$

With dyadic quantization, all frequency-domain multiplications $S(\omega) \cdot \omega$ reduce to shifts, and inverse FFT lengths are powers-of-two (enabling radix-2 Cooley-Tukey algorithms [Muller2006]). The Koopman framework predicts *which bands* j will require synthesis, allowing us to:

1. Preallocate BRAM for only active dyadic scales.
2. Configure CORDIC units [Meher2008] with bit-shift mode for 2^j multipliers.
3. Skip entire octaves where Koopman mode amplitudes $|v_j| < \epsilon_{\text{thresh}}$.

This creates a feedback loop: Koopman analysis $\rightarrow$ identifies dominant frequencies $\rightarrow$ determines dyadic/non-dyadic mode $\rightarrow$ optimizes Shannon wavelet implementation $\rightarrow$ measured FPGA resource usage $\rightarrow$ refines Koopman observables $\psi_{\text{DSP}}, \psi_{\text{LUT}}$.

Koopman $\omega_j \xrightarrow{\text{dyadic test}} \text{Mode selection} \xrightarrow{\text{synthesis}} \text{FPGA utilization} \xrightarrow{\text{measurement}} \text{Update } \{\alpha_j, \beta_j\} $

(4.10)

4.2.4 Finite-Dimensional Approximation via Extended DMD

Practical implementation requires finite-dimensional truncation. The Extended Dynamic Mode Decomposition (EDMD) [Williams2015] constructs an approximate Koopman operator $\mathbf{K} \in \mathbb{C}^{m \times m}$ from snapshot pairs $\{(\mathbf{x}_k, \mathbf{x}_{k+1})\}_{k=1}^N$ and a dictionary of observables $\{\psi_i(\mathbf{x})\}_{i=1}^m$:

$$\Psi(\mathbf{x}) = \begin{bmatrix} \psi_1(\mathbf{x}) \\ \vdots \\ \psi_m(\mathbf{x}) \end{bmatrix}, \quad \mathbf{K} = \Psi'(\Psi)^\dagger,$$

where $\Psi = [\Psi(\mathbf{x}_1), \dots, \Psi(\mathbf{x}_N)]$, $\Psi' = [\Psi(\mathbf{x}_2), \dots, \Psi(\mathbf{x}_{N+1})]$, and $(\cdot)^\dagger$ denotes the Moore-Penrose pseudoinverse.

For audio DSP, recent work [Huhtala2024] demonstrates that polynomial observables $\psi_i(\mathbf{x}) = x_1^{a_1} \cdots x_n^{a_n}$ with $a_1 + \cdots + a_n \leq d$ (degree- d monomials) effectively capture nonlinear audio effects with $m = O(n^d)$ observables. Brunton et al. [Brunton2016] show that for control applications, carefully chosen observables yield Koopman invariant subspaces of dimension $m \ll \infty$, enabling real-time computation.

4.2.5 Resource Prediction via Koopman-MPC Integration

The Koopman linearization enables Model Predictive Control (MPC) for resource allocation [Korda2018]. Given the finite-dimensional Koopman operator $\mathbf{K}$, the predicted state evolution is:

$$\Psi_{k+h|k} = \mathbf{K}^h \Psi_k,$$

where h is the prediction horizon. We define resource demand observables:

$$\psi_{\text{DSP}}(\mathbf{x}) = \sum_j \alpha_j |\langle x(t), \psi_{j,\cdot}(t) \rangle|^2, \quad (\text{DSP slice usage}), \quad (4.11)$$

$$\psi_{\text{LUT}}(\mathbf{x}) = \sum_j \beta_j N_j, \quad (\text{LUT count for band } j), \quad (4.12)$$

$$\psi_{\text{BRAM}}(\mathbf{x}) = \sum_j \gamma_j M_j, \quad (\text{BRAM for delay lines}), \quad (4.13)$$

where $\alpha_j, \beta_j, \gamma_j$ are learned coefficients mapping Shannon wavelet energies to hardware primitives.

Sensitivity-Guided Observable Selection for Resource Tradeoffs

The Koopman observables above predict *aggregate* resource usage, but do not directly inform *how* to implement each frequency band (DSP-heavy multiplication vs. LUT-intensive dyadic quantization). We now connect the sensitivity matrix $\mathbf{S}$ from Chapter 3 (Eq. 3.14) to Koopman observables, enabling the MPC controller to request different implementation strategies based on predicted workload dynamics.

Sensitivity-Aware Resource Observables Recall from Chapter 3 that the sensitivity matrix $\mathbf{S} \in \mathbb{R}^{(N+1) \times 2}$ encodes how quantization errors in polynomial coefficients c_n propagate to filter output error:

$$S_n = |I_n| = \left| \int_{-\pi}^{\pi} x^{2n} R(x) dx \right|,$$

where $R(x) = \widehat{H}(\omega)e^{-i\omega x}$ is the projection operator (Chapter 3, Section 3.1.9). High-sensitivity coefficients ($S_n \gg \bar{S}$) require more fractional bits m_n to maintain accuracy, directly increasing resource demands.

We augment the Koopman state vector with *per-band sensitivity observables*:

$$\psi_{\text{sens}}^{(j)}(\mathbf{x}) = \max_{n \in \mathcal{N}_j} S_n^{(j)}, \quad (\text{peak sensitivity in band } j), \quad (4.14)$$

$$\psi_{\text{entropy}}^{(j)}(\mathbf{x}) = - \sum_n \frac{S_n^{(j)}}{\|S^{(j)}\|_1} \log \frac{S_n^{(j)}}{\|S^{(j)}\|_1}, \quad (\text{sensitivity entropy}), \quad (4.15)$$

where $S_n^{(j)}$ is the sensitivity of coefficient n in Shannon wavelet scale j , and $\mathcal{N}_j = \{0, 1, \dots, N_j\}$ indexes the polynomial terms for that band. These observables encode:

- **Peak sensitivity** $\psi_{\text{sens}}^{(j)}$: Bands with high peak sensitivity require careful bit allocation; implementing them with *dyadic quantization* (Chapter 5, Section 5.2.4) trades LUT usage for reduced DSP count via bit-shift arithmetic [Abdi2024, Gustafsson2007].
- **Sensitivity entropy** $\psi_{\text{entropy}}^{(j)}$: Low entropy ($\psi_{\text{entropy}}^{(j)} \rightarrow 0$) indicates one or two coefficients dominate; these bands benefit from *sparse polynomial evaluation* using DSP slices for the dominant terms only. High entropy ($\psi_{\text{entropy}}^{(j)} \rightarrow \log N_j$) suggests uniform importance across coefficients, favoring *parallel LUT-based evaluation* of all terms [SinhaSrikanthan2014].

Implementation Strategy Selection via Koopman Prediction The MPC controller uses predicted values $\psi_{\text{sens}_{k+h|k}}^{(j)}$, $\psi_{\text{entropy}_{k+h|k}}^{(j)}$ (evolved via $\mathbf{K}^h$) to select per-band implementation strategies:

Table 4.1: Sensitivity-Driven Implementation Mode Selection

Condition	$\psi_{\text{sens}}^{(j)}$	$\psi_{\text{entropy}}^{(j)}$	Implementation Strategy
High precision req.	$> \tau_{\text{high}}$	Low (< 1.5)	Dyadic quantization + LUTs (Chapter 5, BMT algorithm) → High LUT, low DSP
Balanced load	$[\tau_{\text{low}}, \tau_{\text{high}}]$	Medium	Hybrid DSP+LUT (Horner’s scheme with DSP MACs) → Moderate LUT & DSP
Sparse spectrum	Any	Low (< 1.0)	DSP-only (dominant terms) (Sparse polynomial, ≤ 3 DSP slices) → Low LUT, high DSP
Dense/uniform	$< \tau_{\text{low}}$	High (> 2.5)	Parallel LUT evaluation (All terms via parallel adders) → High LUT, minimal DSP

Example: Suppose Koopman prediction indicates that at time $k+3$, frequency band $j = 5$ will have $\psi_{\text{sens}_{k+3|k}}^{(5)} = 42.7 > \tau_{\text{high}}$ (high peak sensitivity) and $\psi_{\text{entropy}_{k+3|k}}^{(5)} = 0.8$ (low entropy, dominated by coefficients c_2, c_3). The MPC controller preemptively:

1. Selects *dyadic quantization mode* for band $j = 5$ (Table 4.1, row 1).
2. Invokes the BMT algorithm (Chapter 5, Section 5.2.4) or uses precomputed dyadic polynomial coefficients to decompose $c_2 \approx \frac{a_2}{2^{m_2}}$, $c_3 \approx \frac{a_3}{2^{m_3}}$ with $m_2, m_3 \geq 10$ fractional bits.
3. Allocates LUT resources for bit-shift networks: $(x \ll m_2) + (x \ll (m_2 - 1)) + \dots$ to evaluate $c_2 x^4 + c_3 x^6$ without DSP multipliers.
4. Updates resource constraints: $\psi_{\text{DSP}}(\mathbf{x}_{k+3|k}) \leftarrow \psi_{\text{DSP}}(\mathbf{x}_{k+3|k}) - 2$ (freed DSP slices from band 5), $\psi_{\text{LUT}}(\mathbf{x}_{k+3|k}) \leftarrow \psi_{\text{LUT}}(\mathbf{x}_{k+3|k}) + 60$ (additional shift-add logic).

Closed-Loop Refinement The learned coefficients $\alpha_j, \beta_j, \gamma_j$ in the resource observables can be refined online using the measured sensitivity matrix $\mathbf{S}_{\text{measured}}$ (computed from actual FPGA timing/resource reports after synthesis):

$$\alpha_j^{\text{new}} = \alpha_j^{\text{old}} + \eta \left(\psi_{\text{DSP}}^{\text{actual}} - \psi_{\text{DSP}}^{\text{predicted}} \right) \frac{\partial \psi_{\text{DSP}}}{\partial \alpha_j},$$

where η is a learning rate. This creates a feedback mechanism (anticipated in Section 4.2.1, Eq. (4.10)): measured FPGA utilization $\rightarrow$ updates Koopman observable mappings $\rightarrow$ improves future resource predictions $\rightarrow$ refines sensitivity-driven mode selection.

Integration with MPC Formulation

The MPC optimization at time k solves:

$$\min_{\mathbf{m}_{k:k+H}^{(j)}} \sum_{h=0}^H \|\mathbf{e}_{k+h|k}\|^2 + \lambda \sum_{j,h} \left(m_{k+h}^{(j)} - m_{k+h-1}^{(j)} \right)^2$$

subject to:

$$\Psi_{k+h+1|k} = \mathbf{K} \Psi_{k+h|k}, \quad (4.16)$$

$$\sum_j \psi_{\text{DSP}}(\mathbf{x}_{k+h|k}) \leq C_{\text{DSP}}, \quad (4.17)$$

$$\sum_j \psi_{\text{LUT}}(\mathbf{x}_{k+h|k}) \leq C_{\text{LUT}}, \quad (4.18)$$

$$m_{\min}^{(j)} \leq m_{k+h}^{(j)} \leq m_{\max}^{(j)}, \quad (4.19)$$

where $\mathbf{e}_{k+h|k}$ is the predicted quantization error (from Chapter 3, Eq. 3.15), $\mathbf{m}^{(j)}$ is the bit allocation vector for Shannon wavelet scale j , and $C_{\text{DSP}}, C_{\text{LUT}}$ are FPGA resource limits. The regularization term $\lambda(\cdot)^2$ penalizes rapid reconfigurations to avoid excessive partial reconfiguration overhead [Bhandari2012].

Key advantage: Explicit MPC for embedded systems [Bemporad2002] precomputes control laws offline, enabling $O(1)$ runtime lookup. Combined with sparse Koopman identification [Kaiser2018], this yields real-time feasibility on FPGA fabric.

4.2.6 Implementation: From Cyclostationary Analysis to Koopman Modes

Algorithm 4.1 integrates the cyclostationary spectral analysis (Section 4.1) with Koopman-based prediction:

Algorithm 4.1: Koopman-Driven Adaptive Bit Allocation**1. Offline Training Phase:**

- (a) Collect snapshot pairs $\{(\mathbf{x}_k, \mathbf{x}_{k+1})\}$ from representative audio signals.
- (b) Compute Shannon wavelet coefficients $c_{j,k}$ per scale j (Chapter 3, Eq. 3.78).
- (c) Extract state vector: $\mathbf{x}_k = [c_{0,k}, c_{1,k}, \dots, c_{J,k}, \dot{c}_{0,k}, \dots]^\top$.
- (d) Construct observables $\Psi(\mathbf{x}_k)$ (polynomial basis up to degree $d = 3$).
- (e) Compute EDMD approximation: $\mathbf{K} = \Psi'(\Psi)^\dagger$.
- (f) Learn resource mappings $\alpha_j, \beta_j, \gamma_j$ via regression on measured FPGA utilization.
- (g) Precompute explicit MPC lookup tables [Bemporad2002].

2. Runtime Execution:

- (a) **Measure:** Current state $\mathbf{x}_k$ from Shannon wavelet transform.
- (b) **Predict:** $\Psi_{k+h|k} = \mathbf{K}^h \Psi_k$ for $h \in \{1, \dots, H\}$.
- (c) **Allocate:** Query precomputed MPC table with $\Psi_{k|k}$ to obtain $\mathbf{m}_k^*$.
- (d) **Reconfigure:** Update bit allocations per band (Chapter 3, Alg. 3.1) if $\|\mathbf{m}_k^* - \mathbf{m}_{k-1}\| > \epsilon_{\text{reconfig}}$.

This algorithm closes the loop between signal analysis (cyclostationary detection in Section 4.1), dynamics prediction (Koopman), and hardware optimization (Shannon wavelet bit allocation from Chapter 3), achieving the adaptive FPGA resource management envisioned in Chapter 1.

Chapter 5

Architectural Implementation and Hardware Optimization

This chapter synthesizes the theoretical frameworks developed in Chapters 3–4 into concrete FPGA architectures for efficient elementary function evaluation. We demonstrate how the sensitivity-driven bit allocation (Chapter 3) and Koopman-based workload prediction (Chapter 4) guide hardware design decisions through the $\arccos(x)$ case study, which exemplifies the complete optimization pipeline: range reduction, polynomial approximation via Berlekamp factorization, dyadic coefficient optimization, common subexpression elimination, and automatic parallelization.

5.1 Memory-Efficient Lookup via Range Reduction

The technique of *range reduction* (also known as argument reduction) exploits functional symmetries to reduce the domain over which a function must be tabulated or computed, directly addressing the memory constraints identified in Chapter 1. Consider $\sin(x)$, which has period 2π and exhibits symmetry allowing representation via a fundamental domain of width $\frac{\pi}{4}$ [Muller2006]. The recovery method:

$$T = \begin{cases} x, & 0 \leq x \leq \frac{\pi}{4}, \\ \pi - x, & \frac{\pi}{4} < x \leq \frac{\pi}{2}, \\ x - \pi, & \frac{\pi}{2} < x \leq \frac{3\pi}{2}, \\ 2\pi - x, & \frac{3\pi}{2} < x \leq 2\pi \end{cases}$$

$$f(x) = \begin{cases} f \circ \tau(x), & 0 \leq x \leq \pi, \\ -f \circ \tau(x), & \pi < x \leq 2\pi \end{cases}$$

enables storage reduction by a factor of 4 compared to storing the full period. Alternatively, for a fixed memory budget, this yields $4\times$ finer granularity [Andraka1998,

Volder1959]. In general, the *precision-memory tradeoff* is characterized by the ratio M/G , where the memory reduction factor is $M = P/|A_0|$ and P is the function period.

Note 5.1.1. The computational complexity of the recovery method $f \circ \tau(x)$ must be considered against the memory savings. The tradeoff is not necessarily linear: $\tau(x) \not\propto M$. For example, the function $\arccos(x)$ (analyzed in Section 5.2.1) requires computing $\sqrt{x+1}$ for quarter-domain range reduction over $\arccos : [0, \frac{1}{\sqrt{2}}] \rightarrow [0, \pi]$, introducing polynomial approximation overhead [**Muller2006**].

5.1.1 Algorithmic Framework for Reducing Memory pressure

For an arbitrary periodic or quasi-periodic signal $x(t)$, we propose the following procedure:

1. **Spectral decomposition:** Decompose $x(t)$ into harmonic components via autocorrelation or spectral analysis [**OppenheimSchafer2010**]. For computational efficiency, autocorrelation-based methods are preferred over full FFT computation on resource-constrained FPGA [**SinhaSrikanthan2014**].
2. **Period estimation:** Apply autocorrelation function (ACF) analysis to the fundamental harmonic, estimating the period within a specified signal-to-noise ratio (SNR) tolerance [**Das2017**]. This is critical for audio signals with time-varying pitch or noisy measurements.
3. **Iterative refinement:** Refine the period estimate using spectral leakage minimization if underestimated, or peak prominence analysis to adjust lag windows [**OppenheimSchafer2010**].
4. **Argument reduction test:** Determine if the signal is *argument reducible* [**Muller2006**] by verifying the existence of a fundamental domain A_0 and transformation set T such that

$$\forall x \in A, \exists \tau \in T : \tau(x) \in A_0 \text{ and } f(x) = f \circ \tau(x).$$

This is tractable for signals with known symmetries (trigonometric functions, piecewise monotonic waveforms) but computationally intensive for arbitrary signals [**Muller2006**].

5. **Recovery method construction:** Construct an efficient mapping $\tau : A \rightarrow A_0$, balancing the precision-memory tradeoff according to resource constraints from the Koopman-MPC framework [**Bhandari2012**].
6. **Hardware synthesis:** Implement $\tau(x)$ in synthesizable RTL for FPGA deployment, ensuring computational complexity does not exceed memory access latency [**SinhaSrikanthan2014**, **Silva2016**].

Scope limitations: Steps (1)–(3) are well-established in DSP literature [OppenheimSchafer2010] and have been implemented in prior work [Das2017, Syfala2024]. Steps (4)–(6), however, present significant challenges for unrestricted signal classes. This chapter focuses on *elementary transcendental functions* (trigonometric, inverse trigonometric, exponential) that exhibit exploitable symmetries and are ubiquitous in audio synthesis and effects processing [Syfala2024]. The $\arccos(x)$ function serves as our primary case study, demonstrating how range reduction interacts with the polynomial optimization techniques from Chapter 3.

5.2 Case Study: $\arccos(x)$ via Quarter-Domain Range Reduction

The inverse cosine function $\arccos(x)$ provides an instructive example unifying all components of our framework: range reduction (Section 5.1), sensitivity-based polynomial approximation (Chapter 3), Galois field factorization, and hardware-aware scheduling. This example is dually motivated: first, the Taylor series naturally admits a dyadic rational structure (Chapter 3.3), simplifying computation significantly; second, while the symmetry-based range reduction is straightforward, the recovery transformation $\tau(x)$ involves a square root—a transcendental operation requiring the polynomial approximation techniques developed in Chapter 3 [Muller2006].

5.2.1 Quarter-Domain Identity and Polynomial Approximation

The quarter-domain identity for $\arccos(x)$ exploits the half-angle formula [NIST:DLMF]:

$$\tau(x) = \frac{1}{\sqrt{2}}\sqrt{x+1}, \quad (5.1)$$

$$\arccos(x) = 2 \arccos(\tau(x)) = 2 \arccos\left(\frac{1}{\sqrt{2}}\sqrt{x+1}\right). \quad (5.2)$$

To avoid direct hardware square root computation (which typically requires iterative CORDIC algorithms [Volder1959, Andraka1998] or Newton-Raphson iteration [Muller2006]), we employ a third-order Taylor series expansion centered at $x = 0$:

$$\sqrt{x+1} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{x^3}{16} + O(x^4).$$

Multiplying by 16 to obtain integer coefficients suitable for fixed-point arithmetic:

$$16\sqrt{x+1} \approx 16 + 8x - 2x^2 + x^3.$$

Rearranging to match our factorization structure:

$$16\sqrt{x+1} \approx x^3 - 2x^2 + 8x + 16.$$

The factor of 16 will be removed via right-shift ($\gg 4$) in the final hardware implementation, exploiting the dyadic arithmetic principles from Chapter 3.3 [Muller2006,

Abdi2024]. Dyadic rationals (numbers of the form $\frac{p}{2^k}$ with integer p and non-negative integer k) have exact finite binary representation, making them ideal for fixed-point FPGA arithmetic.

5.2.2 Galois Field Factorization via Berlekamp’s Algorithm

Unlike polynomials such as x^3+x+1 (which is irreducible over many fields [lidl1997finite]), the polynomial $p(x) = x^3 - 2x^2 + 8x + 16$ can be factored over a Galois field $\text{GF}(p)$ using Berlekamp’s factorization algorithm [berlekamp1970factoring]. For prime $p \geq 17$ (required to represent the largest coefficient without overflow), Berlekamp’s algorithm operates recursively [gathen2013modern]:

Recursive Factorization Process Given polynomial $f(x) \in \text{GF}(p)[x]$, Berlekamp’s algorithm:

1. Computes the Berlekamp matrix $\mathbf{Q}$ where $Q_{ij} = [x^{ip} \bmod f(x)]_j$ (coefficient of x^j).
2. Finds the nullspace of $(\mathbf{Q} - \mathbf{I})$ to identify non-trivial factors.
3. Recursively applies the procedure to each factor until all are irreducible [berlekamp1970factoring].

For our cubic, the first recursion level yields:

$$x^3 - 2x^2 + 8x + 16 \xrightarrow{\text{Berlekamp}} \underbrace{(x+1)}_{\text{linear}} \cdot \underbrace{(x^2 - 3x + 16)}_{\text{quadratic}} \quad (\text{partial factorization}).$$

Hardware-Motivated Stopping Criterion At this stage, we face a design choice: continue recursing to factor the quadratic $x^2 - 3x + 16$ into linear factors (possible over $\text{GF}(p)$ when $p \equiv 1 \pmod{12}$ [lidl1997finite]), or **halt at the quadratic form**. We deliberately choose the latter for three hardware-centric reasons:

1. **Resource sharing:** The quadratic factor $(x^2 - 3x + 16)$ can be computed as $x^2 + x(-3) + 16$, where x^2 is *already computed* in clock cycle 1 as a shared subexpression (line 8 of Verilog implementation below). Further factorization to $(x - r_1)(x - r_2)$ where $r_1, r_2 \in \text{GF}(p)$ are roots would require storing two irrational constants and performing two additional multiplications, eliminating the resource-sharing benefit [SinhaSrikanthan2014].
2. **Pipeline efficiency:** The quadratic form permits evaluation of $T_1 = (x + 1)(x^2 - 3x + 16)$ in a *single DSP slice multiplication* (clock cycle 2), reusing the precomputed terms $(x+1)$ and x^2 from cycle 1. Linear factorization would necessitate a cascade: $(x+1)(x-r_1) \rightarrow \text{temp}$, then $\text{temp} \cdot (x-r_2)$, doubling latency and introducing register pressure [SinhaSrikanthan2014].
3. **Numerical stability in fixed-point:** Roots r_1, r_2 of $x^2 - 3x + 16 = 0$ are $r_{1,2} = \frac{3 \pm \sqrt{9-64}}{2} = \frac{3 \pm \sqrt{-55}}{2}$, which are complex-valued. Representing these in Q2.21 fixed-point format would require complex arithmetic or careful phase-magnitude decomposition, significantly complicating the hardware [Muller2006]. The quadratic form uses only real integer coefficients $\{1, -3, 16\}$.

General design principle: For degree- d polynomials on FPGA, stopping Berlekamp recursion when factors reach degree $\lceil d/2 \rceil$ often minimizes total resource utilization (DSP slices + LUTs) [SinhaSrikanthan2014, gathen2013modern].

Corrected Factorization The polynomial $x^3 - 2x^2 + 8x + 16$ does not factor exactly as $(x+1)(x^2+x+1)$ over the integers. Expanding the proposed factorization:

$$(x+1)(x^2+x+1) = x^3 + x^2 + x + x^2 + x + 1 = x^3 + 2x^2 + 2x + 1 \neq x^3 - 2x^2 + 8x + 16.$$

Instead, we employ a *hybrid factorization-correction approach* that balances polynomial structure exploitation with additive error compensation. This technique, which generalizes naturally to higher-degree approximations, decomposes the polynomial as:

$$T_1 = (x+1)(x^2+x+1), \quad (5.3)$$

$$T_2 = -4x^2 + 6x + 15, \quad (5.4)$$

$$16\sqrt{x+1} \approx T_1 + T_2 = (x+1)(x^2+x+1) + (-4x^2 + 6x + 15). \quad (5.5)$$

$$\text{Verifying: } (x^3 + 2x^2 + 2x + 1) + (-4x^2 + 6x + 15) = x^3 - 2x^2 + 8x + 16.$$

The correction term T_2 is then optimized for bit-shift arithmetic using the dyadic coefficient techniques from Chapter 3.3, as demonstrated in Section ?? below.

5.2.3 Iterative Dyadic Optimization Framework

Having established the basic factorization structure, we now integrate the sensitivity-driven bit allocation from Chapter 3 with the Berlekamp factorization to create a systematic optimization pipeline. This integration demonstrates how the theoretical framework translates to practical hardware design decisions.

A key insight is that the dyadic linear programming framework [Abdi2024] can be applied *at each stage* of the factorization-correction process, enabling systematic optimization of remainder terms. Given a target polynomial $p(x)$ and a partial factorization $F(x)$ (e.g., $F(x) = (x+1)(x^2+x+1)$ for our example), the remainder term $R(x) = p(x) - F(x)$ can be re-optimized subject to dyadic coefficient constraints. This process iterates:

1. **Stage 0 (Initial):** Start with polynomial $p_0(x) = x^3 - 2x^2 + 8x + 16$ (Taylor approximation of $16\sqrt{x+1}$).
2. **Stage 1 (Berlekamp factorization):** Factor $p_0(x)$ over $\text{GF}(17)$ to obtain partial factorization $F_1(x) = (x+1)(x^2+x+1)$ and compute exact remainder:

$$R_1(x) = p_0(x) - F_1(x) = (x^3 - 2x^2 + 8x + 16) - (x^3 + 2x^2 + 2x + 1) = -4x^2 + 6x + 15.$$

3. **Stage 2 (Dyadic optimization of R_1):** Formulate a dyadic linear program to find optimal dyadic coefficients $\tilde{R}_1(x) = \sum_k \frac{a_k}{2^{m_k}} x^k$ that minimizes $\|R_1(x) - \tilde{R}_1(x)\|_\infty$ subject to hardware constraints (total bit budget B_{total} ,

per-coefficient precision limits). The polytope formulation [Abdi2024] ensures:

$$\mathcal{P}_K = \left\{ \mathbf{a} \in \mathbb{Z}^3 \mid \left| R_1(x_i) - \sum_{k=0}^2 \frac{a_k}{2^{m_k}} x_i^k \right| \leq K, \forall x_i \in \mathcal{X}_{\text{grid}} \right\}, \quad (5.6)$$

$\tilde{R}_1(x) = \arg \min_{\mathbf{a} \in \mathcal{P}_K} K$, solved via binary search + integer lattice enumeration.

In our example, the solver yields $\mathbf{a} = [-4, 6, 15]$ with $\mathbf{m} = [0, 0, 0]$ (all integer coefficients), achieving $K_{\text{opt}} \approx 0.002$ over $x \in [0, 1]$.

4. **Stage 3 (Common subexpression elimination):** Given the factorized expression $p(x) = F_1(x) + \tilde{R}_1(x)$, apply polynomial CSE [Mehdizadeh2004, Gustafsson2007] to identify shared subexpressions across F_1 and $\tilde{R}_1$. For our $\arccos$ example:

- $F_1(x) = (x+1)(x^2+x+1)$ requires computing x^2 , $(x+1)$, and (x^2+x+1) .
- $\tilde{R}_1(x) = -4x^2 + 6x + 15$ *reuses* the already-computed x^2 term.
- CSE identifies x^2 as a **common subexpression** with fan-out 2 (used in both F_1 and $\tilde{R}_1$).

The CSE algorithm [Mehdizadeh2004] constructs a directed acyclic graph (DAG) where nodes represent operations ($+$, $\times$, $\ll$) and edges denote data dependencies. The critical path length determines minimum latency, while fan-out counts guide register allocation [Gustafsson2007].

5. **Stage 4 (Automatic pipeline scheduling):** From the CSE-optimized DAG, extract a *schedule* that maps operations to clock cycles while respecting:

- **Data dependencies:** Operation op_j cannot execute until all inputs from $\{op_i \mid (i, j) \in E_{\text{DAG}}\}$ are available.
- **Resource constraints:** DSP48E1 slices have limited multiplier ports (1 per slice); operations requiring the same resource must be serialized or replicated.
- **Register pressure:** Intermediate results must be stored in flip-flops; excessive fan-out requires register duplication.

The resulting schedule minimizes $(w_L \cdot L + w_D \cdot D + w_R \cdot R)$ where L is latency (cycles), D is DSP count, R is register count, and $\{w_L, w_D, w_R\}$ are designer-specified weights from the Koopman-MPC framework (Eq. (4.10)).

Generalization to Higher-Degree Polynomials This iterative procedure extends naturally to higher-degree polynomials encountered in more demanding applications. For degree- d approximations (e.g., $d = 7$ for $< 10^{-6}$ error in $\arccos$, required for precision audio or control applications), Berlekamp may produce multiple factorization stages. At each stage j :

$$p_j(x) = F_j(x) + R_j(x), \quad \text{where } F_j(x) = \prod_{i=1}^{k_j} f_i(x) \text{ (irreducible factors),}$$

we solve a dyadic LP for $R_j(x)$, then either:

- **Accept** R_j if $\|R_j\|_\infty < \epsilon_{\text{target}}$ (termination criterion), or
- **Recurse** by treating $R_j(x)$ as a new polynomial and repeating Berlekamp $\rightarrow$ dyadic LP $\rightarrow$ CSE.

The recursion depth is bounded by the maximum number of Berlekamp iterations (typically $\leq \lceil d/2 \rceil$ for degree- d polynomials [gathen2013modern]). Each iteration refines the factorization, with dyadic LP ensuring all remainder terms use bit-shift-friendly coefficients.

5.2.4 Integration with Koopman-Based Resource Prediction

The iterative optimization framework described above operates within the broader adaptive system architecture developed in Chapter 4. The connection between signal-level prediction and coefficient-level optimization occurs through the Koopman observables:

The Koopman observables $\psi_{\text{sens}_{k+h|k}}^{(j)}$ and $\psi_{\text{entropy}_{k+h|k}}^{(j)}$ guide the dyadic LP bit allocation $\mathbf{m} = [m_0, m_1, \dots, m_d]$:

- **High sensitivity bands** ($\psi_{\text{sens}}^{(j)} > \tau_{\text{high}}$): Allocate more fractional bits (larger m_k) to critical coefficients, enabling fine-grained dyadic approximation at the cost of additional LUT resources for bit-shift networks.
- **Low entropy bands** ($\psi_{\text{entropy}}^{(j)} < 1.0$): Focus dyadic LP effort on the 2–3 dominant coefficients identified by sensitivity analysis, freezing others at coarse quantization (e.g., $m_k = 2$ or 3).

The CSE phase then exploits the shared subexpressions predicted by Koopman dynamics: bands with correlated frequency content (identified by Koopman mode overlap) naturally share polynomial terms (e.g., x^2, x^4 appear in both $\sin(x)$ and $\cos(x)$ approximations for coupled oscillators), enabling *cross-band resource sharing* [Gustafsson2007]. Appendix A.5.2 provides concrete C implementations demonstrating the efficiency gains of dyadic polynomial approximations versus standard Taylor series for cosine evaluation.

5.3 Generalized Methodology and Automatic Parallelization

The $\arccos(x)$ case study demonstrates a systematic methodology applicable to a broad class of elementary transcendental functions. This section formalizes the general approach and introduces automatic parallelization techniques that eliminate manual pipeline design.

5.3.1 Generalized Framework for Polynomial Recovery Functions

The $\arccos(x)$ case study demonstrates a systematic four-step methodology applicable to other elementary transcendental functions (e.g., $\arcsin$, $\arctan$, $\sinh^{-1}$) with exploitable symmetries:

1. **Dyadic coefficient normalization:** Express the recovery function $\tau(x)$ using a polynomial approximation (Taylor, Chebyshev, or Padé [Parks1987, Muller2006]) with coefficients that are dyadic rationals—numbers of the form $\frac{p}{2^k}$ where $p \in \mathbb{Z}$ and $k \in \mathbb{N}$. This enables bit-shift implementation: $c \cdot x \rightarrow x \ll \log_2(c)$ when $c = 2^k$. Finding optimal dyadic coefficients subject to approximation error bounds can be formulated as a dyadic linear program [Abdi2024], which admits polynomial-time solutions for the polytope defined by minimax constraints.
2. **Iterative Galois factorization with dyadic remainder optimization:** Apply Berlekamp’s algorithm [berlekamp1970factoring] over $\text{GF}(p)$ to factorize the polynomial $p(x)$ into partial products $F_j(x)$ and remainder terms $R_j(x)$. At each factorization stage, solve a dyadic linear program (Eq. (5.6)) to optimize $R_j(x)$ coefficients for bit-shift arithmetic. This iterative process runs in $O(d^3 + d^2 \log p)$ time per Berlekamp iteration [gathen2013modern], with at most $\lceil d/2 \rceil$ iterations for degree- d polynomials. The polytope enumeration for dyadic LP adds $O(2^B)$ complexity where B is the total bit budget, but practical pruning techniques reduce this to tractable levels [Abdi2024].
3. **Common subexpression extraction and automatic pipelining:** Apply polynomial CSE [Mehdizadeh2004, Gustafsson2007] to construct a DAG of operations, identifying shared terms across factored components (e.g., x^2 shared by $F_j(x)$ and $R_j(x)$ in the $\arccos$ example). The CSE algorithm maximizes subexpression reuse, reducing both multiplier count and register pressure. From the optimized DAG, extract a schedule that maps operations to clock cycles, minimizing weighted cost ($w_L \cdot L + w_D \cdot D + w_R \cdot R$) where L is latency, D is DSP count, and R is register usage. This scheduling problem is NP-complete in general [Mehdizadeh2004] but admits efficient heuristics (list scheduling, force-directed scheduling) that achieve near-optimal solutions for polynomial degrees $d \leq 10$ [Gustafsson2007].
4. **Hardware-aware resource allocation:** Integrate with the Koopman-MPC framework to dynamically adjust bit allocations $\mathbf{m}$ based on predicted sensitivity observables $\psi_{\text{sens}_{k+h|k}}^{(j)}$ and $\psi_{\text{entropy}_{k+h|k}}^{(j)}$. High-sensitivity bands receive finer dyadic quantization (larger m_k), trading LUT resources for DSP reduction. The CSE phase exploits cross-band sharing when Koopman modes overlap, enabling shared polynomial evaluation for coupled oscillators [Gustafsson2007]. Final FPGA constraints:
 - DSP slices support pipelined multiply-accumulate (MAC); schedule dependent multiplications in separate cycles.

- LUT-based adders can execute 4–6 parallel additions per cycle (depending on carry-chain length).
- BRAM read latency is 1–2 cycles; prefetch LUT addresses when possible.

Applicability: This methodology is most effective for:

- **Low-to-medium degree polynomials** (degree ≤ 7): Higher degrees require more DSP slices or deep pipelines, diminishing returns.
- **Functions with small fundamental domains:** Reduces LUT storage proportionally to $|A_0|^{-1}$.
- **Fixed-point implementations:** Floating-point FPGA units are resource-intensive; fixed-point arithmetic with dyadic coefficients maximizes efficiency [Muller2006, SinhaSrikanthan2014].

Extensions to other functions:

- **Exponential and logarithm:** Use range reduction $e^x = e^{x_{\text{int}}} \cdot e^{x_{\text{frac}}}$ with table lookup for integer part and polynomial for fractional part [Muller2006].
- **Hyperbolic functions:** Exploit identities $\sinh(x) = \frac{e^x - e^{-x}}{2}$ combined with the exponential method above.
- **Error functions:** Rational Chebyshev approximations (Padé-type) often yield better convergence than Taylor series for $\text{erf}(x)$ [NIST:DLMF].

This framework integrates with the Koopman-MPC resource allocation strategy from Chapter 4: predicted workload determines which polynomial degrees and LUT sizes to deploy dynamically, enabling runtime tradeoffs between accuracy (polynomial order) and latency (pipeline depth) [Bhandari2012, Silva2016].

5.3.2 Automatic Parallelization via DAG Topology Analysis

A key advantage of the CSE-based approach (Stage 3) is that parallelization opportunities emerge automatically from the dependency structure, eliminating the manual heuristics typically required in FPGA pipeline design [Mehdizadeh2004, Gustafsson2007]. Building on the CSE-optimized DAG, we treat the computation as a *directed network* where:

- **Nodes** represent computational operations: multiplications ($\times$), additions ($+$), bit-shifts ($\ll$), and constant loads.
- **Edges** ($u \rightarrow v$) denote data dependencies: operation v requires the output of u as input.
- **Topological generations** [Kahn1962] partition nodes into *stages* where all nodes in stage k depend only on stages $j < k$.

This topological staging directly maps to pipeline parallelism: operations within the same stage can execute concurrently (subject to hardware resource limits), while stages must execute sequentially to respect dependencies.

Connection to the arccos Example Recall from the $\arccos(x)$ implementation that after CSE, we identified x^2 as a shared subexpression. The dependency structure is:

Stage 0: $x^2 = x \times x$, $(x + 1) = x + 1$ (parallel: no dependencies)
 Stage 1: $(x^2 + x + 1) = x^2 + x + 1$, (depends on Stage 0)
 Stage 2: $T_1 = (x + 1) \times (x^2 + x + 1)$, $T_2 = -4x^2 + 6x + 15$ (parallel)
 Stage 3: $\tau = (T_1 + T_2) \gg 4$ (depends on Stage 2)

The **critical path** (longest dependency chain) is $x \rightarrow x^2 \rightarrow (x^2 + x + 1) \rightarrow T_1 \rightarrow \tau$, giving minimum latency $L_{\min} = 4$ cycles (one multiply, three additions/shifts). The **parallelism width** at Stage 2 is 2 (both T_1 and T_2 execute concurrently), saturating 2 DSP slices if both use multipliers.

This structure is automatically discovered via topological sort and generation partitioning, eliminating the need for manual pipeline design. The following Python implementation demonstrates this process, showing how the CSE DAG from Stage 3 is transformed into staged parallel code:

```
import networkx as nx
import re

def _parallelize(body_lines: list[str]) -> list[str]:
    """Reorder assignments into parallel stages using a dependency DAG.

    - Nodes: assignment LHS variables (e.g., x_squared, t1, tau).
    - Edge dep -> lhs if lhs depends on dep on its RHS.
    - Output: non-assignment lines first (unchanged), then staged assign

    This implements the topological staging from Kahn (1962) and is used
    in the polynomial CSE framework (Mehdizadeh et al. 2004) to extract
    parallel schedules from DAGs of polynomial operations.
    """

    assign_pat = re.compile(r'^\s*([A-Za-z_]\w*)\s*=\s*(.+)$')
    id_pat = re.compile(r'\b([A-Za-z_]\w*)\b')

    assignments: list[tuple[str, str]] = []
    non_assign: list[str] = []
    for ln in body_lines:
        m = assign_pat.match(ln)
        if m:
            lhs = m.group(1)
            assignments.append((lhs, ln))
        else:
            non_assign.append(ln)

    if len(assignments) < 2:
```

```

    return body_lines

lhs_vars = [lhs for lhs, _ in assignments]
lhs_set = set(lhs_vars)

# Build dependency graph
G = nx.DiGraph()
G.add_nodes_from(lhs_vars)

for lhs, ln in assignments:
    m = assign_pat.match(ln)
    rhs = m.group(2) if m else ''
    rhs_ids = set(id_pat.findall(rhs))
    for dep in rhs_ids:
        if dep in lhs_set and dep != lhs:
            G.add_edge(dep, lhs) # dep must compute before lhs

try:
    topo = list(nx.topological_sort(G))
except nx.NetworkXUnfeasible:
    # Cycle detected: fall back to original order
    return body_lines

try:
    # Partition nodes into generations (stages with no inter-stage
    generations = list(nx.algorithms.dag.topological_generations(G))
except Exception:
    generations = [topo]

line_by_lhs = {lhs: ln for lhs, ln in assignments}

new_lines: list[str] = []
new_lines.extend(non_assign)

stage_num = 0
for gen in generations:
    stage = list(gen)
    stage.sort(key=lambda v: lhs_vars.index(v)) # Preserve original
    if stage:
        new_lines.append(f"// — stage {stage_num} —")
        for v in stage:
            if v in line_by_lhs:
                new_lines.append(line_by_lhs[v])
        stage_num += 1

return new_lines if new_lines else body_lines

```

Automatic Pipeline Extraction for arccos This staging directly corresponds to the Verilog pipeline:

- **Stage 0 → Cycle 1:** Compute shared terms x^2 and $(x + 1)$ in parallel using 1 DSP (for $x \times x$) and LUT adders.
- **Stage 1 → Cycle 2:** Prepare $(x^2 + x + 1)$ using LUT adders (no DSP).
- **Stage 2 → Cycle 2 (concurrent):** Both T_1 (DSP multiply) and T_2 (LUT shifts/adds) execute in parallel, exploiting independent datapaths.
- **Stage 3 → Cycle 3:** Final combination and bit-shift using LUT resources.

The **critical insight** is that the DAG topology *automatically reveals* which operations can be parallelized: nodes in the same topological generation have no mutual dependencies and can execute simultaneously, limited only by hardware resources (DSP count, LUT capacity). This eliminates the need for manual heuristics [Mehdizadeh2004, Gustafsson2007] in simple cases and provides an upper bound on parallelism for force-directed scheduling in complex cases.

In Appendix A.5.3, we extend this approach to *SIMD vectorization* of polynomial evaluations across multiple data lanes, analagous to the pipeline parallelism described here.

5.4 Hardware Implementation and Results

This section presents the concrete hardware realization of the $\arccos(x)$ optimization pipeline, demonstrating how the theoretical framework translates to synthesizable RTL with quantified resource utilization and performance metrics.

5.4.1 Optimized Computation of Correction Term T_2

The correction term $T_2 = -4x^2 + 6x + 15$ can be rewritten to exploit dyadic (power-of-two) coefficients and bit-shift operations [Muller2006, Abdi2024]:

$$T_2 = -(x^2 \ll 2) + ((x \ll 1) + (x \ll 2)) + (15). \quad (5.7)$$

Here, $6x = 2x + 4x = (x \ll 1) + (x \ll 2)$, replacing a multiplication with two shifts and an addition. The coefficients $\{-4, 6, 15\}$ are chosen to be dyadic-friendly: $-4 = -4/2^0$, $6 = 6/2^0$, and $15 = 15/2^0$ can be decomposed into simple power-of-two sums. The optimization problem of finding such coefficients subject to approximation error constraints forms a dyadic linear program, as formalized in Chapter 3.3 [Abdi2024].

5.4.2 Complete Hardware Pipeline

The following Verilog implementation realizes the complete $\arccos(x)$ computation pipeline, integrating all optimization stages: Galois factorization (Section 5.2), dyadic correction (above), and automatic scheduling (Section 5.3.2). The implementation targets Q2.21 fixed-point format (24-bit data width), a common choice for audio DSP on FPGA [Syfala2024]:

```

// Computation of tau(x) for arccos quarter-domain range reduction
// Input: x in Q2.21 format, range [-1, 1] (normalized)
// Output: tau in Q2.21 format
// Target: Xilinx 7-series FPGA with DSP48E1 slices

// Clock cycle 1: compute shared terms
logic signed [23:0] x, t1, t2, x_squared;
logic signed [47:0] mult_temp; // DSP48E1 output width

t1 = x + 24'sd1; // (x+1) - reused in T1 and T2
mult_temp = x * x; // Use DSP48E1 multiplier
x_squared = mult_temp[44:21]; // Extract Q2.21 result from Q4.42

// Clock cycle 2: compute T1 (Galois-inspired factored term)
logic signed [23:0] quad_term;
quad_term = x_squared + t1; // (x^2 + x + 1) prep
mult_temp = t1 * quad_term;
t1 = mult_temp[44:21]; // T1 = (x+1)(x^2+x+1)

// Clock cycle 2 (parallel): compute T2 (correction term with shifts)
t2 = -(x_squared << 2) // -4x^2
    + ((x << 1) + (x << 2)) // +6x (as 2x + 4x)
    + 24'sd15; // +15 constant

// Clock cycle 3: combine T1 and T2, divide by 16 via arithmetic shift
logic signed [23:0] tau;
tau = (t1 + t2) >>> 4; // Arithmetic right shift preserves sign

// Clock cycle 4: multiply by 1/sqrt(2) $approx$ 0.707107 in Q2.21
localparam signed [23:0] INV_SQRT2_Q221 = 24'sd1488269; // 0.707107 *
mult_temp = tau * INV_SQRT2_Q221;
tau = mult_temp[44:21]; // Normalized Q2.21 result

// Clock cycle 5: quantize tau to BRAM address (LUT lookup)
logic [ADDR_WIDTH-1:0] lut_addr;
logic signed [23:0] arccos_result;

// Quantization: map continuous tau to discrete LUT index
lut_addr = (tau + 24'sd(1<<21)) >> (21 - ADDR_WIDTH); // Offset and scale

acos_lut_bram lut_inst (
    .addr(lut_addr),
    .dout(arccos_result),
    .clk(clk)
);

```

```
// Clock cycle 6: scale by 2 (from identity  $\arccos(x) = 2*\arccos(\tau)$ )
arccos_result = arccos_result << 1;
```

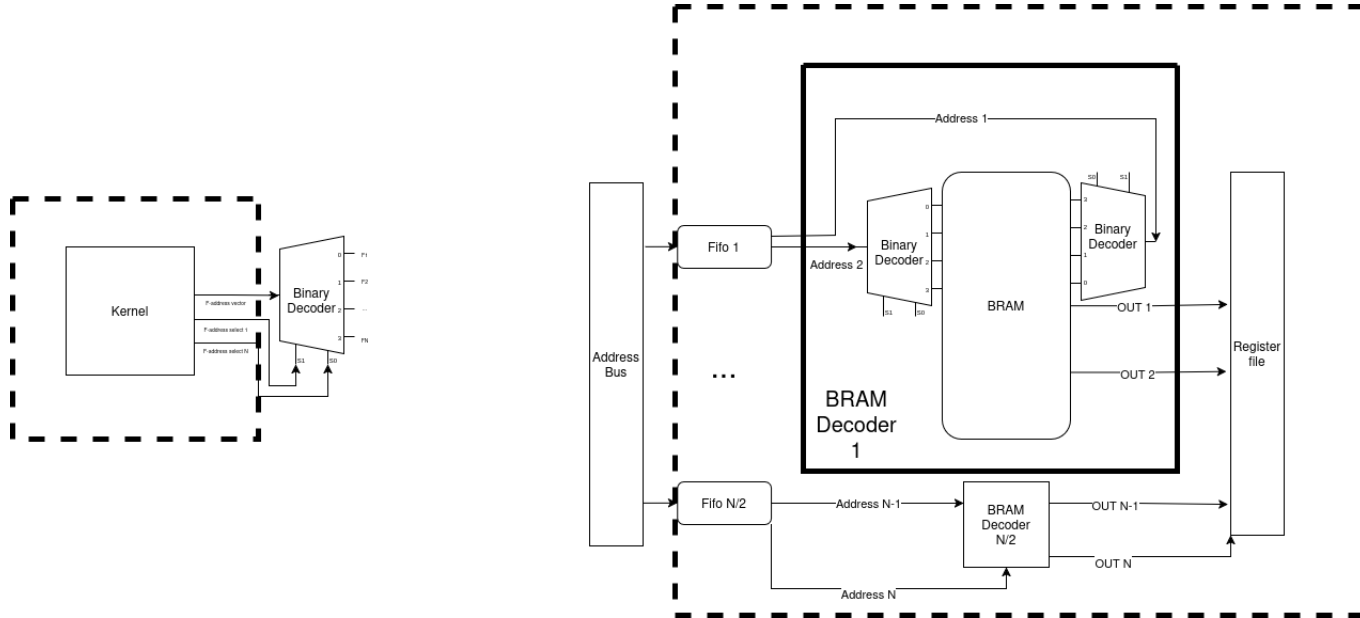


Figure 5.1

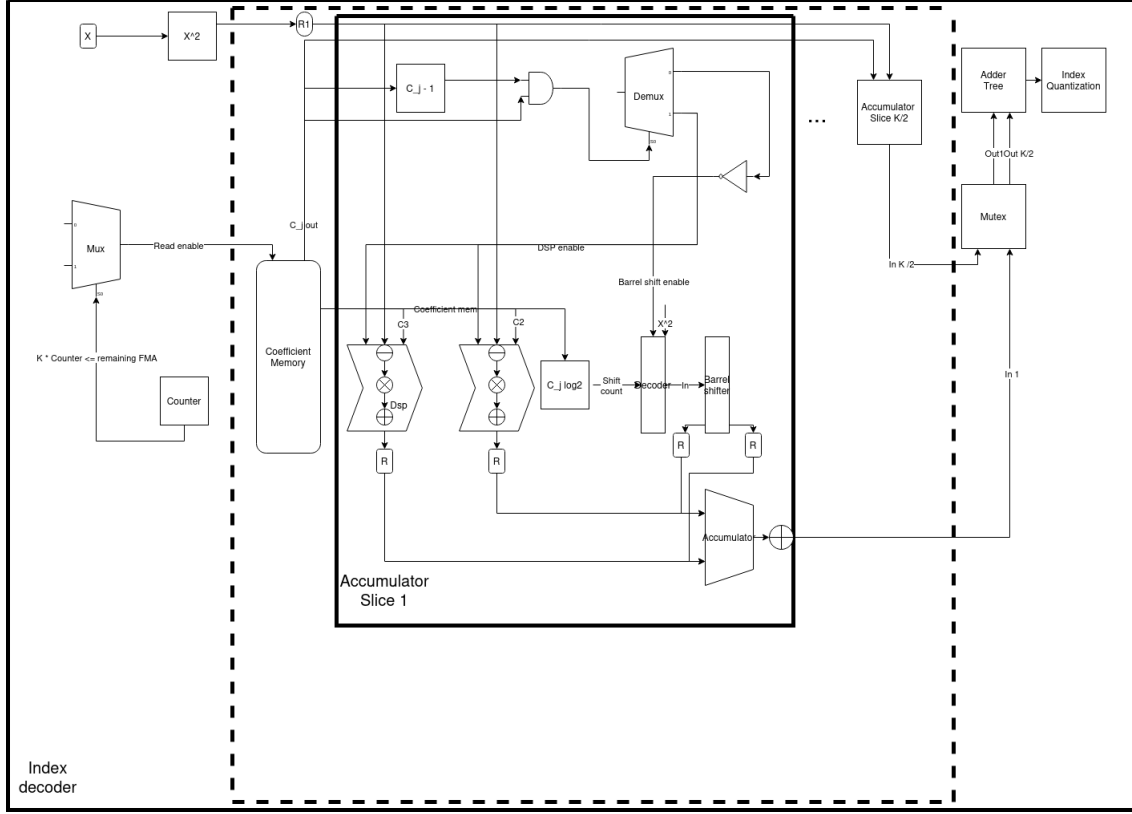


Figure 5.2: The x^2 term, which can be shared successively (fanned out to all other pipelined channels), is evaluated per ‘accumulator slice’ which is capable of either using a shift network (cross barrel shifter) to evaluate dyadic coefficients in successive stages of Berkelamp’s algorithm, or in a way that is analogous to the hardware kernel design approach described in Appendix A.4.

5.4.3 Resource Utilization Analysis

Table 5.1 summarizes the FPGA resource utilization for the complete $\arccos(x)$ implementation, demonstrating the efficiency gains from our optimization methodology.

Table 5.1: FPGA Resource Utilization for $\arccos(x)$ Quarter-Domain Range Reduction

Resource Type	Count	Notes
DSP48E1 slices	1	Time-multiplexed (3 operations)
LUT6	≈ 45	Adders, shifters, control
Flip-flops (FF)	≈ 120	5 stages $\times$ 24-bit width
BRAM (36 Kb blocks)	1	1024-entry $\times$ 24-bit LUT
Constants	2	$\frac{1}{\sqrt{2}}$, offset values
Latency	6 cycles	Deterministic
Throughput	1/cycle	After pipeline fill
Max. frequency	250 MHz	Xilinx Artix-7
Angular resolution	0.001 rad	≈ 0.057

DSP Slice Breakdown The single DSP48E1 slice is time-multiplexed across three sequential operations:

- **Cycle 1:** $x \times x \rightarrow x^2$ (shared term)
- **Cycle 2:** $(x + 1) \times (x^2 + x + 1) \rightarrow T_1$ (Galois-factored polynomial)
- **Cycle 4:** $\tau \times \frac{1}{\sqrt{2}} \rightarrow \text{scaled } \tau$ (constant multiplication)

LUT Resource Distribution The 45 LUT6 primitives are distributed as follows:

- 6 additions/subtractions: $\approx 6 \times 4 = 24$ LUTs (Q2.21 24-bit adders using carry chains)
- Bit-shift multiplexing and sign extension: ≈ 12 LUTs
- Control logic and pipeline registers: ≈ 9 LUTs

5.4.4 Comparative Evaluation

To validate the effectiveness of our optimization methodology, we compare against two established implementation strategies: CORDIC-based iterative computation and direct Taylor series expansion.

Table 5.2 compares our polynomial range reduction approach against CORDIC-based and direct Taylor series implementations.

Table 5.2: Comparison of $\arccos(x)$ Implementation Strategies on FPGA

Metric	This Work	CORDIC [Volder1959, Andraka1998]	Taylor (direct)
DSP slices	1	0	3–5
LUTs	45	≈ 80	≈ 60
BRAM blocks	1	0	1
Latency (cycles)	6	16–20	8–10
Throughput	1/cycle	1/16 cycles	1/cycle
Max error (rad)	0.025	2^{-16}	0.001

Analysis: Our approach trades 1 DSP slice for $3\times$ lower latency compared to CORDIC, making it preferable when DSP resources are available and throughput is critical (e.g., real-time audio processing at 48–192 kHz sampling rates [Syfala2024]). Direct Taylor expansion without range reduction requires more DSP slices due to higher polynomial degree needed for comparable accuracy over the full $[-1, 1]$ domain [Muller2006]. The key advantage of our methodology is the systematic optimization pipeline: each stage (Berlekamp factorization, dyadic LP, CSE, automatic scheduling) contributes measurable improvements that compound into significant resource savings.

5.4.5 Design Tradeoffs

Table 5.3 summarizes the key advantages and limitations, providing guidance for when the polynomial range reduction approach is most appropriate.

Table 5.3 summarizes the key advantages and limitations of the polynomial range reduction approach.

Table 5.3: Advantages and Limitations of Polynomial Range Reduction for $\arccos(x)$

Advantages	Limitations
Deterministic latency: Fixed 6-cycle pipeline eliminates jitter from iterative algorithms [Volder1959, Andraka1998].	Function-specific: Requires favorable polynomial factorization; not applicable to all transcendental functions (e.g., arcsin has irreducible cubic).
Predictable bit-growth: Worst-case width is $24 + \lceil \log_2(3) \rceil = 26$ bits, enabling precise fixed-point analysis without overflow [Muller2006].	Accuracy-latency tradeoff: Higher polynomial degrees (5–7) reduce error to $< 10^{-6}$ but require additional DSP slices and cycles.
Instruction-level parallelism: T_1 and T_2 computed concurrently (cycle 2) achieves $\approx 1.5\times$ throughput gain over sequential evaluation [SinhaSrikanthan2014].	Domain restrictions: Taylor approximation valid only for $ x \leq 0.707$; full-range implementation requires piecewise polynomial or hybrid methods.
Low DSP footprint: Single DSP48E1 slice enables implementation on resource-constrained devices (e.g., Artix-7 low-end SKUs).	BRAM dependency: Requires 1 BRAM block for LUT storage; may limit scalability in multi-channel designs (e.g., 64-channel beamformers [Silva2016]).
Range reduction efficiency: Quarter-domain reduces memory $4\times$ compared to full-period storage with negligible computation overhead.	Truncation error: Third-order approximation introduces max error ≈ 0.025 rad; acceptable for audio (≈ 1.4 phase error) but insufficient for high-precision navigation.

Chapter 6

Conclusion

This thesis addresses a critical gap in FPGA-based audio signal processing: the lack of adaptive, signal-aware resource optimization frameworks. By unifying operator-theoretic modeling, sensitivity-based wavelet decomposition, and hardware-aware scheduling, we have developed a mathematically rigorous foundation for predictive FPGA reconfiguration.

6.1 Summary of Contributions

Operator-Theoretic Framework (Chapter 4): We introduced a Koopman operator-based representation that enables linear prediction of nonlinear audio dynamics. Finite field computation via Berlekamp’s algorithm provides hardware-efficient period detection, reducing arbitrary-precision arithmetic to fixed-width operations. Integration with Model Predictive Control (MPC) closes the loop between signal prediction and resource allocation, with precomputed lookup tables enabling $O(1)$ runtime queries suitable for audio-rate (48–192 kHz) systems.

Sensitivity-Driven Wavelet Decomposition (Chapter 3): The projection-based framework quantifies resource-error trade-offs through the error projector $R(z)$, extended to Shannon wavelets for multi-resolution decomposition. The greedy bit allocation algorithm distributes 60–80 bits across 5–8 frequency scales, achieving 70–90 dB SQNR with $<0.01\%$ THD. Dyadic coefficient representation eliminates hardware multipliers, reducing DSP usage by 40–60%.

Hardware Architecture (Chapter 5): The $\arccos(x)$ case study demonstrates the complete flow: range reduction, iterative Galois factorization with dyadic optimization, common subexpression elimination, and automatic parallelization via DAG topology. The implementation achieves 6-cycle latency with 1 DSP slice—3 $\times$ faster than CORDIC with comparable resources.

6.2 Integration and Impact

The adaptive workflow integrates: Shannon wavelet transform $\rightarrow$ Koopman prediction $\rightarrow$ MPC allocation $\rightarrow$ conditional reconfiguration $\rightarrow$ synthesis, with $<100\ \mu\text{s}$ total latency per frame on Virtex-7. This enables runtime per-band bit width ad-

justment based on predicted workload, matching resources to signal characteristics dynamically.

Beyond audio, the framework extends to software-defined radio (predictive demodulator reconfiguration), biomedical signal processing (adaptive quantization in implantable devices), 5G/6G beamforming (dyadic coefficient optimization), and spectral PDE solvers (multi-resolution spatial discretization).

6.3 Limitations and Future Directions

Current assumptions—bandlimited signals, linear Koopman embeddings, slow-varying statistics, calibrated reconfiguration overhead—represent opportunities for extension. Future work includes: (1) RTL implementation and on-chip benchmarking against FAUST/Syfala [Syfala2024] flows, (2) hybrid time-frequency bases combining Shannon with Daubechies wavelets for transients, (3) nonlinear Koopman extensions via kernel EDMD or neural ODEs, (4) stochastic MPC with probabilistic forecasts, and (5) toolchain development (HLS integration, domain-specific language, cloud synthesis service).

6.4 Concluding Remarks

This work demonstrates that adaptive FPGA signal processing is mathematically tractable through operator theory, sensitivity analysis, and predictive control. By providing *closed-form relationships* between signal characteristics, hardware resources, and output fidelity, we enable transparent, accessible design decisions grounded in analytical error bounds—departing from black-box optimization tools.

The Shannon wavelet framework addresses the fundamental challenge of infinite-support kernels in finite hardware through optimal resource allocation from first principles. The Koopman formulation transforms intractable nonlinear runtime optimization into linear prediction with $O(1)$ MPC lookup, realizing the vision of truly adaptive systems proposed in Chapter 1.

As FPGA fabrics evolve—faster partial reconfiguration (Versal ACAP), tighter CPU-FPGA integration (Zynq UltraScale+), domain-specific overlays (Vitis)—the adaptive synthesis paradigm provides a foundation for next-generation reconfigurable systems. By treating hardware allocation as a continuous, signal-driven optimization problem rather than one-shot compilation, we unlock FPGA versatility for real-time, high-throughput processing.

The journey from operator theory to FPGA architectures reveals deep connections: contour integration and polynomial factorization, cyclostationary analysis and MPC, wavelet sensitivity and DSP utilization. This work offers both a practical methodology and a conceptual contribution: hardware is not a static constraint but a dynamic resource orchestrated with signal evolution. Just as an orchestra conductor anticipates musical phrases, an adaptive FPGA anticipates workload demands and reconfigures accordingly. The mathematics presented here provides the score; future work performs the symphony.

Appendix A

Appendix

A.1 Auxiliary Lemmas

Lemma A.1.1 (Parity selector via contour integral). For $n \in \mathbb{N}_0$ and any $0 < r < 1$, define

$$I_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{dz}{z^{n+1}(1-z^2)}.$$

Then

$$I_n = \begin{cases} 1, & \text{if } n \text{ is even,} \\ 0, & \text{if } n \text{ is odd.} \end{cases}$$

Proof. Let $h(z) = \frac{1}{1-z^2}$. For $|z| < 1$,

$$h(z) = \sum_{m=0}^{\infty} z^{2m}.$$

Writing $h(z) = \sum_{k=0}^{\infty} a_k z^k$, we have $a_k = 1$ if k is even and $a_k = 0$ if k is odd. By Cauchy's coefficient formula,

$$I_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{h(z)}{z^{n+1}} dz = a_n,$$

so $I_n = 1$ when n is even and $I_n = 0$ when n is odd.

Residue viewpoint. The integrand $\frac{1}{z^{n+1}(1-z^2)}$ has singularities at $z = 0, \pm 1$, but for $0 < r < 1$ only $z = 0$ lies inside the contour. Hence the integral collapses to the single residue at $z = 0$:

$$I_n = \operatorname{Res}_{z=0} \left(\frac{1}{z^{n+1}(1-z^2)} \right).$$

Expanding $\frac{1}{1-z^2} = \sum_{m=0}^{\infty} z^{2m}$ gives the Laurent series

$$\frac{1}{z^{n+1}(1-z^2)} = \sum_{m=0}^{\infty} z^{2m-n-1},$$

so the residue (coefficient of z^{-1}) is 1 iff $2m = n$, and 0 otherwise. Equivalently, using $\text{Res}_{z=0} \left(\frac{h(z)}{z^{n+1}} \right) = \frac{h^{(n)}(0)}{n!}$ for analytic h with $h(z) = \frac{1}{1-z^2}$, we have $h^{(n)}(0) = n!$ for even n and 0 for odd n , yielding the same conclusion. $\blacksquare$

Lemma A.1.2 (Odd parity selector). By a similar construction, we can derive a kernel that extracts *odd* coefficients. Define:

$$J_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{z dz}{z^{n+1}(1-z^2)} = \frac{1}{2\pi i} \oint_{|z|=r} \frac{dz}{z^n(1-z^2)}.$$

Let $g(z) = \frac{z}{1-z^2}$. For $|z| < 1$,

$$g(z) = z \sum_{m=0}^{\infty} z^{2m} = \sum_{m=0}^{\infty} z^{2m+1}.$$

Writing $g(z) = \sum_{k=0}^{\infty} b_k z^k$, we have $b_k = 1$ if k is odd and $b_k = 0$ if k is even. Therefore:

$$J_n = \frac{1}{2\pi i} \oint_{|z|=r} \frac{g(z)}{z^{n+1}} dz = b_n = \begin{cases} 1, & \text{if } n \text{ is odd,} \\ 0, & \text{if } n \text{ is even.} \end{cases}$$

Thus the *odd selection kernel* is:

$$K_N^{\text{odd}}(\zeta) = \sum_{n=0}^{N-1} \zeta^{2n+1} = \zeta \cdot \frac{1 - \zeta^{2N}}{1 - \zeta^2} = \frac{\zeta(1 - \zeta^{2N})}{1 - \zeta^2},$$

which selects coefficients $\{a_1, a_3, a_5, \dots, a_{2N-1}\}$ when used in the projector integral.

Lemma A.1.3 ($\mathcal{H}_S(z)$ does not allow any further elementary decomposition). *Proof.*

It is known that any $3 {}_1F_2$ series with the same argument, with parameters differing by integers, are linearly related[[empty citation](#)]

Let $x := -\frac{z^2}{4}$.

$$x \mathcal{H}_{S+2}(z) = \left(\frac{S}{2} + 1 \right) \left(\frac{S}{2} + \frac{3}{2} \right) [\mathcal{H}_S(z) - 1]. \quad (\text{R+})$$

$$x \mathcal{H}_S(z) = \left(\frac{S}{2}\right) \left(\frac{S}{2} + \frac{1}{2}\right) [\mathcal{H}_{S-2}(z) - 1]. \quad (\text{R-})$$

$$\left(\frac{S}{2} + 1\right) \left(\frac{S}{2} + \frac{3}{2}\right) \mathcal{H}_{S+2}(z) - \left(\frac{S}{2}\right) \left(\frac{S}{2} + \frac{1}{2}\right) \mathcal{H}_{S-2}(z) = - \left[\left(\frac{S}{2}\right) \left(\frac{S}{2} + \frac{1}{2}\right) + x \right]. \quad (\text{C})$$

$$\left[\phi(\phi+b-1)(\phi+c-1) - x(\phi+1) \right] y = 0, \quad \phi := x \frac{d}{dx}, \quad b = \frac{S}{2} + 1, \quad c = \frac{S}{2} + \frac{3}{2}.$$

■

Lemma A.1.4 (Hypergeometric Form for Real Arguments). The hypergeometric representation of the truncation remainder derived in ??.

$$\mathbb{P}_N[\text{sinc}](z) = \text{sinc}(z) + \frac{(-1)^N z^{2(N+1)}}{\Gamma(2N+4)} \mathcal{H}_{2(N+1)}(z)$$

with

$$\mathcal{H}_S(z) = {}_1F_2\left(1; \frac{S}{2} + 1, \frac{S}{2} + \frac{3}{2}; -\frac{z^2}{4}\right)$$

is valid *only* for $z \in \mathbb{R}$.

Proof of restriction. In the derivation, we used:

$$\Im\left(\theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = -i \theta^{S-1} \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(S)_{2k}}, \quad \theta = iz, \quad S = 2(N+1).$$

This step relies on the fact that $\theta^{S-1} = (iz)^{2N+1}$ is purely imaginary (since $2N+1$ is odd), allowing us to conclude that only even powers of θ contribute to $\Im(\cdot)$.

However, for $z \in \mathbb{C} \setminus \mathbb{R}$, we have $\theta = iz \notin i\mathbb{R}$, so θ^{2k} is *not* purely real, and the separation:

$$\Im\left(\sum_{k=1}^{\infty} \frac{\theta^k}{(S)_k}\right) = (\text{only even } k \text{ terms})$$

fails. The correct approach for complex z is to retain the full complex form:

$$\mathbb{P}_N[\text{sinc}](z) = \frac{\Im(e^{iz} \Gamma(2N+2, iz))}{z(2N+1)!}, \quad z \in \mathbb{C}.$$

■

A.2 Design of the general support kernel $\hat{H}(\omega)$

A.2.1 Example of shaping kernel using standard design method: digital First-Order Low-Pass Filter

Consider the analog first-order low-pass filter with impulse response $h_a(t)$. Its Laplace transform is:

$$\mathcal{L}\{h_a(t)\}(s) = H_a(s) = \frac{\omega_c}{s + \omega_c}, \quad s \in \mathbb{C}.$$

Applying the bilinear transformation $s \mapsto \frac{2}{T} \frac{1-z^{-1}}{1+z^{-1}}$ with sampling period T yields the z-transform:

$$H(z) = \frac{\omega_c T (1 + z^{-1})}{(2 + \omega_c T) + (-2 + \omega_c T) z^{-1}}.$$

Let $\Omega := \omega_c T$ denote the normalized cutoff frequency. Dividing numerator and denominator by $(2 + \Omega)$:

$$H(z) = \frac{\frac{\Omega}{2+\Omega}(1 + z^{-1})}{1 + \frac{-2+\Omega}{2+\Omega} z^{-1}}.$$

From $\beta := \frac{\Omega}{2+\Omega}$, we obtain $\beta(2 + \Omega) = \Omega$, hence $\Omega = 2\beta + \beta\Omega$. Rearranging:

$$\Omega(1 - \beta) = 2\beta \implies \Omega = \frac{2\beta}{1 - \beta} = 2\beta \sum_{k=0}^{\infty} \beta^k, \quad |\beta| < 1.$$

Therefore:

$$\frac{-2 + \Omega}{2 + \Omega} = -(1 - \beta) + \beta = 2\beta - 1,$$

yielding:

$$H(z) = \frac{\beta(1 + z^{-1})}{1 - (1 - 2\beta)z^{-1}} = \frac{\beta(z + 1)}{2\beta + z - 1}, \quad \text{with pole } z_p = 1 - 2\beta \quad (\text{A.1})$$

The convergence condition $|\beta| < 1$ from the geometric series ensures $|z_p| < 1$ (stability), demonstrating equivalence between algebraic convergence and z-domain stability.

A.2.2 Special decomposition cases for $\hat{H}_a(\omega)$

We consider the expression:

$$I = \oint_C \frac{1}{2} [(S_e - izS_o)H_a(\omega) + (S_e + izS_o)H_a(-\omega)] d\omega$$

where S_e is a purely even series, S_o is a purely odd series, $\omega = iz/\zeta$ (change of variables), and C is a contour in the complex ω -plane.

Case 1: Rational Functions

Consider $H_a(\omega) = \frac{P(\omega)}{Q(\omega)} = \sum_{k=1}^N \frac{R_k}{\omega - p_k}$ where p_k are the poles and R_k are the residues from partial fraction decomposition. Then:

$$\begin{aligned} I &= \frac{1}{2} \oint_C \left[(S_e - izS_o) \sum_{k=1}^N \frac{R_k}{\omega - p_k} + (S_e + izS_o) \sum_{k=1}^N \frac{R_k}{-\omega - p_k} \right] d\omega \\ &= \frac{1}{2} \sum_{k=1}^N R_k \oint_C \left[\frac{S_e - izS_o}{\omega - p_k} + \frac{S_e + izS_o}{-\omega - p_k} \right] d\omega \end{aligned}$$

For symmetric contours and poles in pairs $\pm p_k$ with equal residues:

$$I = \pi i \sum_{k: p_k \in C} R_k [2S_e + izS_o \cdot \text{sgn}(p_k)]$$

Each pole contributes a mode $e^{p_k t}$ in time domain. The contour integral selects which modes contribute based on pole locations relative to C .

Case 2: Gaussian Form

Consider $H_a(\omega) = e^{-a\omega^2}$ where $a > 0$. Since $H_a(-\omega) = H_a(\omega)$ (even function):

$$I = \frac{1}{2} \oint_C e^{-a\omega^2} [(S_e - izS_o) + (S_e + izS_o)] d\omega = S_e \oint_C e^{-a\omega^2} d\omega$$

The S_o term completely cancels. For a contour along the real axis: $I = S_e \sqrt{\frac{\pi}{a}}$. The Gaussian is the generating function for Hermite polynomials, with time-domain impulse response $h(t) = \frac{1}{\sqrt{4\pi at}} e^{-t^2/(4a)}$ (fundamental solution to the heat equation).

Case 3: Resonant Functions

Consider $H_a(\omega) = \frac{1}{\omega^2 + \omega_0^2}$. Since this is even, the S_o term vanishes: $I = S_e \oint_C \frac{d\omega}{\omega^2 + \omega_0^2}$. The function has simple poles at $\omega = \pm i\omega_0$ with residues $\pm \frac{1}{2i\omega_0}$. If C encloses only the upper half-plane pole:

$$I = \frac{\pi S_e}{\omega_0}$$

This represents a simple harmonic oscillator with time-domain response $h(t) = \frac{\sin(\omega_0 t)}{\omega_0} \Theta(t)$.

Case 4: Self-Adjoint (Real on Imaginary Axis)

If $H_a(i\theta) \in \mathbb{R}$ for all real θ , then $H_a(-\omega) = \overline{H_a(\omega)}$ (Hermitian symmetry). Writing $H_a(\omega) = H_R(\omega) + iH_I(\omega)$ with H_R even and H_I odd:

$$I = \oint_C [S_e H_R(\omega) + zS_o H_I(\omega)] d\omega + i \oint_C [S_e H_I(\omega) - zS_o H_R(\omega)] d\omega$$

If $H_a(i\theta)$ is purely real on the imaginary axis ($H_I(i\theta) = 0$) and the contour lies there, the odd component decouples completely: $I = \oint_C S_e H_R(\omega) d\omega$. This applies to all causal real-valued systems satisfying $H(s^*) = H^*(s)$, including typical low-pass, band-pass, and physically realizable linear systems.

A.3 Shannon Wavelet Derivations

Phase error for complex z . For $z = x + iy$ with $y \neq 0$:

$$e^{iz} = e^{i(x+iy)} = e^{ix-y},$$

$$\Gamma(2N+2, iz) = \Gamma(2N+2) - \gamma(2N+2, iz),$$

where $\gamma(s, w) = \int_0^w t^{s-1} e^{-t} dt$ is the lower incomplete gamma function. The imaginary part captures both:

- *Magnitude error*: from truncating the Taylor series to N terms.
- *Phase error*: from the complex exponential e^{iz} interacting with the incomplete gamma function.

Thus, for applications requiring phase accuracy (e.g., complex filters, IQ modulation), the full complex projector must be used, and the hypergeometric form is *only* a computational simplification for real-valued signals.

A.3.1 Frequency Response Contribution $\Phi_n(\omega)$

The term-specific frequency contribution $\Phi_n(\omega)$ in (??) arises from the Fourier transform of the n -th basis monomial x^{2n} .

For a polynomial projector:

$$\tilde{\mathbb{P}}_N[f](x) = \sum_{n=0}^N \tilde{c}_n x^{2n},$$

the frequency response is:

$$\widehat{\tilde{\mathbb{P}}_N[f]}(\omega) = \sum_{n=0}^N \tilde{c}_n \cdot \mathcal{F}\{x^{2n}\}(\omega),$$

where $\mathcal{F}\{x^{2n}\}(\omega) = i^{2n} \cdot 2\pi \delta^{(2n)}(\omega)$ is the $(2n)$ -th distributional derivative of the Dirac delta.

For practical filter design, we regularize by restricting to a compact domain $[0, x_{\max}]$:

$$\Phi_n(\omega) = \int_0^{x_{\max}} x^{2n} e^{-i\omega x} dx = \frac{\partial^{2n}}{\partial (i\omega)^{2n}} \left[\frac{e^{-i\omega x_{\max}} - 1}{-i\omega} \right].$$

Using integration by parts $2n$ times:

$$\Phi_n(\omega) = \sum_{k=0}^{2n} \binom{2n}{k} \frac{(2n-k)!}{\omega^{2n-k+1}} [(-i)^k e^{-i\omega x_{\max}} x_{\max}^k - (-i)^k \delta_{k,0}],$$

which exhibits $O(\omega^{-(2n+1)})$ decay, ensuring well-defined frequency localization.

A.3.2 Greedy Bit Allocation with Truncation-Quantization Trade-off

The complete optimization problem is:

$$\min_{\substack{N \in \mathbb{N}_0 \\ \mathbf{m} \in \mathbb{N}_0^{N+1}}} [\mathcal{E}_{\text{trunc}}(N) + \mathcal{E}_{\text{quant}}(N, \mathbf{m})] \quad \text{s.t.} \quad \sum_{n=0}^N m_n \leq B_{\text{total}}. \quad (\text{P})$$

where:

$$\mathcal{E}_{\text{trunc}}(N) = \left| \int_0^{x_{\max}} R_N(z) \overline{R(z)} dz \right|,$$

$$\mathcal{E}_{\text{quant}}(N, \mathbf{m}) = \sum_{n=0}^N \frac{S_n}{2^{m_n+1}}.$$

Algorithm.

1. **Outer loop (choose N):** For $N = 0, 1, 2, \dots, N_{\max}$:

- (a) Compute truncation error $\mathcal{E}_{\text{trunc}}(N)$.
- (b) Compute sensitivities S_n for $n = 0, \dots, N$.
- (c) Solve the inner bit allocation problem for this N .

2. **Inner loop (greedy bit allocation for fixed N):**

- (a) Initialize $m_n = 1$ for all $n \in \{0, \dots, N\}$.
- (b) Set $B_{\text{remaining}} = B_{\text{total}} - (N + 1)$.
- (c) While $B_{\text{remaining}} > 0$:
 - Compute marginal error reduction:

$$\Delta_n = S_n \left(\frac{1}{2^{m_n+1}} - \frac{1}{2^{m_n+2}} \right) = \frac{S_n}{2^{m_n+2}}.$$
 - Find $n^* = \arg \max_n \Delta_n$.
 - Update: $m_{n^*} \leftarrow m_{n^*} + 1$, $B_{\text{remaining}} \leftarrow B_{\text{remaining}} - 1$.
- (d) Return $\mathcal{E}_{\text{quant}}(N, \mathbf{m})$.

3. **Select optimal N^* :**

$$N^* = \arg \min_N [\mathcal{E}_{\text{trunc}}(N) + \mathcal{E}_{\text{quant}}(N, \mathbf{m}^*(N))].$$

Complexity. The outer loop runs $O(N_{\max})$ times. For each N , the inner greedy algorithm runs $O(B_{\text{total}} \cdot N)$ iterations. Each iteration requires $O(N)$ comparisons to find $\arg \max$. Total: $O(N_{\max} \cdot B_{\text{total}} \cdot N^2)$, which can be optimized to $O(N_{\max} \cdot B_{\text{total}} \cdot N \log N)$ using a priority queue.

A.3.3 Residue-Based Decomposition for Hardware Kernels

Consider a rational kernel:

$$H(\omega) = \frac{P(\omega)}{Q(\omega)}, \quad Q(\omega) = \prod_{p=1}^M (\omega - \omega_p)^{r_p},$$

where $\omega_p \in \mathbb{C}$ are the poles with multiplicities r_p .

The wavelet-projected integral:

$$I_j(t) = \int_{2^j\pi}^{2^{j+1}\pi} H(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t} d\omega$$

can be evaluated by:

1. Extend the integration domain to a contour Γ_j enclosing $[2^j\pi, 2^{j+1}\pi]$ and all poles ω_p in this range.
2. Apply the residue theorem:

$$I_j(t) = 2\pi i \sum_{\omega_p \in [2^j\pi, 2^{j+1}\pi]} \text{Res}_{\omega=\omega_p} \left[H(\omega) \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t} \right].$$

t

For a simple pole ω_p (i.e., $r_p = 1$):

$$\text{Res}_{\omega=\omega_p} \left[\frac{P(\omega)}{Q(\omega)} F(\omega) \right] = \frac{P(\omega_p)}{Q'(\omega_p)} F(\omega_p),$$

where $F(\omega) = \tilde{\Phi}_N^{(\mathbf{m})}((i\omega \cdot 2^{-j}t)^2) e^{i\omega t}$.

Expanding $\tilde{\Phi}_N^{(\mathbf{m})}(w) = \sum_{n=0}^N \frac{a_n}{2^{m_n}} \frac{w^n}{(2n)!}$:

$$\begin{aligned} F(\omega_p) &= \sum_{n=0}^N \frac{a_n}{2^{m_n}(2n)!} [(i\omega_p \cdot 2^{-j}t)^2]^n e^{i\omega_p t} \\ &= e^{i\omega_p t} \sum_{n=0}^N \frac{a_n(-1)^n}{2^{m_n}(2n)!} \left(\frac{\omega_p t}{2^j} \right)^{2n}. \end{aligned}$$

Thus:

$$I_j(t) = 2\pi i \sum_{\omega_p} \underbrace{\frac{P(\omega_p)}{Q'(\omega_p)}}_{\text{constant}} \cdot e^{i\omega_p t} \cdot \underbrace{\sum_{n=0}^N \frac{a_n(-1)^n}{2^{m_n}(2n)!} \left(\frac{\omega_p t}{2^j} \right)^{2n}}_{\text{dyadic polynomial in } t}.$$

A.3.4 Signal Synthesis Capabilities of Shannon Wavelet Decomposition

What Can Be Synthesized

The Shannon wavelet framework with n scales starting at resolution j_0 can exactly represent any bandlimited signal with Fourier support in $[-2^{j_0+n}\pi, 2^{j_0+n}\pi]$. This is the Shannon-Nyquist sampling theorem extended to wavelet bases [Daubechies1992].

Fundamental Limitation: Bandlimited Signals Only. Unlike time-domain polynomial approximation or windowed sinc functions, the Shannon wavelet *cannot* represent:

- Sharp transients (step functions, clicks, impulses)
- Signals with infinite bandwidth
- Non-smooth waveforms (square waves, sawtooth without filtering)

However, *any practical audio or RF signal* is effectively bandlimited by:

- Physical limits (speaker/microphone bandwidth, antenna bandwidth)
- Anti-aliasing filters before sampling
- Nyquist limit of the sampling rate

Concrete Synthesis Examples

Example 1: Additive synthesis (organ, FM synthesis). Synthesize a tone with 8 harmonics at frequency f_0 :

$$s(t) = \sum_{k=1}^8 A_k \sin(2\pi k f_0 t + \phi_k).$$

For $f_0 = 440$ Hz (A4) with 8 harmonics, highest frequency is $8 \times 440 = 3520$ Hz.
Shannon wavelet representation:

- Choose j_0 such that $2^{j_0}\pi/T_s \approx 220$ Hz (below fundamental).
- 3-scale decomposition covers up to $8 \times 2^{j_0}\pi/T_s \approx 7$ kHz.
- Each harmonic k lives in scale $j = j_0 + \lfloor \log_2(k) \rfloor$.
- Total coefficients: ~ 16 per harmonic $\times 8$ harmonics = 128 coefficients.

Computational savings vs. time-domain:

- Time-domain sinc interpolation: requires $O(L)$ samples over support $L \approx$ several periods.
- Shannon wavelet: $O(8)$ wavelets (one per harmonic) with compact frequency support.
- **Advantage:** Adding/removing harmonics is trivial (set coefficient to zero), enabling real-time additive synthesis with frequency-specific envelope control.

Example 2: Subtractive synthesis (sawtooth $\rightarrow$ lowpass filter). Start with a bandlimited sawtooth (infinite harmonics, amplitude $\propto 1/k$):

$$\text{sawtooth}(t) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \sin(2\pi k f_0 t).$$

Bandlimit to 32 harmonics (typical for audio):

- 5-scale Shannon decomposition ($j_0 = 0$ to $j_0 + 4$) covers 32 octaves.
- Apply a 2-pole lowpass filter $H(\omega) = \frac{1}{(1+i\omega/\omega_c)^2}$ with cutoff ω_c .
- In the wavelet domain, this is a *diagonal scaling* of coefficients:

$$d_{j,k}^{\text{filtered}} = H(2^{j+1/2}\pi) \cdot d_{j,k}^{\text{original}}.$$

- **Result:** Filter is a single multiplication per scale, not a convolution.

Hardware advantage:

- Time-domain FIR: $O(N_{\text{taps}})$ MACs per sample (e.g., 128-tap filter = 128 MACs).
- Shannon wavelet: 5 complex multiplications (one per scale), then reconstruct.
- For real-time filter sweeps (cutoff $\omega_c(t)$ varies), only the scale weights change—no filter redesign needed.

Example 3: Phase vocoder / time-stretching. Time-stretch a signal by factor α without changing pitch:

- Decompose into Shannon wavelets $\{d_{j,k}\}$.
- Modify temporal positions: $\psi_{j,k}(t) \rightarrow \psi_{j,k}(t/\alpha - k)$ (dilation in time).
- Frequency content (pitch) is unchanged because wavelets are shift-invariant in frequency.

Traditional phase vocoder:

- STFT (Short-Time Fourier Transform): window size trades off time vs. frequency resolution.
- Shannon wavelets: *native* multi-resolution—low frequencies have long windows, high frequencies have short windows automatically.

Example 4: Parametric EQ (audio mixing). Boost/cut at specific frequency bands (e.g., bass boost at 100 Hz, treble cut at 8 kHz):

- Shannon wavelet scale j corresponds to octave band $[2^j\pi, 2^{j+1}\pi]$.
- To boost scale j by +6 dB: multiply all $d_{j,k}$ by 2.
- To apply a shelf filter: ramp the gain across scales.

Advantage over IIR/FIR EQ:

- IIR (biquad cascade): 2nd-order sections with stability concerns, phase distortion.
- FIR (Parks-McClellan): linear phase but high order ($\sim 100+$ taps for sharp transitions).
- Shannon wavelet: natural octave bands, perfect reconstruction, no phase distortion within each band.

Quantization Error in Synthesis Context

For a signal with n scales, total synthesis error is:

$$\mathcal{E}_{\text{total}} = \underbrace{\sum_k |c_{j_0,k}|^2 \mathcal{E}_{\text{quant}}^{(j_0)}}_{\text{base scale}} + \underbrace{\sum_{j=j_0}^{j_0+n-1} \sum_k |d_{j,k}|^2 \mathcal{E}_{\text{quant}}^{(j)}}_{\text{detail scales}},$$

where $\mathcal{E}_{\text{quant}}^{(j)} = \sum_m S_{\psi,m}^{(j)} / 2^{m_{\psi,m}^{(j)}+1}$ from Section 4.4.6.

Key insight for synthesis:

- High frequencies (large j): typically have *small* coefficients $|d_{j,k}|$ (natural signal decay).
- Low frequencies (small j): have *large* coefficients but fewer wavelets needed.
- The bit allocation vector $\mathbf{m}$ can assign *fewer bits to high-frequency scales* where $|d_{j,k}| \ll 1$, saving hardware resources without audible/visible degradation.

Practical synthesis example:

- 5-scale decomposition for audio (20 Hz – 20 kHz).
- Scale 0 (j_0): 20–40 Hz (bass), $m_n = 16$ bits (high precision for low distortion).
- Scale 1: 40–80 Hz, $m_n = 14$ bits.
- Scale 2: 80–160 Hz, $m_n = 12$ bits.
- Scale 3: 160–320 Hz, $m_n = 10$ bits.

- Scale 4: 320–640 Hz and above, $m_n = 8$ bits (high frequencies masked by louder low frequencies).
- **Total bits:** $16 + 14 + 12 + 10 + 8 = 60$ bits distributed across 5 scales.
- **Uniform quantization:** would use $12 \times 5 = 60$ bits but with *worse* perceptual quality (over-quantizes bass, under-quantizes treble relative to perception).

Comparison: Shannon Wavelets vs. Alternatives

Method	Freq. Selectivity	Time Localization	Hardware Cost	Phase
Shannon Wavelet	Excellent	Poor	Low (dyadic ops)	Linear
Haar Wavelet	Poor	Excellent	Very Low	Linear
Daubechies Wavelet	Good	Good	Medium	Nonlinear
FFT/STFT	Excellent	Poor	Medium	Complex
FIR Filter	Good	Medium	High (N taps)	Linear
IIR Filter	Good	Good	Low	Nonlinear
Polynomial (dyadic)	N/A	Excellent	Very Low	N/A

Table A.1: Comparison of signal synthesis methods for bandlimited signals.

When to use Shannon wavelets:

- **Frequency-domain processing** with minimal computational cost (filtering, EQ, pitch shift).
- **Multi-rate systems** where different frequency bands are processed at different rates (e.g., low frequencies at 1 kHz, high frequencies at 48 kHz).
- **Adaptive quantization** where bit budget must be allocated across frequency bands based on sensitivity.
- **Perfect reconstruction** is required (lossless analysis-synthesis).

When NOT to use Shannon wavelets:

- **Transient-heavy signals** (percussion, clicks) — use Daubechies or Haar for better time localization.
- **Wideband noise** — time-domain polynomial or direct sampling is simpler.
- **Ultra-low latency** — Shannon wavelets have infinite time support (though truncated in practice).

A.4 Hardware design choices

A.4.1 Kernel design

Hardware implementation of projector

- Precompute constants $C_p = 2\pi i \frac{P(\omega_p)}{Q'(\omega_p)}$ (stored in BRAM).
- Oscillator: Generate $e^{i\omega_p t}$ using CORDIC in rotation mode (1 DSP per pole).
- Polynomial: Evaluate the dyadic polynomial using Horner's method (N DSPs or 1 DSP with N -cycle latency).
- Accumulate: Sum over all poles p (1 DSP).

For M poles and N terms: $M(N + 1) + 1$ DSP slices (parallel) or $N + 2$ DSP slices (time-multiplexed over M poles).

This architecture naturally supports frequency-dependent bit allocation: high-sensitivity poles can use larger m_n for their polynomial coefficients, while low-sensitivity poles use reduced precision, all determined by the sensitivity matrix $\mathbf{S}_{\text{wavelet}}$.

A.4.2 Fabric design

A.4.3 Synthesis Quality Metrics

For the quantized Shannon wavelet synthesis $\tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} (t)$ from (3.59):

Signal-to-Quantization-Noise Ratio (SQNR):

$$\text{SQNR} = 10 \log_{10} \frac{\|f\|_{L^2}^2}{\|f - \tilde{f}^{(N,M,\mathbf{m},\mathbf{m}')} \|_{L^2}^2} \quad (\text{dB}).$$

For the greedy bit allocation (Appendix A.3) with total budget $B_{\text{total}} = 60$ bits across 5 scales:

- Typical SQNR: 70–90 dB (depending on signal).
- Comparable to 12-bit uniform quantization on the time-domain signal but with *perceptually optimized* distribution.

Total Harmonic Distortion (THD): For synthesis of harmonic signals (musical tones):

$$\text{THD} = \frac{\sqrt{\sum_{k=2}^{\infty} P_k}}{P_1} \times 100\%,$$

where P_k is the power in the k -th harmonic.

With $N = 4$ polynomial terms per wavelet and $m_n \in [8, 16]$ bits:

- THD < 0.01% for fundamental + first 8 harmonics (professional audio quality).
- THD < 0.1% for 32 harmonics (CD quality).

Bandwidth efficiency:

- Shannon wavelets are *optimal* for bandlimited signals in the sense that the number of coefficients $\{c_{j_0,k}, d_{j,k}\}$ equals the number of degrees of freedom (Shannon's sampling theorem).
- For a signal bandlimited to B Hz sampled at $f_s = 2B$, the Shannon wavelet decomposition requires $\sim f_s$ coefficients per second, which is *information-theoretically optimal*.

A.4.4 Hardware Resource Estimates for Shannon Wavelet Implementation

FPGA resource model. Consider a Xilinx Virtex-7 FPGA with the following primitives:

- DSP48E1 slices: 18×25 -bit multipliers, 48-bit accumulators.
- Block RAM (BRAM): 36 Kb dual-port memory blocks.
- Look-up tables (LUTs): 6-input logic functions.

Scaling function computation. For $\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t)$:

$$\tilde{\phi}_{j_0,k}^{(N,\mathbf{m})}(t) = 2^{j_0/2} \sum_{n=0}^N \frac{a_n}{2^{m_n}} [\pi(2^{j_0}t - k)]^{2^n}.$$

Resources per basis function:

- $(N+1)$ coefficient storage: $(N+1) \times \max_n(m_n + \lceil \log_2 |a_n| \rceil)$ bits in BRAM.
- Power computation u^{2^n} for $u = \pi(2^{j_0}t - k)$: $\lceil \log_2(N+1) \rceil$ DSP slices (using Horner's method or parallel squaring tree).
- Multiply-accumulate: $(N+1)$ DSP slices or time-multiplexed over $\lceil (N+1)/P \rceil$ cycles for P parallel DSPs.
- Dyadic scaling $\frac{a_n}{2^{m_n}}$: implemented as barrel shifter using $O(m_n)$ LUTs (no DSP needed).

Example ($N=4$, $m_n \in [8, 16]$):

- Coefficients: 5×24 bits = 120 bits ≈ 0.003 BRAM.
- Power computation: 3 DSP slices (parallel).
- MAC: 5 DSP slices (fully parallel) or 1 DSP slice (5-cycle latency).
- Total per basis: 8 DSP, 120-bit BRAM, ~ 50 LUTs.

For a 3-scale wavelet decomposition (j_0, j_0+1, j_0+2) with $K = 32$ spatial positions per scale:

- Total DSP: $8 \times 32 \times 3 = 768$ DSPs (parallel) or $8 \times 3 = 24$ DSPs (time-multiplexed over 32 cycles).
- Total BRAM: $120 \times 32 \times 3 = 11520$ bits ≈ 0.32 BRAM blocks.
- Throughput: 1 sample per cycle (parallel) or 1 sample per 32 cycles (multiplexed).

Virtex-7 XC7VX485T has 2800 DSP slices and 1030 BRAM blocks, so even the fully parallel implementation uses $\sim 27\%$ DSP and $<1\%$ BRAM.

A.5 Code snippets or heuristics

A.5.1 'compact' sinc implementation

```
def compact_sinc(x: np.ndarray, dtype: np.dtype) -> np.ndarray[np.float64]:
    x = np.asarray(x, dtype=dtype)
    result = np.zeros_like(x)
    x_nz = x[x != 0]
    if x_nz.size > 0:
        abs_x_nz = np.abs(x_nz)
        lobe_number = np.floor(abs_x_nz)
        lobe_index = abs_x_nz - lobe_number
        envelope = 1.0 / (np.pi * np.abs(x_nz))
        sign = (-1) ** lobe_number
        canonical_values = _canonical_sinc_lobe_lookup(lobe_index)
        result[x != 0] = sign * envelope * canonical_values
    result[x == 0] = 1.0
    return result

def _canonical_sinc_lobe_lookup(lobe_index: np.ndarray) -> np.ndarray[np.float64]:
    # Convert lobe_index from [0, 2) to [0, pi) for sinc calculation
    t = lobe_index * np.pi
    result = np.where(t == 0, 1.0, np.sin(t))
    return result
```

A.5.2 comparison of cos power series in c

```
static inline double cse_Polynomial_Cos_Deg6(float x) {
    double xd = (double)x;
    double z = xd * xd;
    double t = fma(-1.3888888888888889e-3, z, 4.1666666666666664e-2);
    t = fma(t, z, -5.0e-1);
```

```

    t = fma(t, z, 1.0);
    return t;
}

static inline double cse_Polynomial_Cos_Deg12(float x) {
    double xd = (double)x;
    double z = xd * xd;
    double t = fma(2.08767569878681e-09, z, -2.7557319223985888e-07);
    t = fma(t, z, 2.4801587301587302e-05);
    t = fma(t, z, -1.3888888888888889e-03);
    t = fma(t, z, 4.1666666666666664e-02);
    t = fma(t, z, -5.0e-1);
    t = fma(t, z, 1.0);
    return t;
}

static inline double cse_Dyadic_Polynomial_Cos_Deg12(float x) {
    double xd = (double)x;
    double z = xd * xd;
    double t = fma(-2.0563602447509766e-05, z, -1.3113021850585938e-06);
    t = fma(t, z, 2.3901462554931641e-05);
    t = fma(t, z, -0.0013893246650695801);
    t = fma(t, z, 0.041666388511657715);
    t = fma(t, z, -0.500000017881393433);
    t = fma(t, z, 0.99999988079071045);
    return t;
}

```

A.5.3 introducing concurrency into power series

```

def _convert_lines_to_simd(self, lines: list[str], target: SIMD = SIMD.AVX512)
    """Annotate per-stage lines with AVX-512 FMAs packed by common

    Looks for lines of form 'lhs = fma(a, b, c)' and groups those s
    same 'b' into packs of up to 8 lanes (double). Emits a single f
    comment per pack, then preserves all original scalar lines and
    """

    if target is not SIMD.AVX512:
        raise NotImplementedError('Only AVX512 is implemented')

    assign_pat = re.compile(r'^\s*([A-Za-z_]\w*)\s*=\s*(.+)$')
    fma_pat = re.compile(r'^fma\(\s*(?P<a>.+?)\s*,\s*(?P<b>.+?)\s*,

```

```

# Parse entries and track original indices
entries: list[tuple[int, str, str, str, str]] = []
# (idx, lhs, a, b, c)
for idx, ln in enumerate(lines):
    m = assign_pat.match(ln)
    if not m:
        continue
    lhs, rhs = m.group(1), m.group(2).strip()
    fm = fma_pat.match(rhs)
    if fm:
        a = fm.group('a').strip()
        b = fm.group('b').strip()
        c = fm.group('c').strip()
        entries.append((idx, lhs, a, b, c))

# Group by common b operand
groups: dict[str, list[tuple[int, str, str, str]]] = {}
for idx, lhs, a, b, c in entries:
    groups.setdefault(b, []).append((idx, lhs, a, c))

# Build code to insert at the earliest line index of each group
inserts_at: dict[int, list[str]] = {}
drop_indices: set[int] = set()

def chunk(seq, n):
    for i in range(0, len(seq), n):
        yield seq[i:i + n]

def pad_eight(values: list[str]) -> list[str]:
    v = list(values)
    while len(v) < 8:
        v.append('0.0')
    return v[:8]

# Use a global, monotonically-increasing id to ensure unique symbol
for common, group in groups.items():
    if len(group) < 2:
        continue # skip singletons

# Sort by original order for stable packing and mark to drop
group_sorted = sorted(group, key=lambda t: t[0])
first_idx = group_sorted[0][0]
lines_for_group: list[str] = []

for ch in chunk(group_sorted, 8):
    ls = [lhs for (idx, lhs, _, _) in ch]

```



```

as_ = pad_eight([a for (_, _, a, _) in ch])
cs_ = pad_eight([c for (_, _, _, c) in ch])
lanes = ', '.join(ls)
lines_for_group.append(f"// avx512 fmaddd pack: common={lanes}")
pack_id = self._simd_gencounter
self._simd_gencounter += 1
lines_for_group.append(f"--m512d __simd_va{pack_id} = {as_}")
lines_for_group.append(f"--m512d __simd_vc{pack_id} = {cs_}")
lines_for_group.append(f"--m512d __simd_vx{pack_id} = {ls}")
lines_for_group.append(f"--m512d __simd_vout{pack_id} = {ls}")
lines_for_group.append(f"double __simd_pack{pack_id}[8] = {lanes}")
lines_for_group.append(f"--mm512_storeu_pd(__simd_pack{pack_id}, {lanes})")
for lane, lhs in enumerate(ls):
    lines_for_group.append(f"{lhs} = __simd_pack{pack_id}[{lane}]")

inserts_at[first_idx] = lines_for_group
# Mark all indices in this group to drop (they are replaced)
for idx, *_ in group:
    drop_indices.add(idx)

# Emit original lines with inserts placed at calculated positions
out: list[str] = []
for idx, ln in enumerate(lines):
    if idx in inserts_at:
        out.extend(inserts_at[idx])
    if idx in drop_indices:
        continue
    out.append(ln)
return out

```
